

计算机组成原理

王鸿鹏

计算机科学与技术学院

wanghp@hit.edu.cn

第六章 存储器

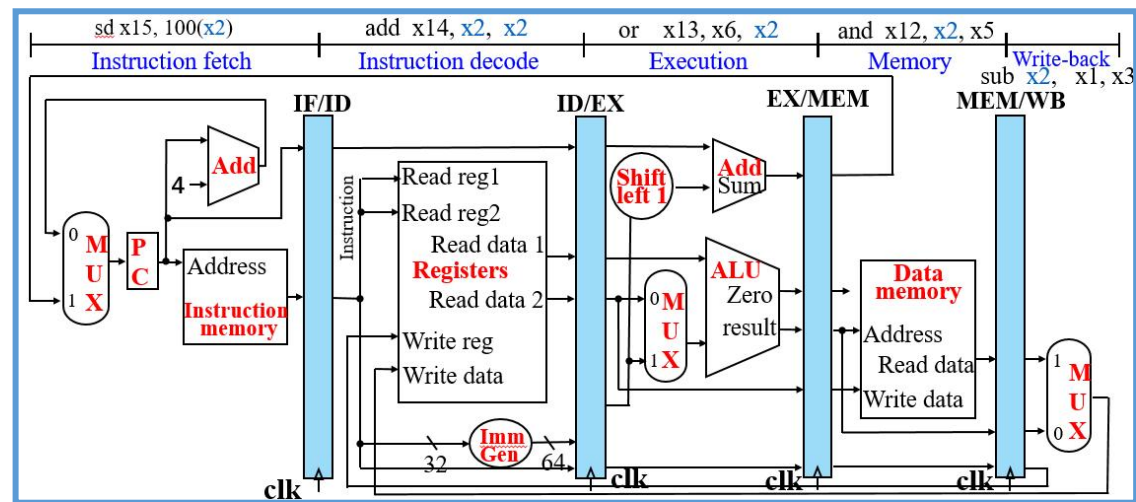
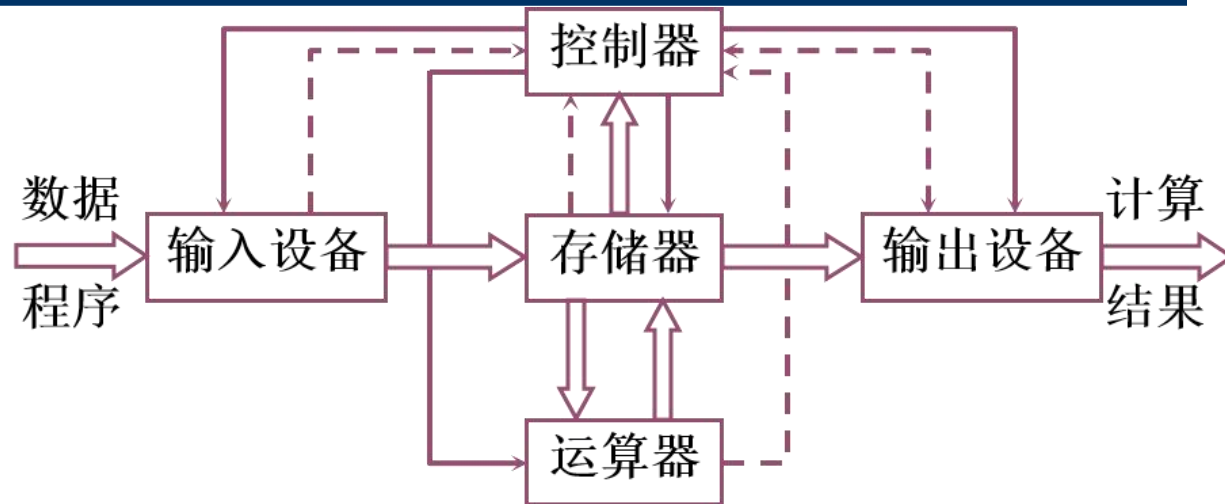
1 存储器概述

2 主存储器

3 高速缓冲存储器

4 虚拟存储

5 辅助存储器



存储器分类——按存储介质分类

1) 半导体存储器	TTL、MOS	易失
2) 磁表面存储器	磁头、载磁体	} 非易失
3) 磁芯存储器	硬磁材料、环状元件	
4) 光盘存储器	激光、磁光材料	

易失性存储器（volatile memory）：在系统停止供电的时候数据丢失，例如内存，cache等。

非易失性存储（NVM: Non-Volatile Memory）：掉电后，数据不丢失的存储器，例如磁盘，固态硬盘等。

存储器按存取方式分类

1) 随机访问存储器——存取时间与物理地址无关

随机存储器: 在程序的执行过程中 可 读 可 写

只读存储器: 在程序的执行过程中 只 读

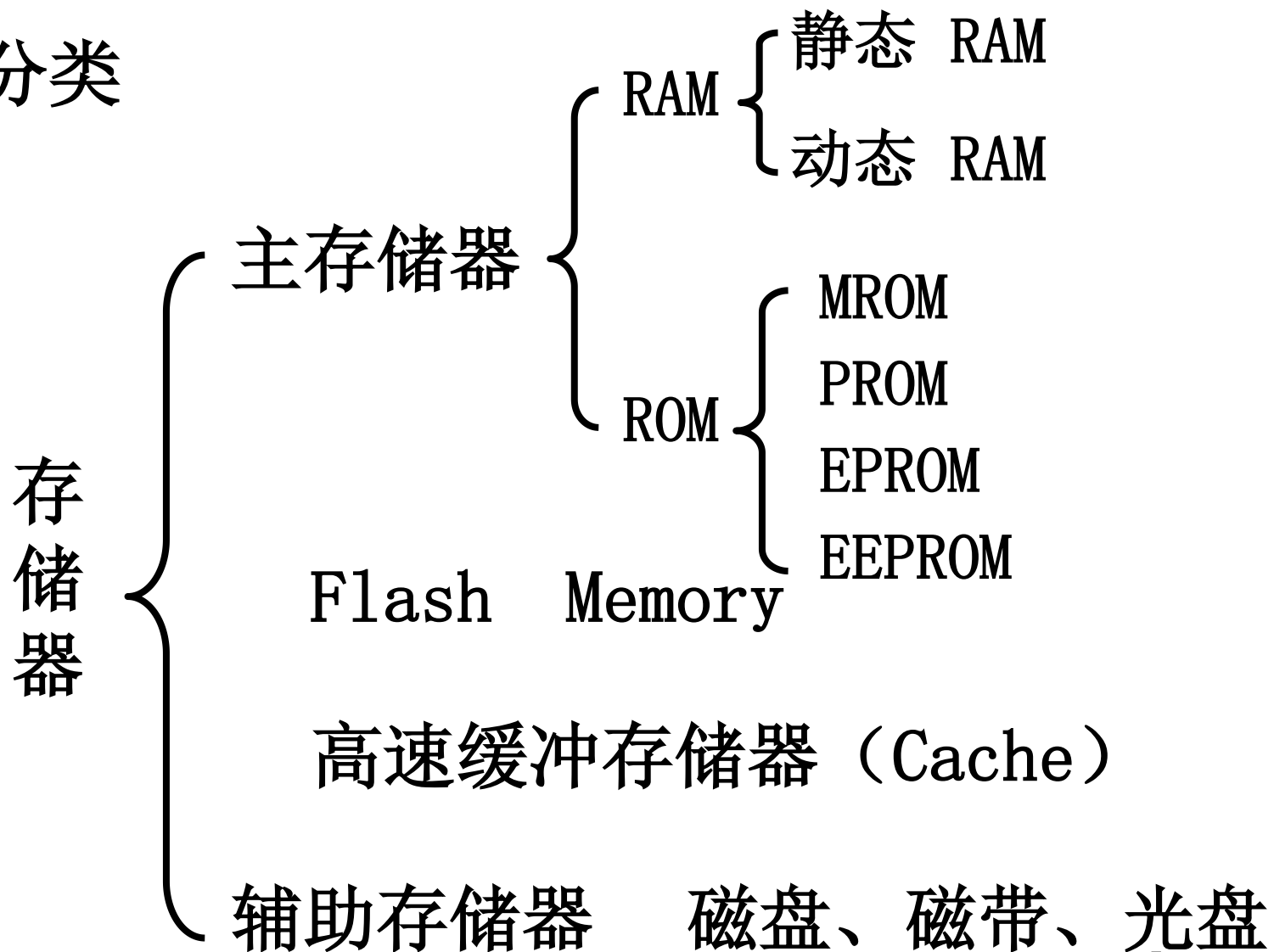
2) 串行访问存储器——存取时间与物理地址有关

顺序存取存储器（磁带）

直接存取存储器（磁盘）——选取信息所在区域（随机存取特性），然后顺序存取（顺序存取特性）

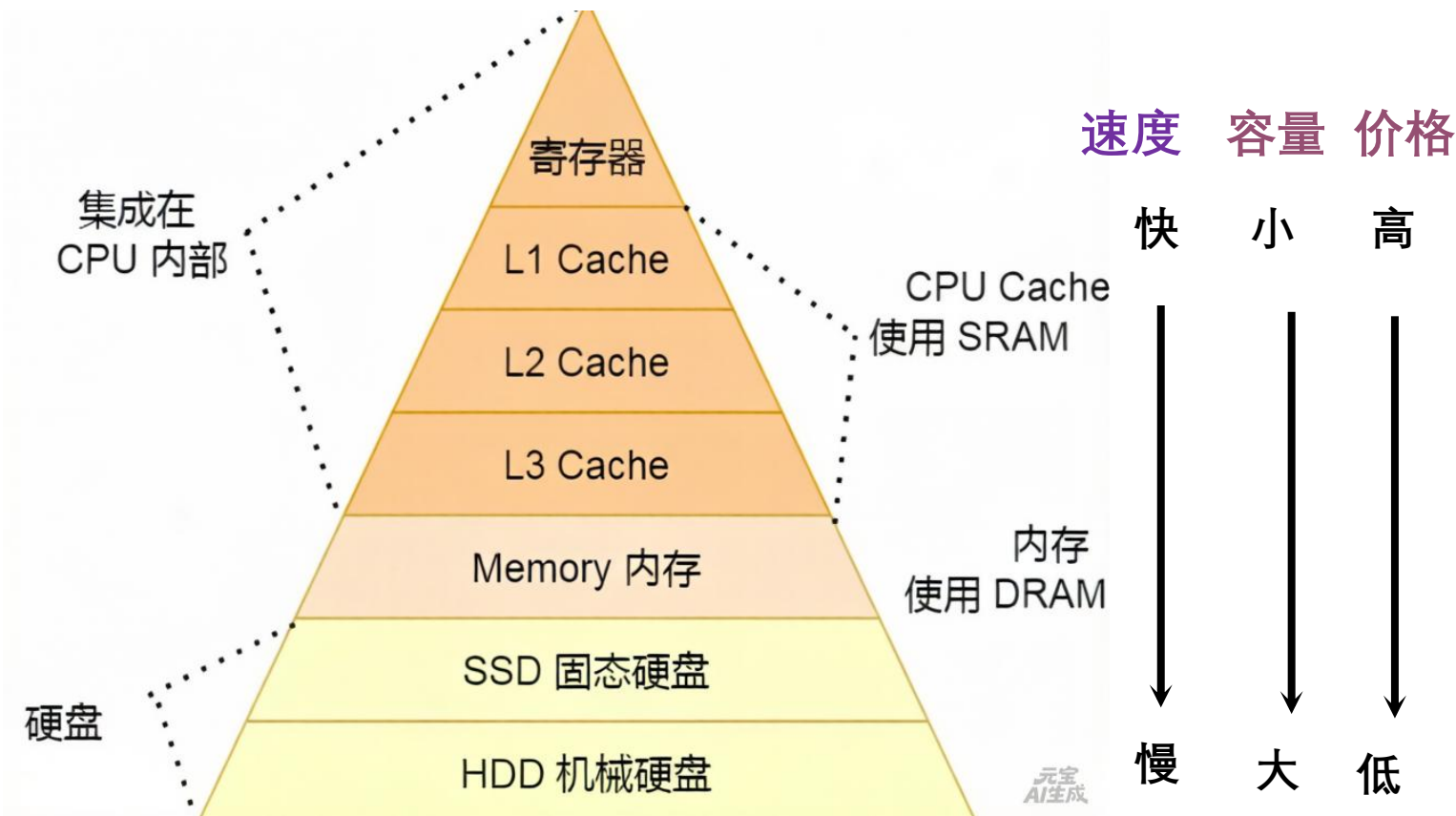
存储器按功能分类

按在计算机中的作用分类



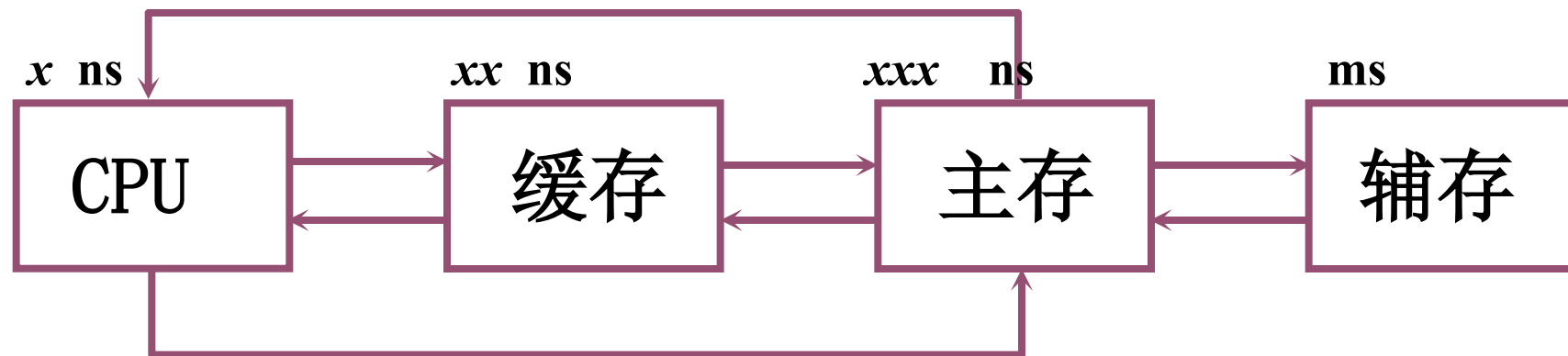
存储器的层次结构

存储器三个主要特性的关系



存储器	硬件介质	单位成本 (美元/MB)	随机访问延时
L1 Cache	SRAM	7	1ns
L2 Cache	SRAM	7	4ns
Memory	DRAM	0.015	100ns
Disk	SSD (NAND)	0.0004	150μs
Disk	HDD	0.00004	10ms

缓存—主存层次和主存—辅存层次



(速度) (容量)
缓存—主存 主存—辅存



主存储器	虚拟存储
实地址	虚地址
物理地址	逻辑地址

程序运行的局部性原理

- 在短**时间**内，最近被访问过的程序和数据很可能再次被访问
- 在存储**空间**上，这些被访问的程序和数据往往集中在一小片存储区
- 在访问**顺序**上，指令顺序执行比分支执行的可能性大(大约**5:1**)

第六章 存储器

6.1 存储器概述

6.2 主存储器

6.3 高速缓冲存储器

6.4 虚拟存储

6.5 辅助存储器

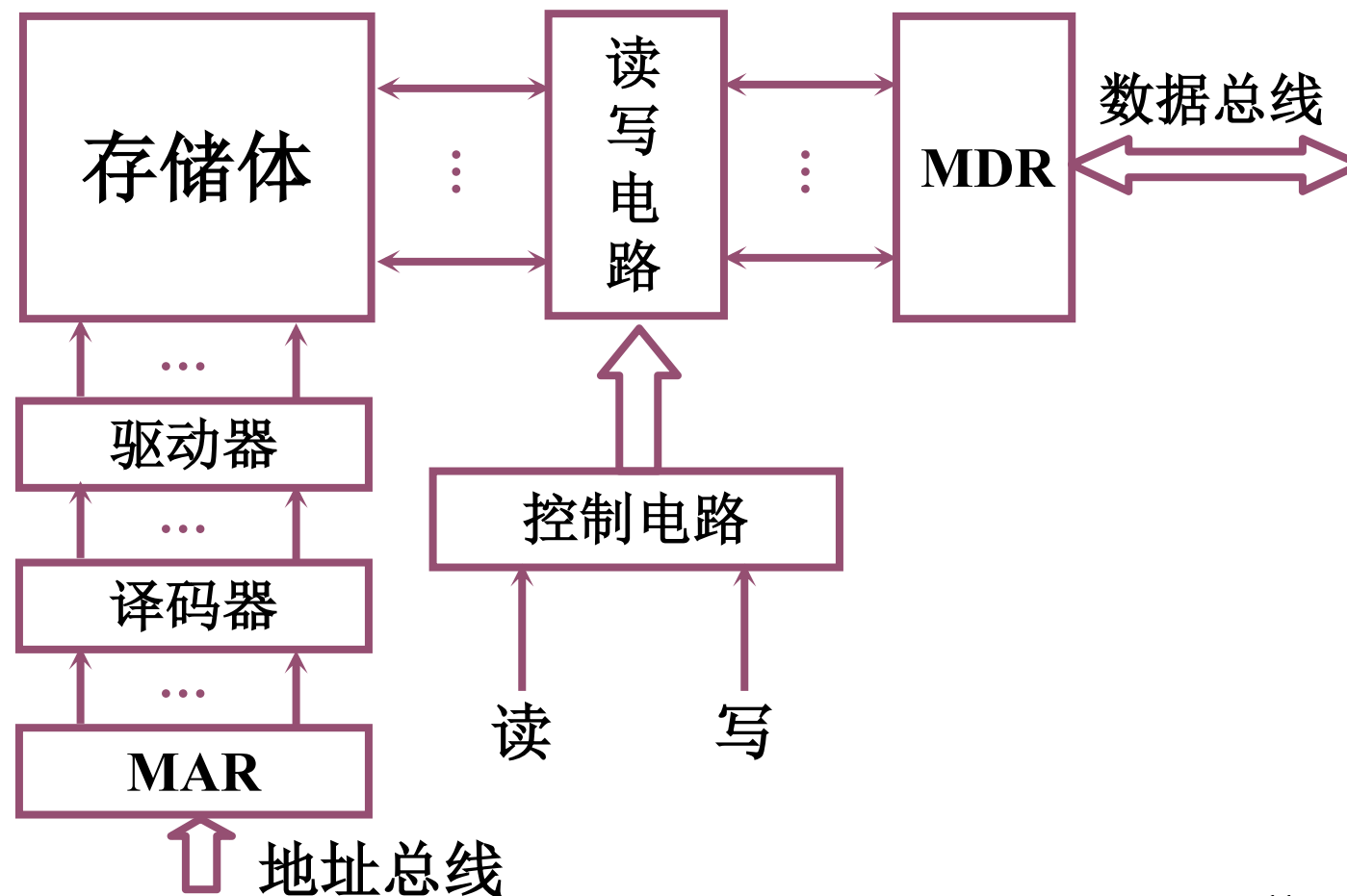
主存储器

1. 概述
2. 半导体存储芯片简介
3. 随机存储器（RAM: Random Access Memory）
4. 只读存储器（ROM: Read Only Memory）
5. 存储器与CPU的连接
6. 存储器校验
7. 存储器的加速

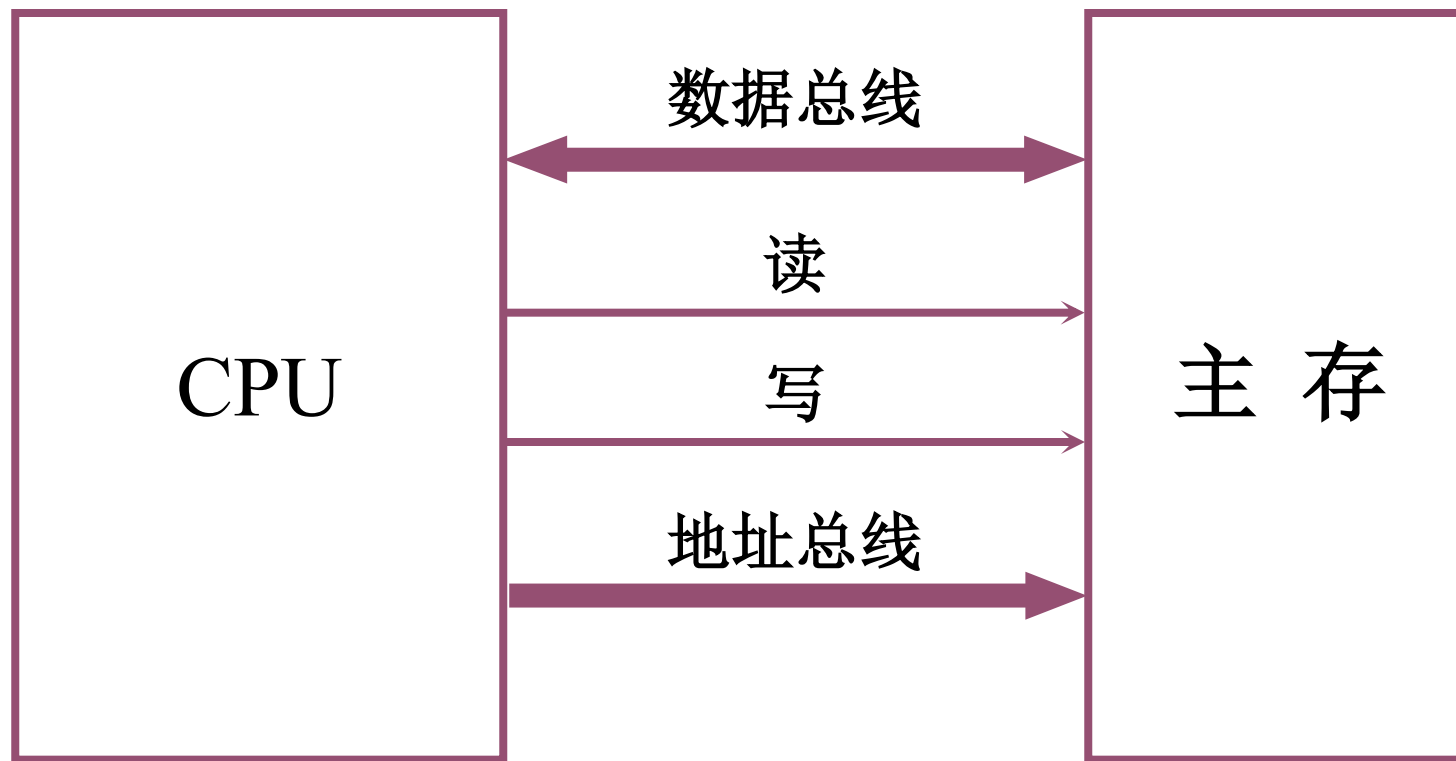
主存储器——主存的基本组成

MDR:Memory **D**ata **R**egisiter

MAR:Memory **A**ddress **R**egisiter



主存储器—主存和CPU的联系



主存储器——主存中存储单元地址的分配

在一个（双）字内部的字节顺序问题

高 低

例如：设地址addr存储的字为 0x89ABCDEF，addr，
addr+1， addr+2， addr+3四个字节存放的分别是什么数据？

地址

addr+3 addr+2 addr+1 addr

大端(字地址为高字节地址)	EF	CD	AB	89
小端(字地址为低字节地址)	89	AB	CD	EF

大端(MIPS, IBM360...)

小端(Intel 80x86, RISC V...)

主存储器—主存中存储单元地址的分配

如 16 MB (2^{27} 位) 的存储器

寻址范围

容量

按 字节 寻址

$$2^{24} = 16 \text{ M}$$

$$2^{24} \times 2^3 = 2^{27} \text{ 位}$$

按 字 (16位) 寻址

$$2^{23} = 8 \text{ M}$$

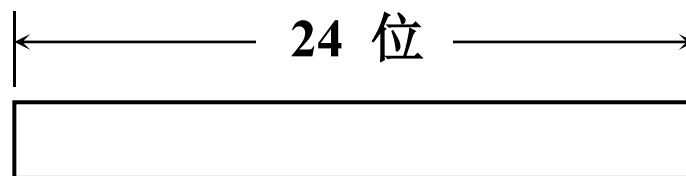
$$2^{23} \times 2^4 = 2^{27} \text{ 位}$$

按 字 (32位) 寻址

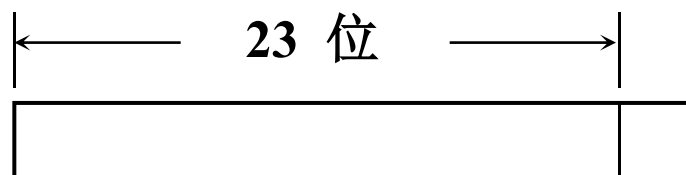
$$2^{22} = 4 \text{ M}$$

$$2^{22} \times 2^5 = 2^{27} \text{ 位}$$

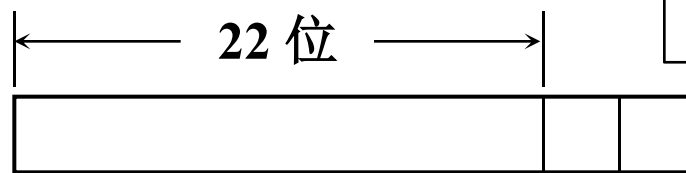
字节 寻址



字 (16位) 寻址



字 (32位) 寻址



字节地址

字节地址

存储器的常用技术指标

(1) 存储容量: 存放二进制数的总位数

(2) 存储速度

存取时间 存储器的 访问时间 (读出时间、写入时间)

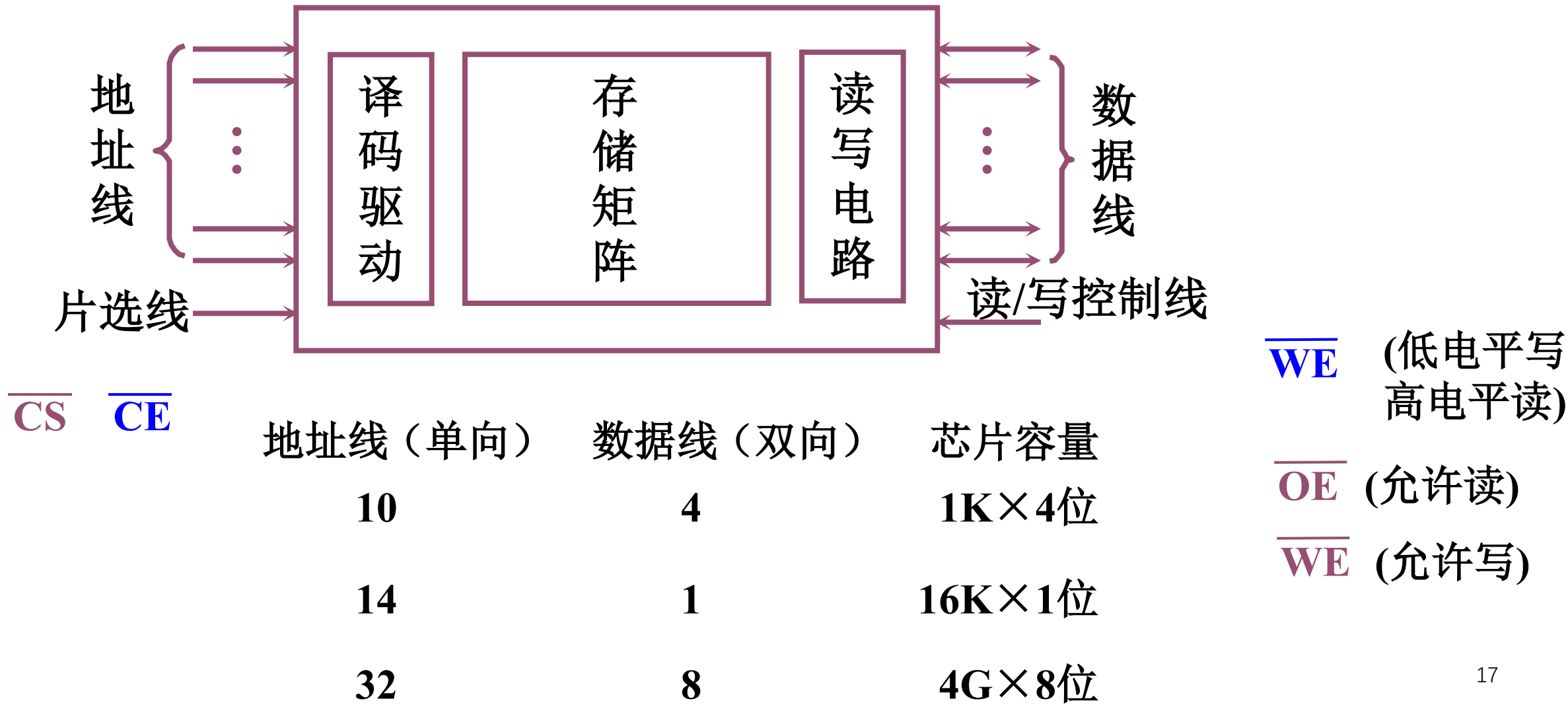
存取周期 连续两次独立的存储器操作
(读或写) 所需的 最小间隔时间 (读周期/写周期)

(3) 存储器的带宽 (位/秒)

主存储器

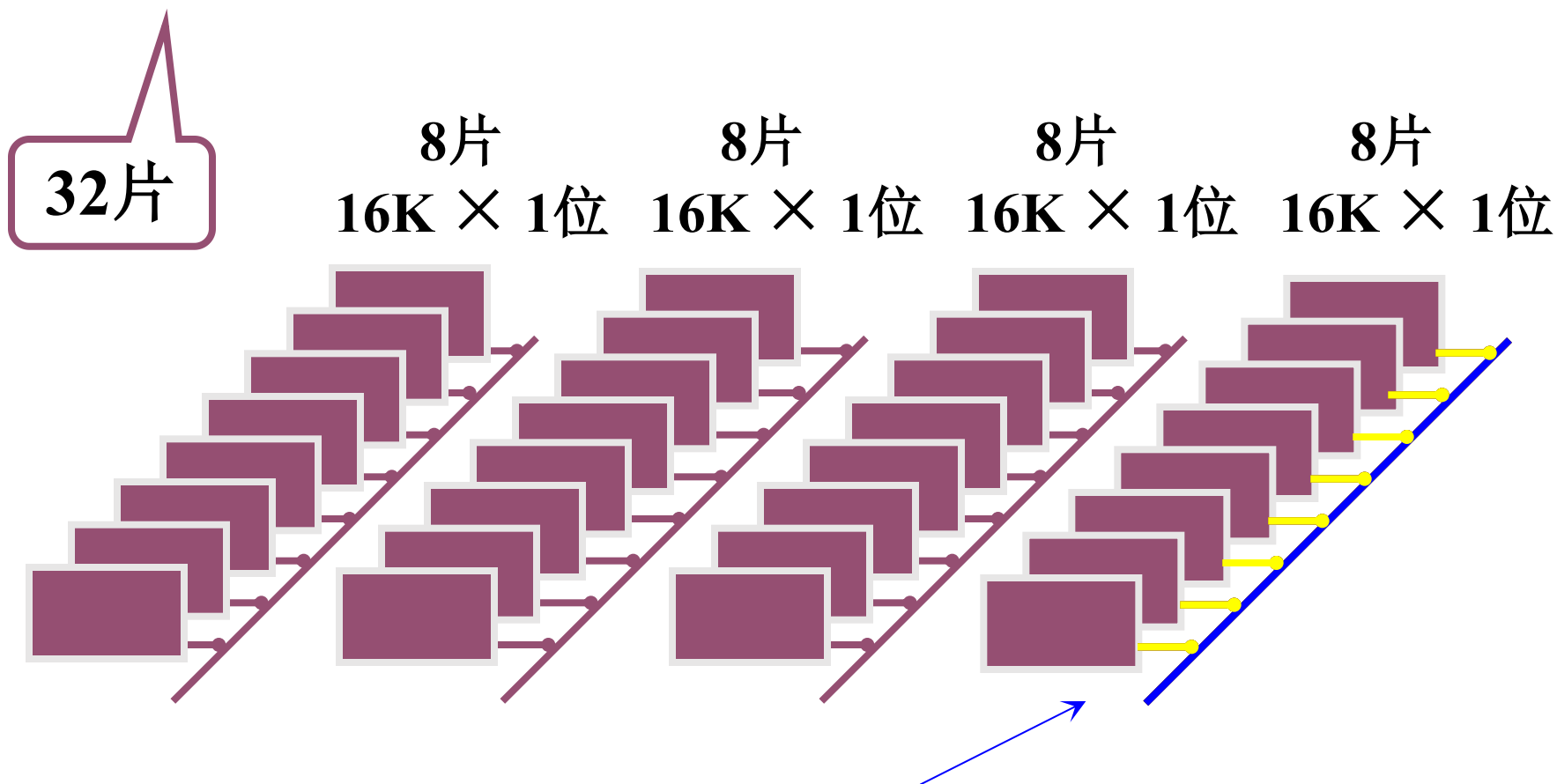
1. 概述
2. 半导体存储芯片简介
3. 随机存储器（RAM: Random Access Memory）
4. 只读存储器（ROM: Read Only Memory）
5. 存储器与CPU的连接
6. 存储器校验
7. 存储器的加速

半导体存储芯片的基本结构



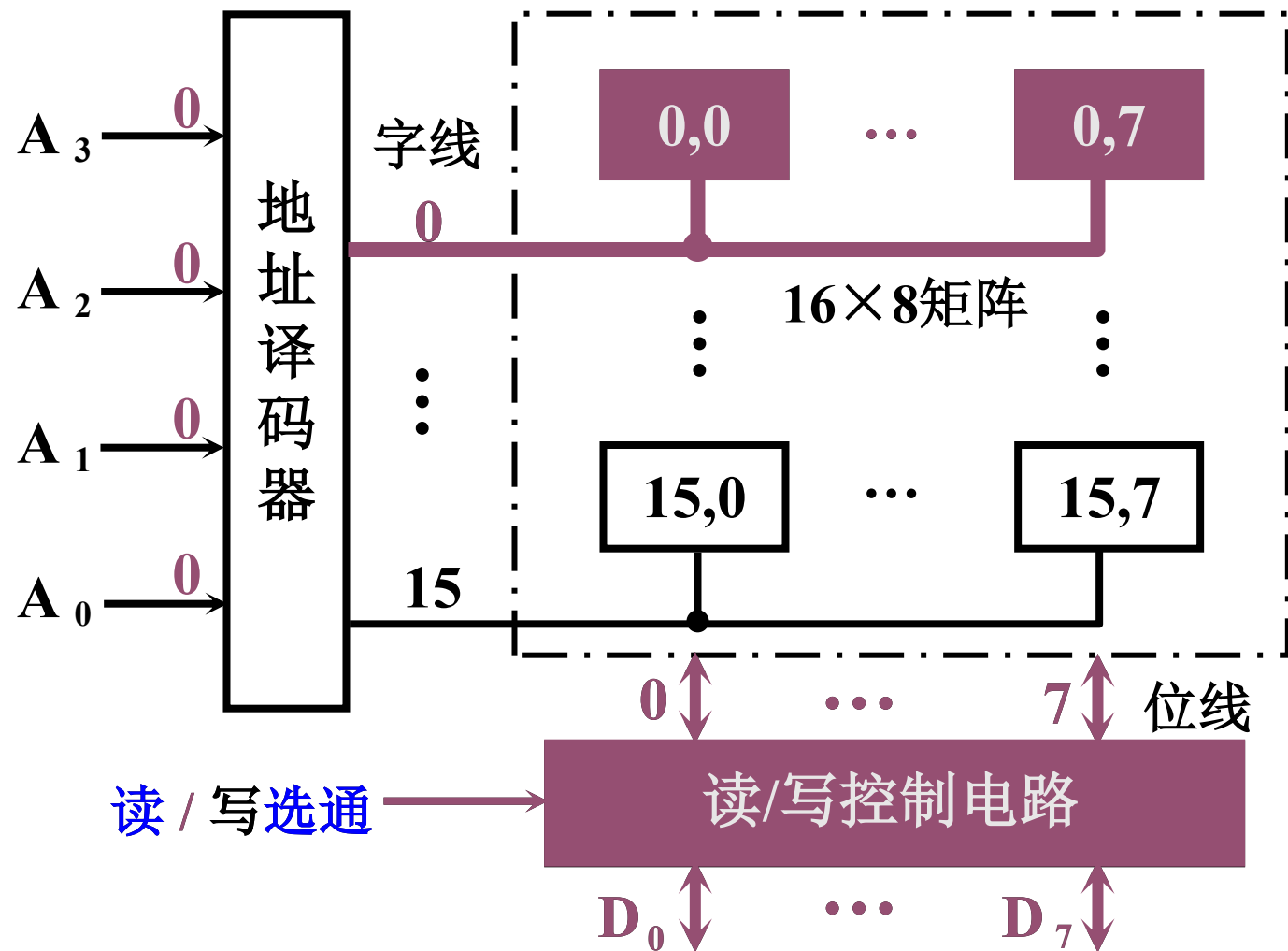
存储芯片片选线的作用

用 $16\text{K} \times 1$ 位的存储芯片组成 $64\text{K} \times 8$ 位的存储器

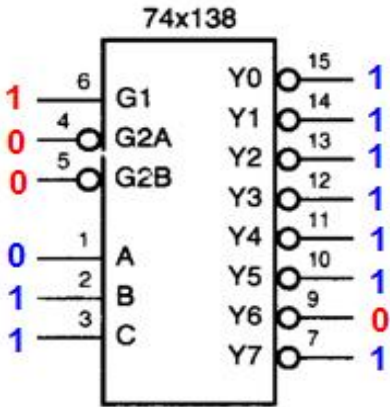


当地址为 65 535 时，此 8 片的片选有效

半导体存储芯片的译码驱动方式——线选法



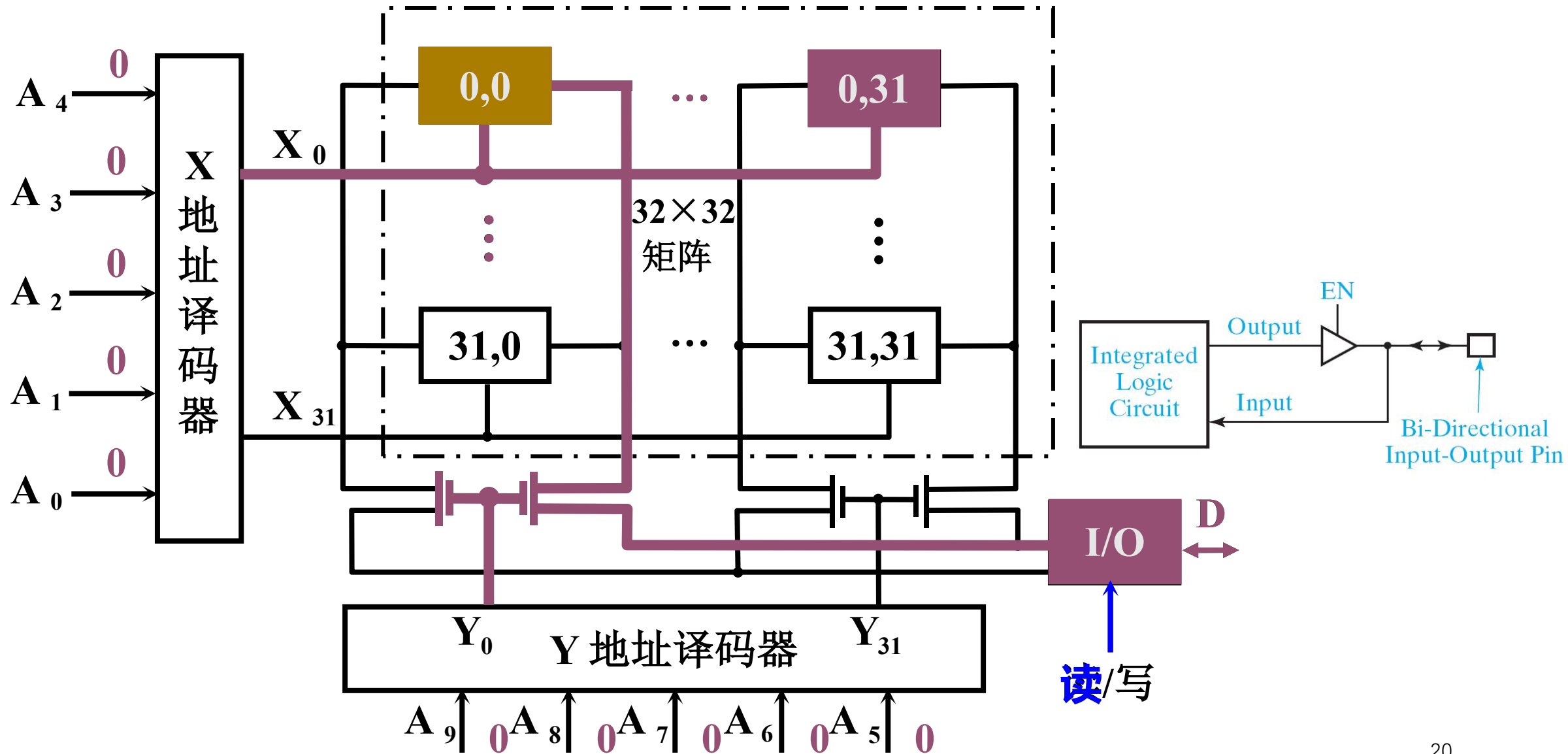
使能端			输入			译码输出							
G_1	G_{2A}	G_{2B}	C	B	A	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	0	1	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	0	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0



译码器输出：低电平有效

$$y_i = \overline{m}_i = M_i$$

半导体存储芯片的译码驱动方式——重合法

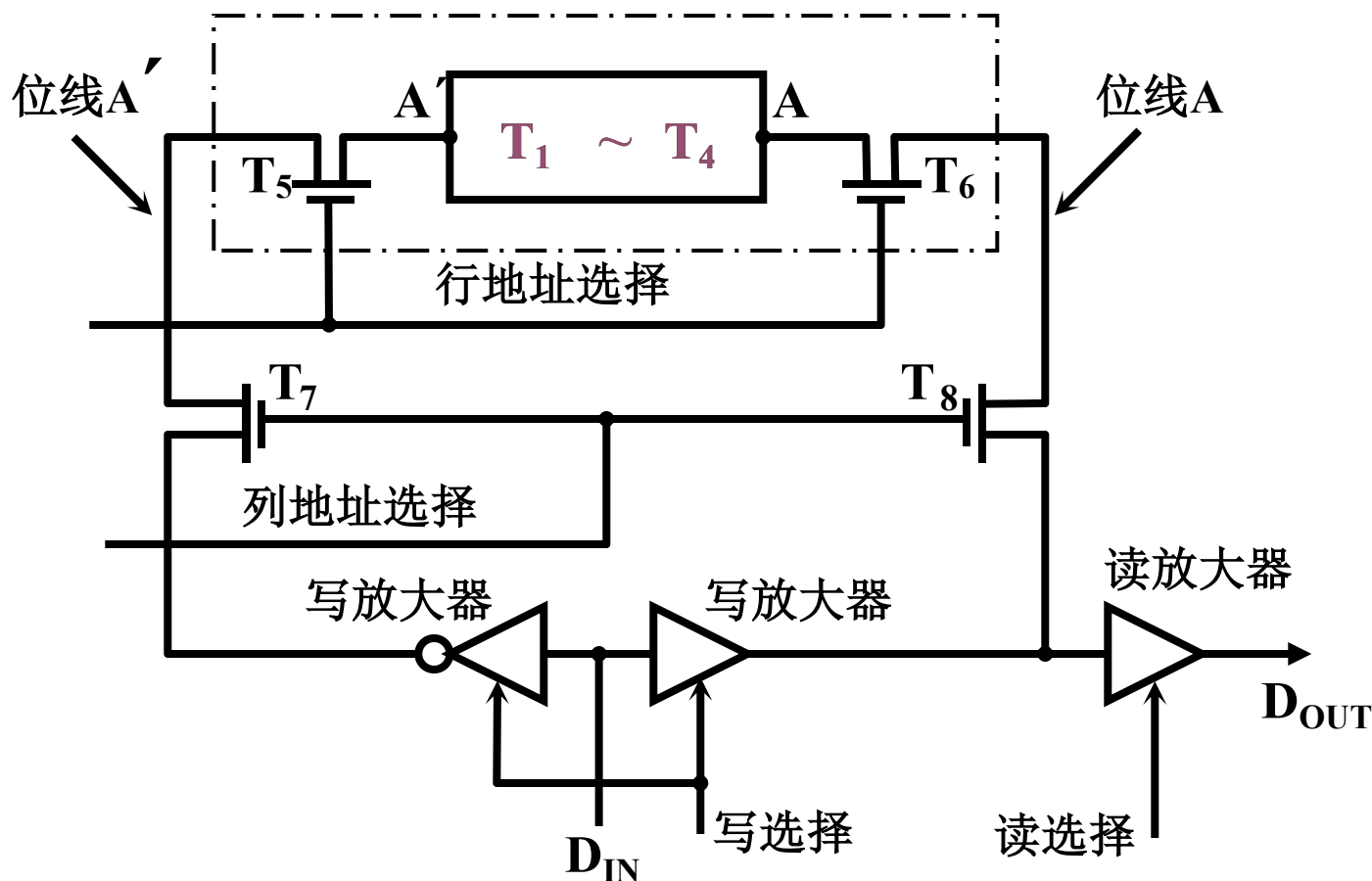


主存储器

1. 概述
2. 半导体存储芯片简介
3. 随机存储器 (**RAM: Random Access Memory**)
4. 只读存储器 (ROM: Read Only Memory)
5. 存储器与CPU的连接
6. 存储器校验
7. 存储器的加速

(1) 静态随机存储器 (SRAM)

Static Random Access Memory



$T_1 \sim T_4$ 触发器

T_5 、 T_6 行开关

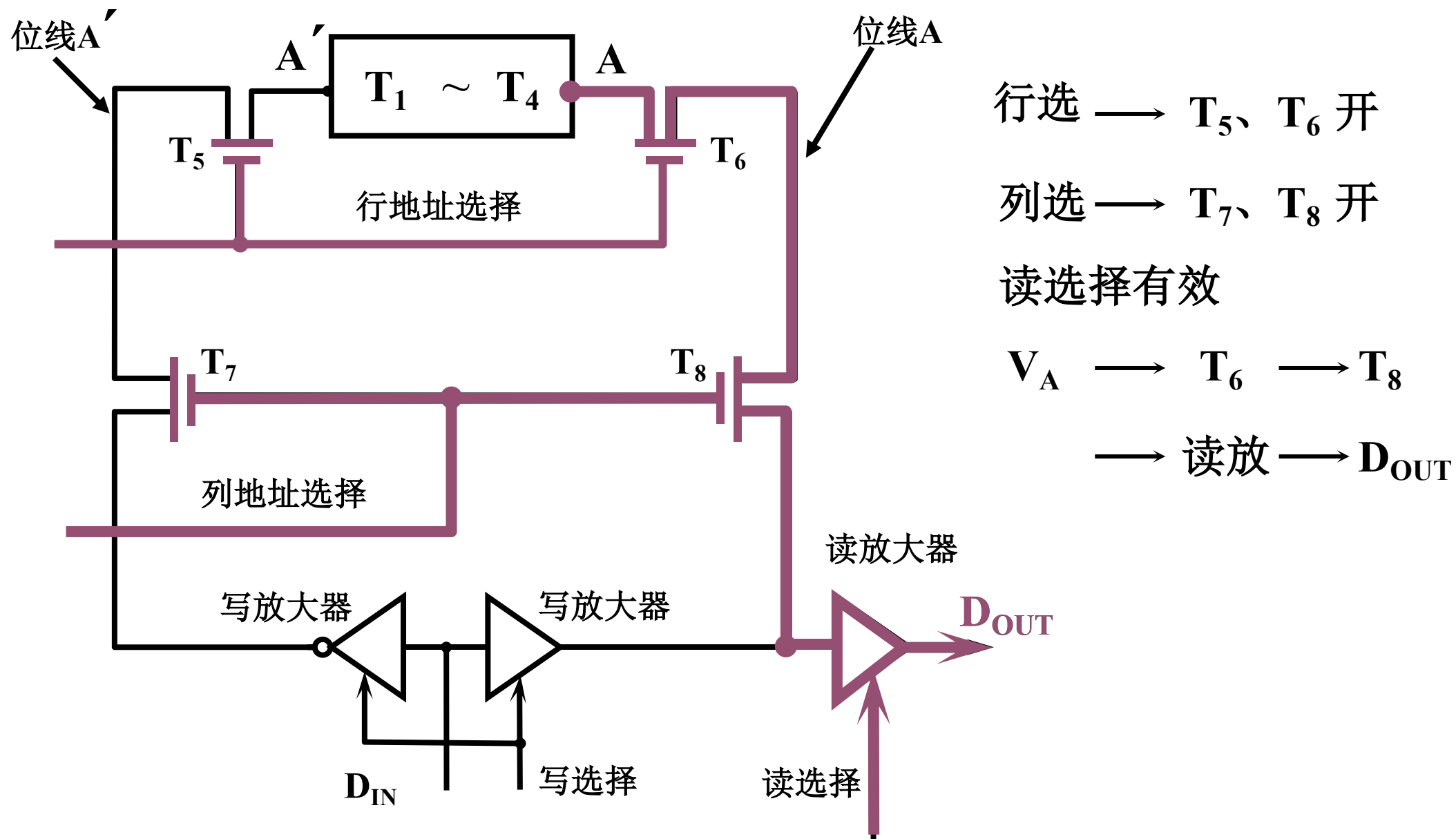
T_7 、 T_8 列开关

T_7 、 T_8 一列共用

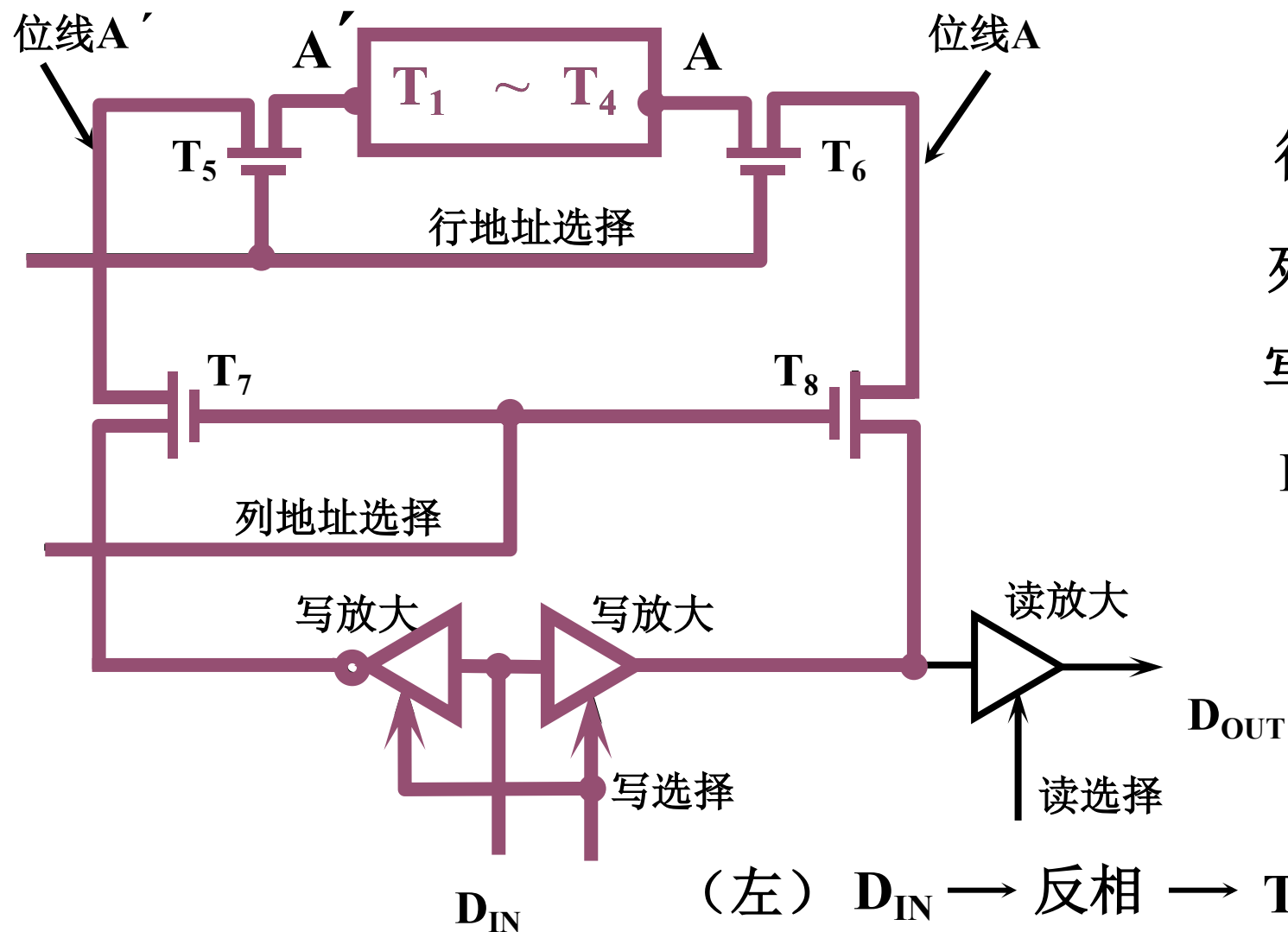
A 触发器原端

A' 触发器非端

① 静态RAM基本电路的读操作



②静态RAM基本电路的写操作



行选 $\rightarrow$ T₅、T₆ 开

列选 $\rightarrow$ T₇、T₈ 开

写选择有效

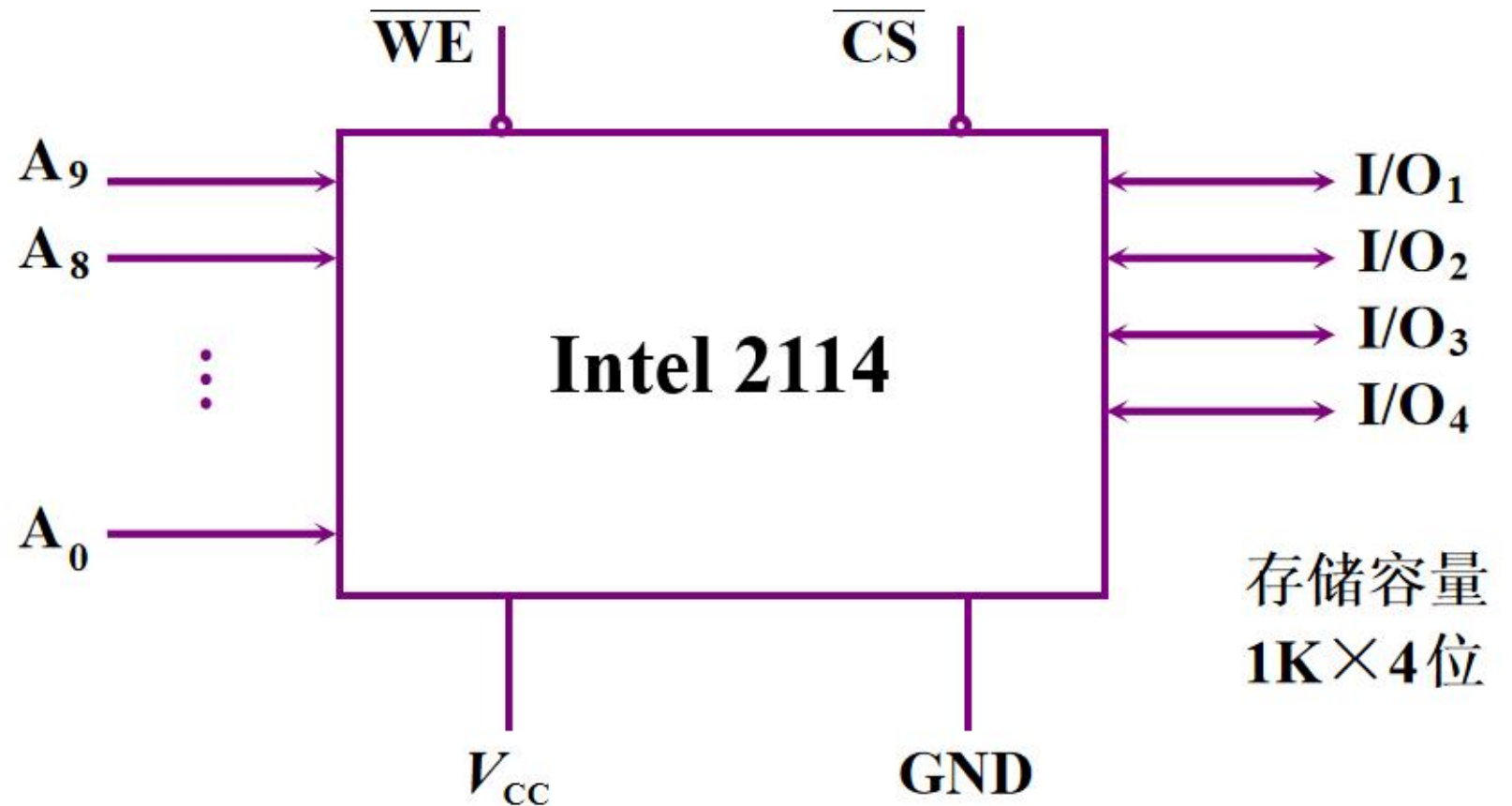
D_{IN} $\rightarrow$ 两个写放大器

(左) D_{IN} $\rightarrow$ 反相 $\rightarrow$ T₇ $\rightarrow$ T₅ $\rightarrow$ A'

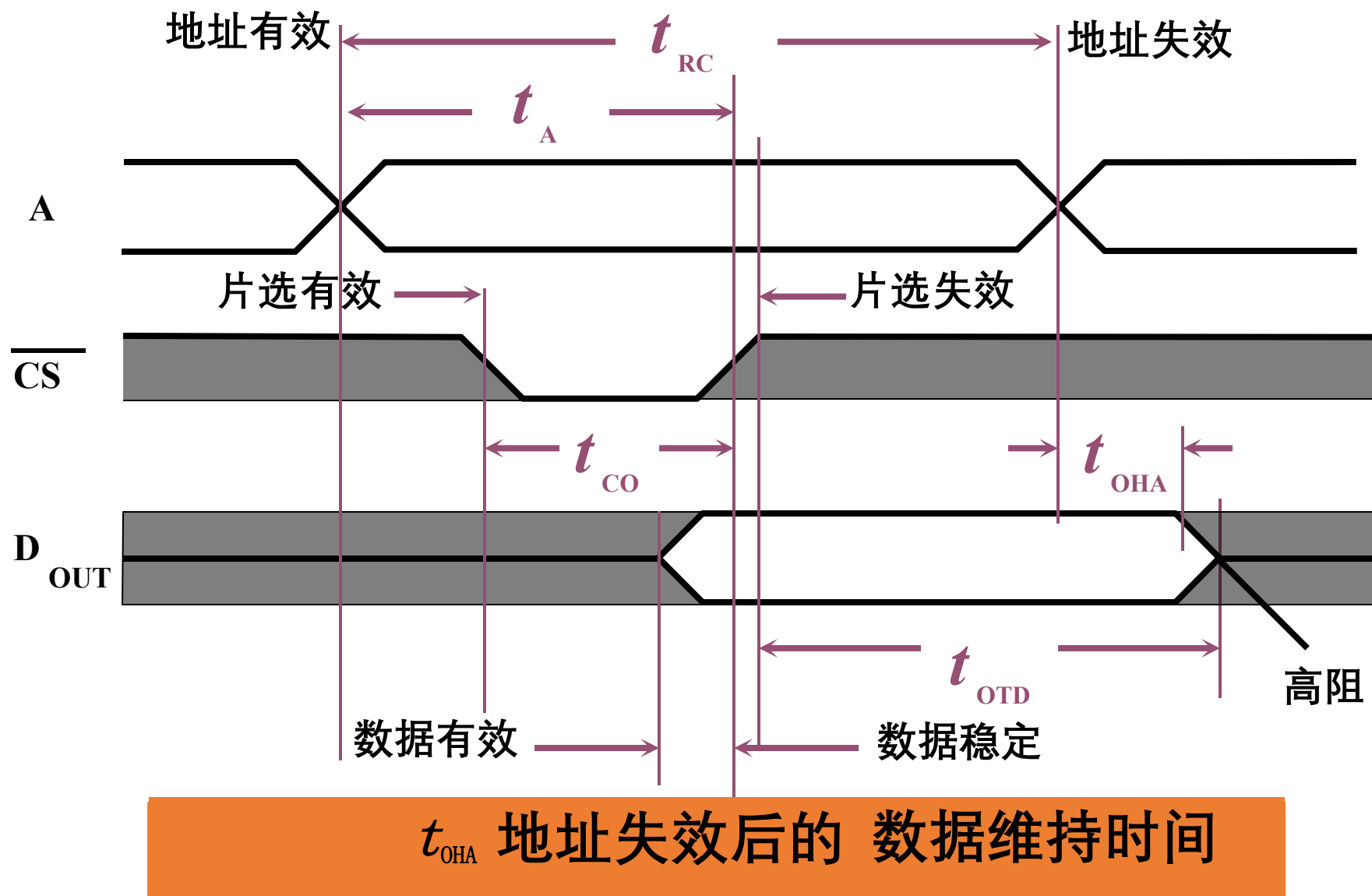
(右) D_{IN} $\rightarrow$ T₈ $\rightarrow$ T₆ $\rightarrow$ A

SRAM芯片举例：Intel2114

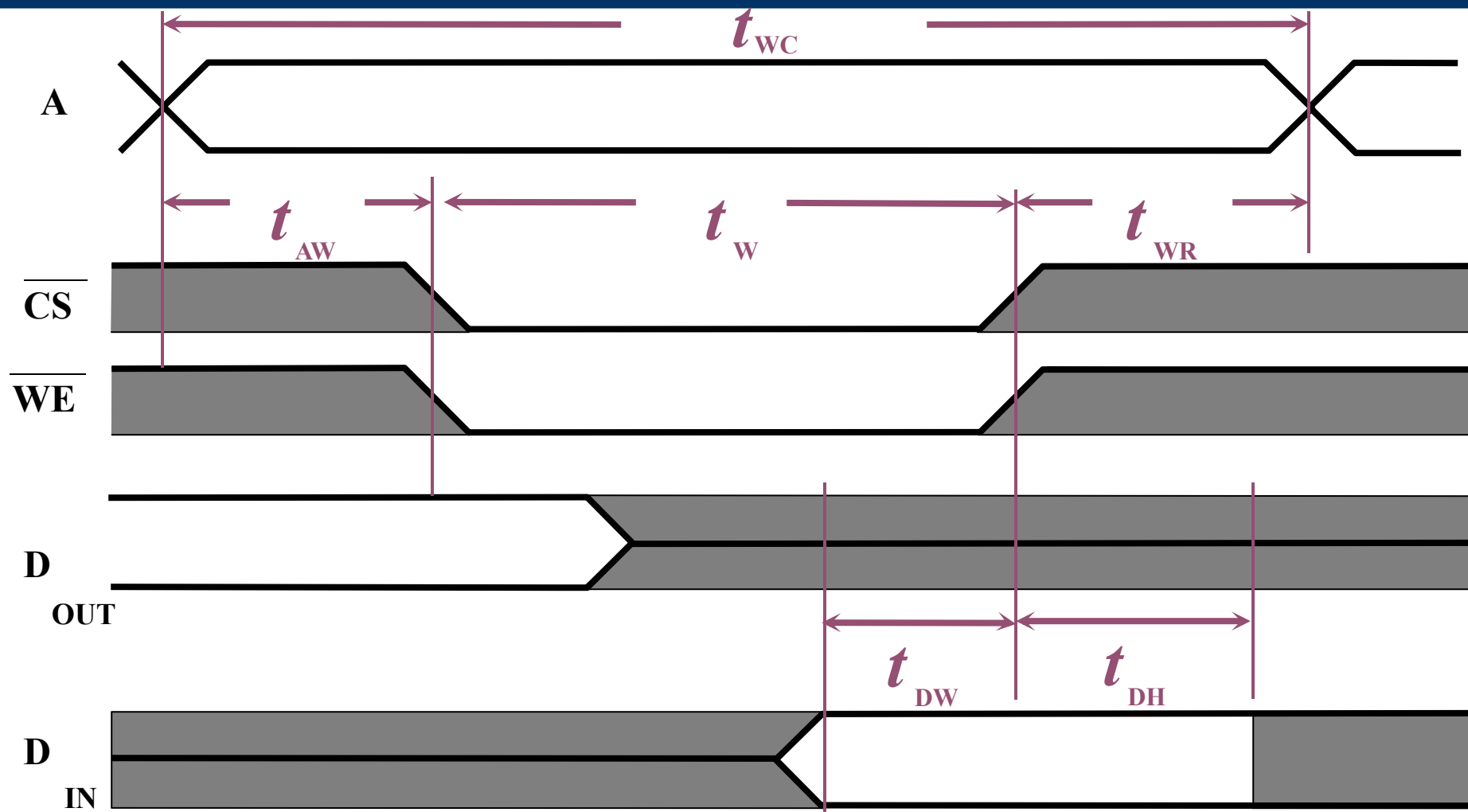
① Intel 2114 外特性



静态RAM (LD2114) 读时序



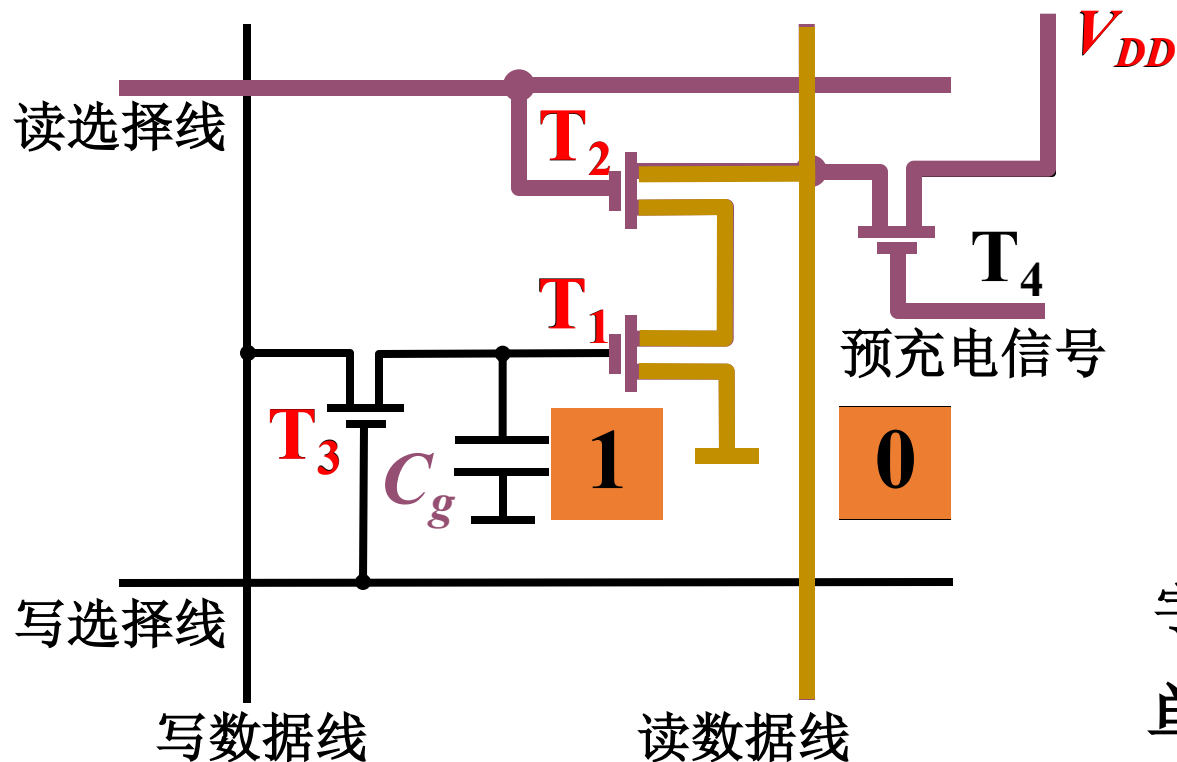
静态 RAM (LD2114) 写时序



t_{DH} $\overline{\text{WE}}$ 失效后的数据维持时间

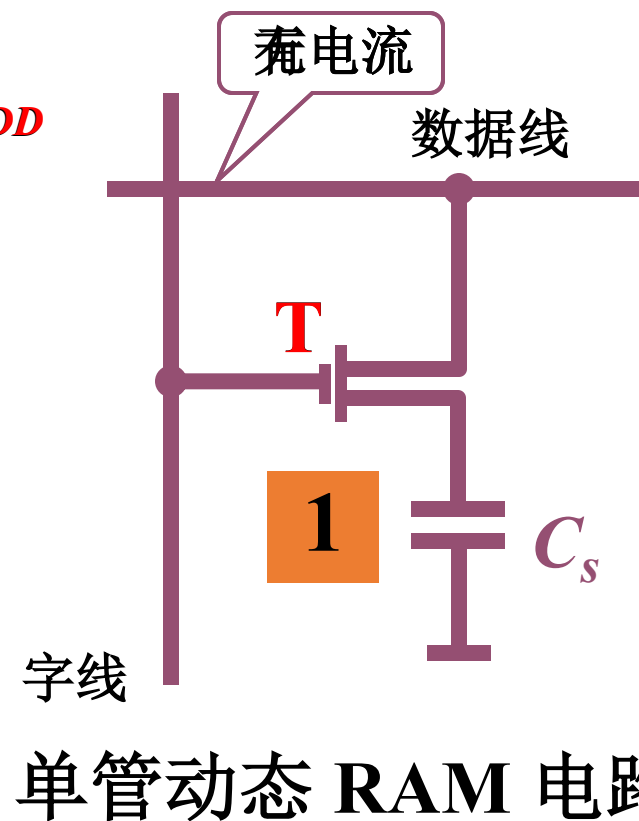
2. 动态 RAM (DRAM)

(1) 三管动态 RAM 基本单元电路



读出与原存信息相反（破坏性读）

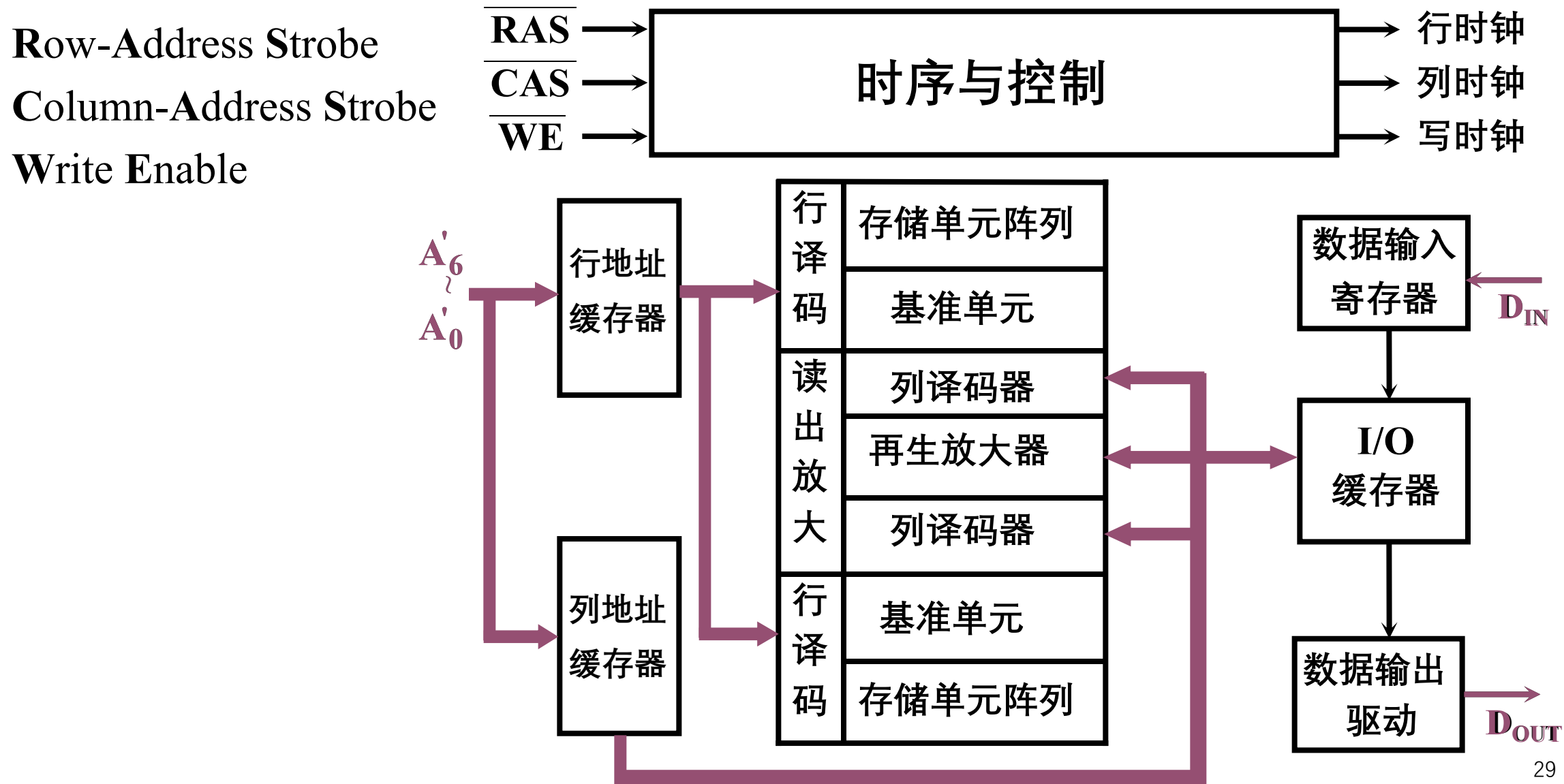
写入与输入信息相同



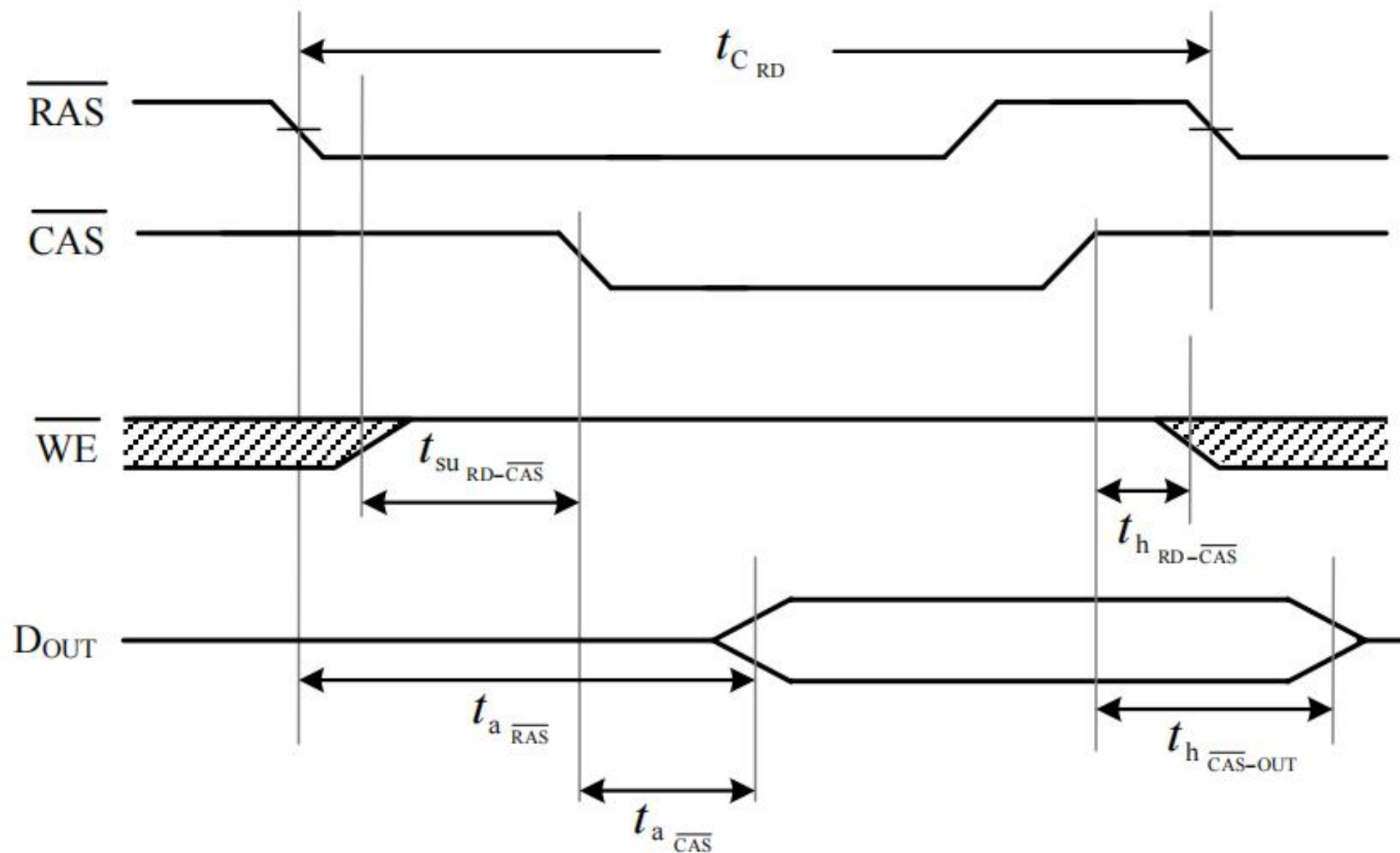
读出时数据线有电流为“1”（破坏性读）

写入时 C_s 充电为“1”放电为“0”

(2) 动态RAM举例 ③ 单管动态 RAM 4116 (16K × 1位) 外特性



动态RAM读操作时序图



(3) 动态RAM时序

行、列地址分开传送

读时序

行地址 $\overline{\text{RAS}}$ 有效
写允许 $\overline{\text{WE}}$ 无效(高)
列地址 $\overline{\text{CAS}}$ 有效
数据 D_{OUT} 有效

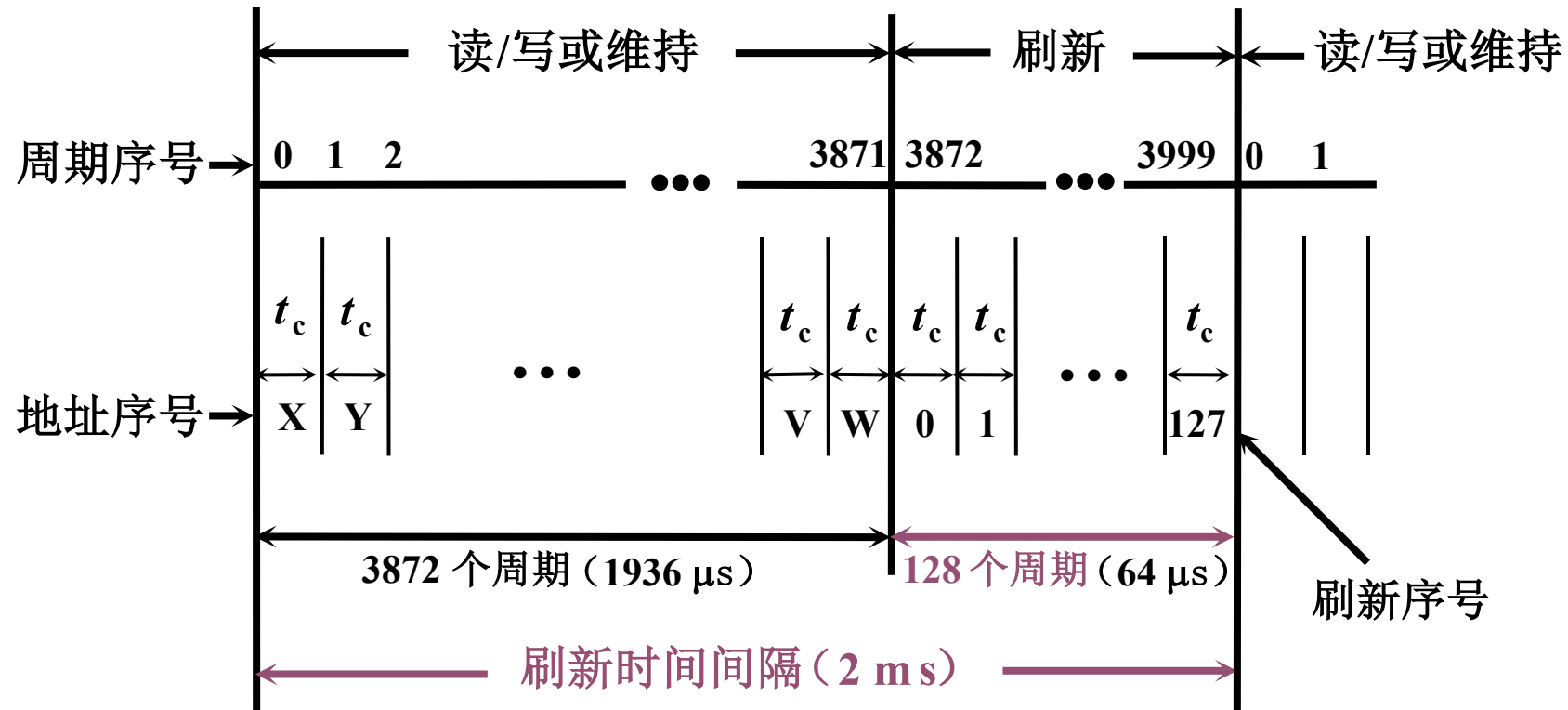
写时序

行地址 $\overline{\text{RAS}}$ 有效
写允许 $\overline{\text{WE}}$ 有效(低)
数据 D_{IN} 有效
列地址 $\overline{\text{CAS}}$ 有效

(4) 动态RAM刷新——以 128×128 矩阵为例

按行刷新:刷新与行地址有关

① 集中刷新(刷新周期一般为1-2ms, 设存取周期为 $0.5 \mu\text{s}$)

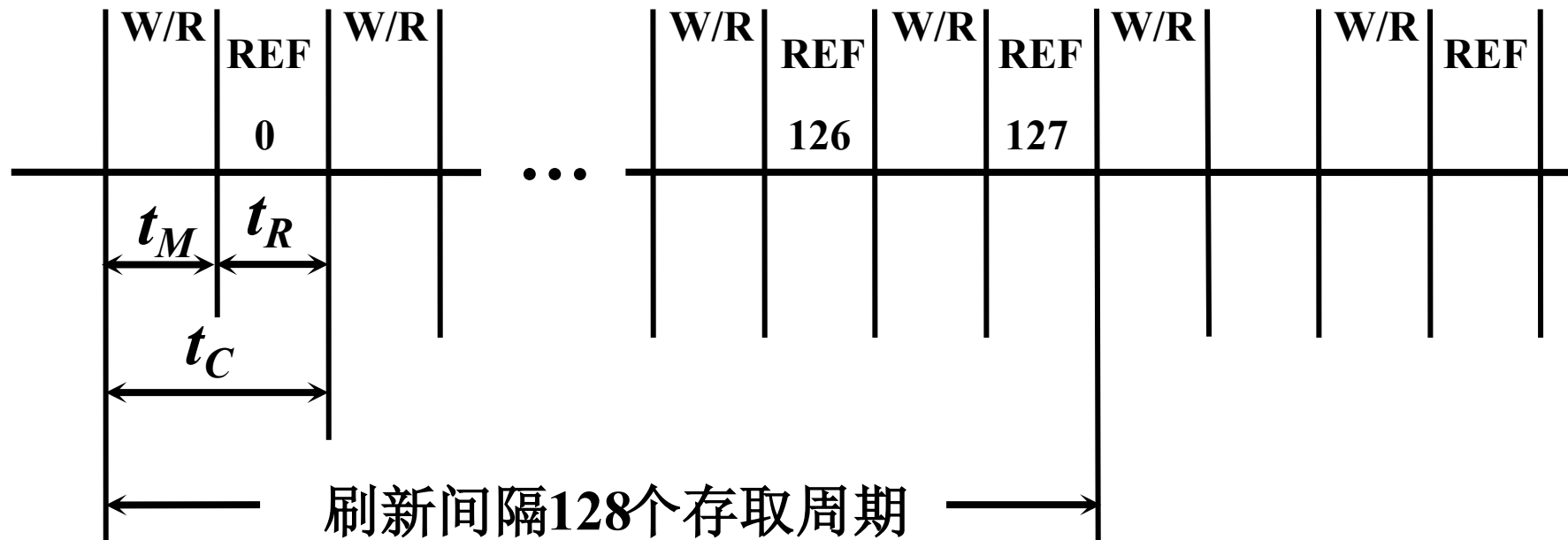


“死区”为 $0.5 \mu\text{s} \times 128 = 64 \mu\text{s}$

“死时间率 (死区占比)”为 $128/4000 = 3.2\%$

(4) 动态RAM刷新——以 128×128 矩阵为例

② 分散刷新(存取周期为 $2 \times 0.5 \mu\text{s}$)



$$t_C = t_M + t_R$$

无“死区”

↓ ↓
读写 刷新

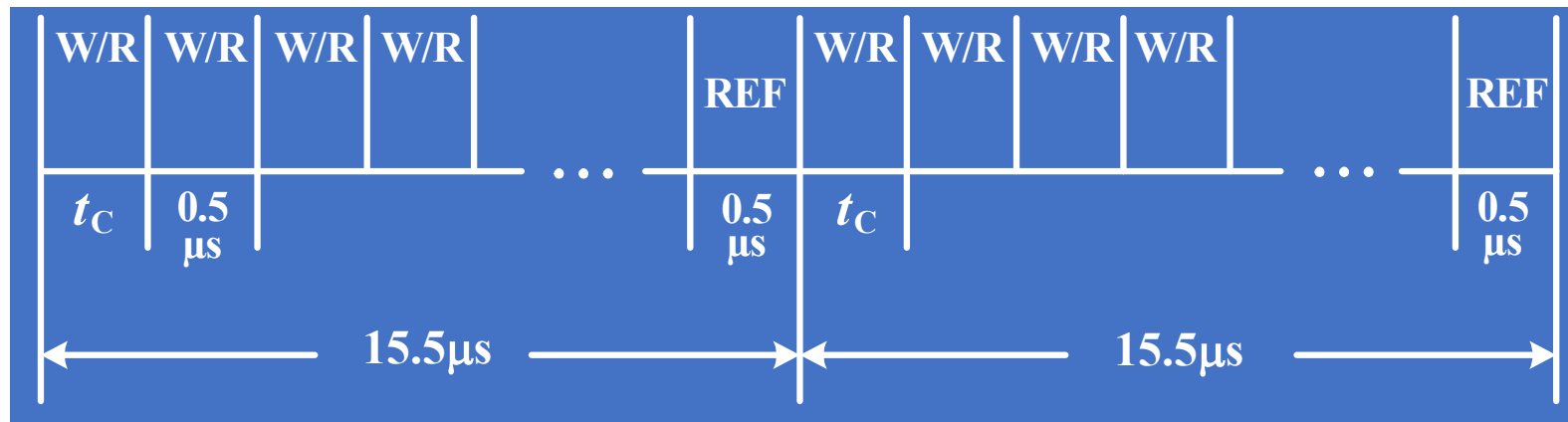
(存取周期为 $0.5 \mu\text{s} + 0.5 \mu\text{s}$)

(4) 动态RAM刷新——以 128×128 矩阵为例

③ 异步刷新(分散刷新与集中刷新结合,存取周期 $0.5 \mu\text{s}$)

每隔 $15.5 \mu\text{s}$ 刷新一行 ($2000\mu\text{s}/128=15.625\mu\text{s} \approx 31$ 个存取周期)

同一行每隔 1.984 ms 刷新一次



“死区” 为 $0.5 \mu\text{s}$

合理安排刷新时间, 可以避免出现 “死区”

动态RAM和静态RAM的比较

	主存 DRAM	SRAM 缓存
存储原理	电容	触发器
集成度	高	低
芯片引脚	少	多
功耗	小	大
价格	低	高
速度	慢	快
刷新	有	无

主存储器

1. 概述
2. 半导体存储芯片简介
3. RAM（StaticRAM&DynamicRAM）及其读写
4. 只读存储器（**ROM: Read Only Memory**）
5. 存储器与CPU的连接
6. 存储器校验
7. 存储器的加速

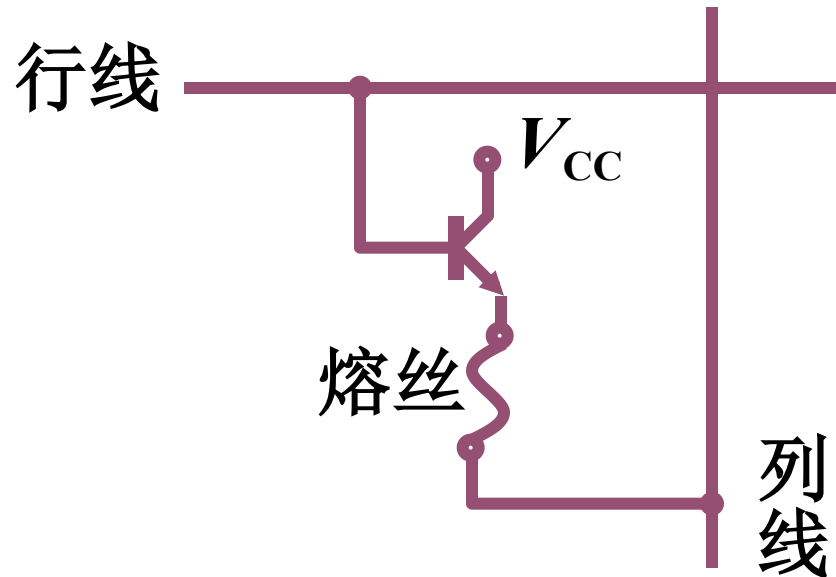
只读存储器——MROM, PROM

1. 掩模 ROM (MROM)

行列选择线交叉处有 MOS 管为 “1”

行列选择线交叉处无 MOS 管为 “0”

2. PROM (一次性编程)



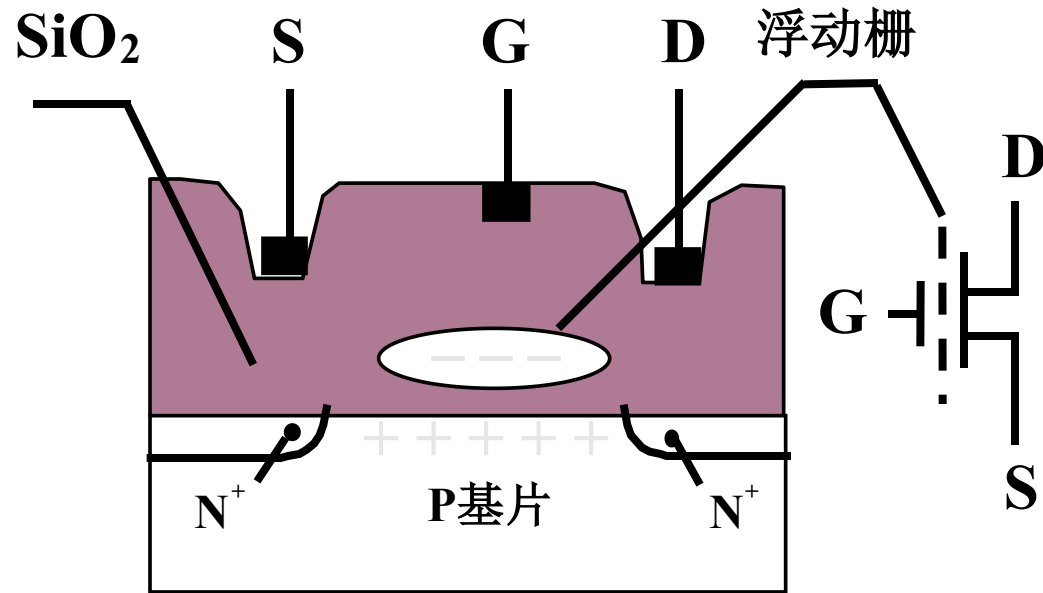
熔丝断 为 “0”

熔丝未断 为 “1”

只读存储器——EPROM

3. EPROM: Electrical Programmable Read Only Memory

例子：N型沟道浮动栅 MOS 电路



G 栅极

S 源

D 漏

紫外线可驱散浮动栅

D 端加正电压

形成浮动栅

S 与 D 不导通为 “0”

D 端不加正电压

不形成浮动栅

S 与 D 导通为 “1”

只读存储器——EEPROM,Flash Memory

4. Electrically Erasable Programmable Read Only Memory

- 电可擦写
- 局部擦写
- 全部擦写

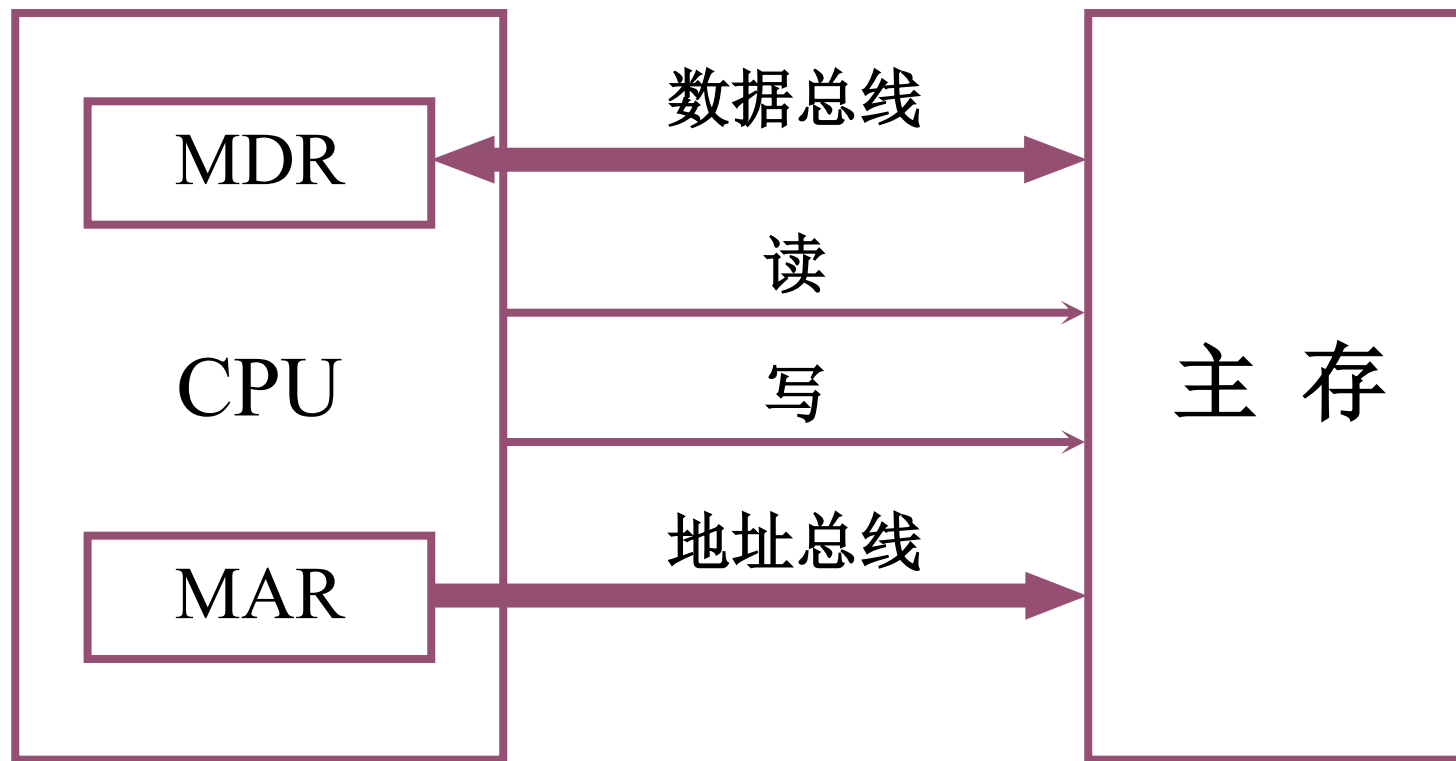
5. Flash Memory (闪速型存储器)

- EPROM 价格便宜 集成度高
- EEPROM 电可擦除——>重写
- Flash Memory 比 EEPROM快, 具备 RAM 功能

主存储器

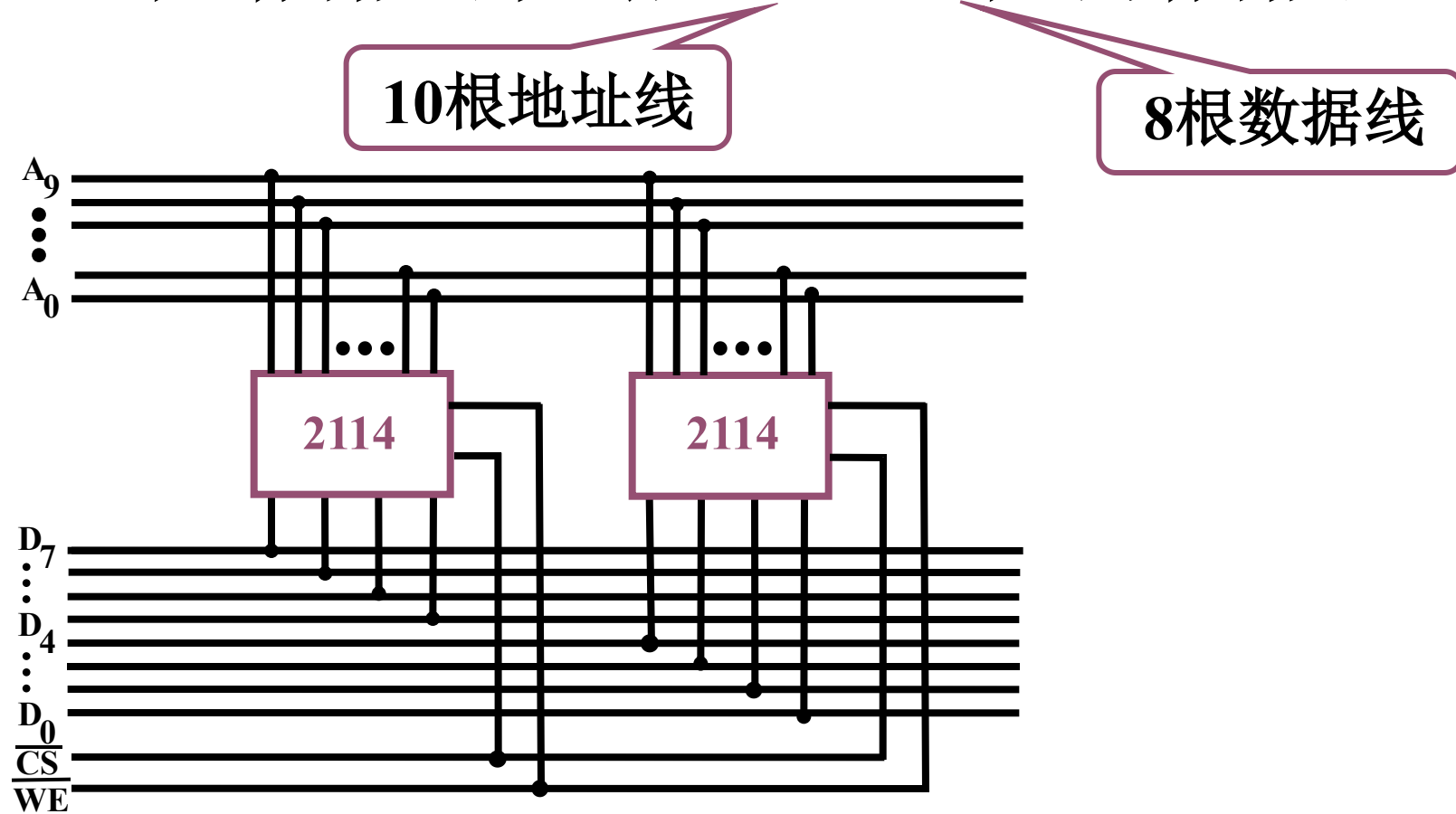
1. 概述
2. 半导体存储芯片简介
3. RAM（StaticRAM&DynamicRAM）及其读写
4. 只读存储器（ROM: Read Only Memory）
- 5. 存储器与CPU的连接**
6. 存储器校验
7. 存储器的加速

存储器与CPU的连接



存储器容量扩展——位扩展（增加存储字长）

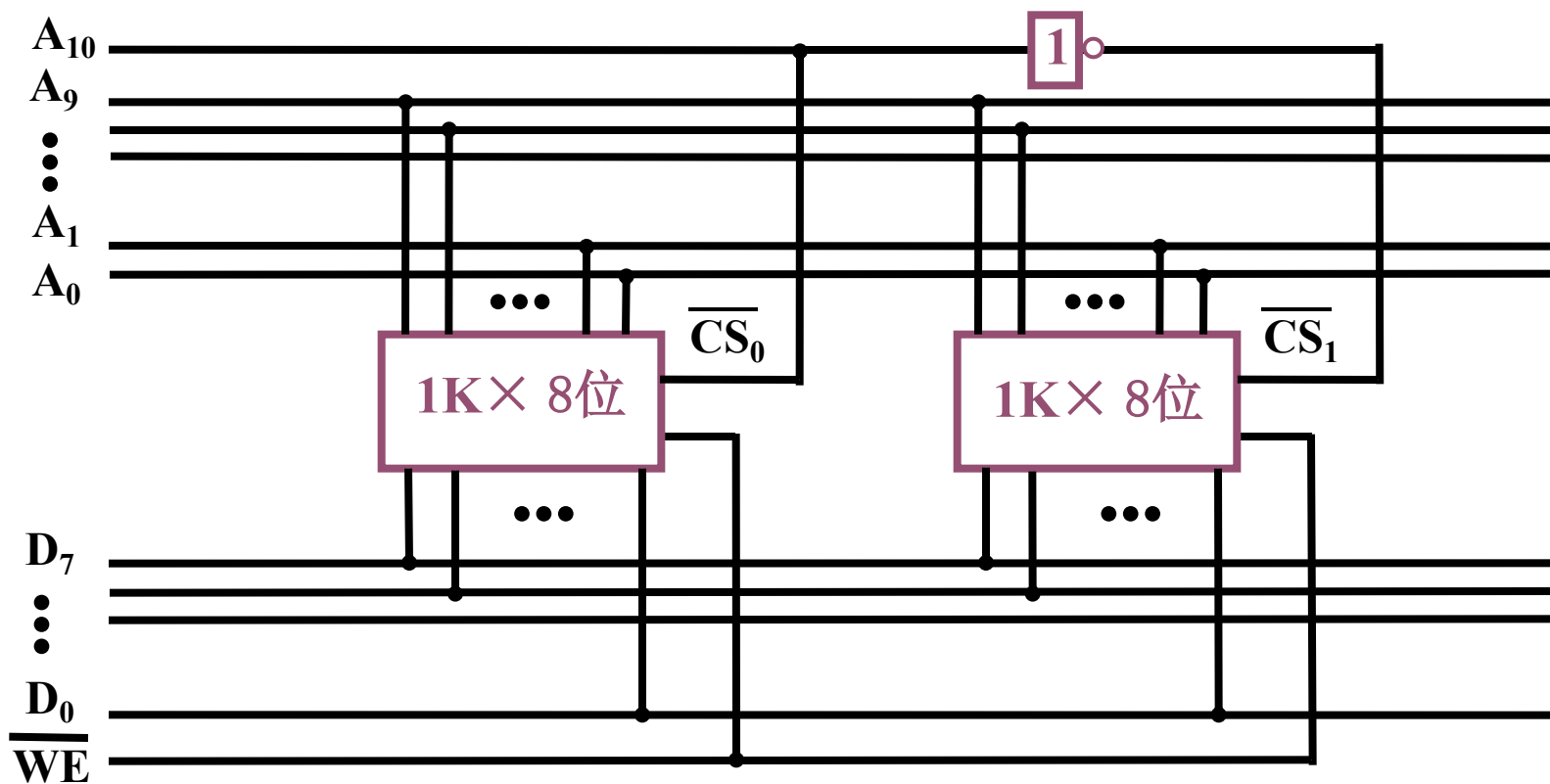
用 2 个 $1\text{K} \times 4$ 位 存储芯片组成 $1\text{K} \times 8$ 位的存储器



存储器容量扩展——字扩展(增加存储字数量)

用 2 个 $1\text{K} \times 8$ 位 存储芯片组成 $2\text{K} \times 8$ 位的存储器

11根地址线

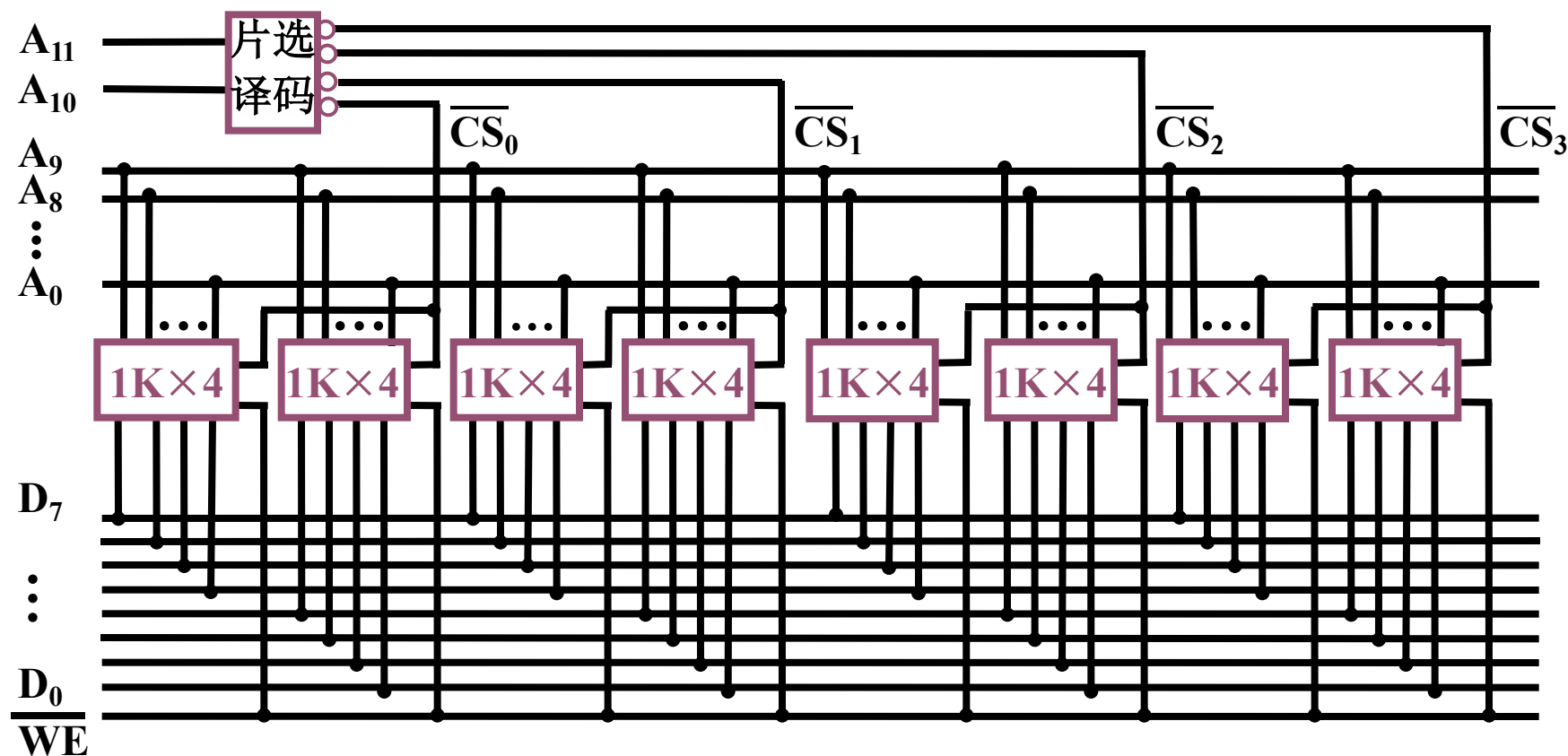


存储器容量扩展——字、位扩展

用 8 个 $1\text{K} \times 4$ 位 存储芯片组成 $4\text{K} \times 8$ 位的存储器

12根地址线

8根数据线



主存储器与 CPU 的连接

- ① 地址线的连接：通常将 CPU 地址线低位与存储芯片地址线相连，高位用于存储芯片扩充，或用做片选信号等。
- ② 数据线的连接：存储芯片与CPU的数据线数不等时，必须对存储芯片进行位扩展，使二者相等。
- ③ 读/写控制线的连接
- ④ 片选线的连接
- ⑤ 合理选择存储芯片
- ⑥ 其他：时序、负载等

例题（唐书94页例4.1）

设CPU有16根地址线，8根数据线， $\overline{\text{MREQ}}$ 为访存控制信号（低电平有效）， WR 为读/写控制信号（高电平为读，低电平为写），现有下列芯片若干：RAM：1K×4位；4K×8位；8K×8位；ROM：2K×8位；4K×8位；8K×8位；74LS138译码器和各种门电路。请画出CPU与存储器的连接图，要求：

① 主存地址空间分配：

6000H~67FFH 为系统程序区；

6800H~6BFFH 为用户程序区。

② 合理选用上述存储芯片，说明各选几片？

③ 详细画出存储芯片的片选逻辑图。

例题解析

(1) 写出对应的二进制地址

系统程序区 $A_{15}A_{14}A_{13}A_{12}A_{11}A_{10}A_9A_8A_7 \dots \dots \dots A_0$

6000H 0 1 1 0 0 0 0 0 0 0

~
⋮

67FFH 0 1 1 0 0 1 1 1 1 1

用户程序区

6800H 0 1 1 0 1 0 0 0 0 0

~
⋮

6BFFH 0 1 1 0 1 0 1 1 1 1

1片 $2K \times 8$ 位

ROM

$2K \times 8$ 位

$1K \times 8$ 位

RAM

2片 $1K \times 4$ 位

(2) 确定芯片的数量及类型

RAM: $1K \times 4$ 位; $4K \times 8$ 位; $8K \times 8$ 位;

ROM: $2K \times 8$ 位; $4K \times 8$ 位; $8K \times 8$ 位;

例题解析——续

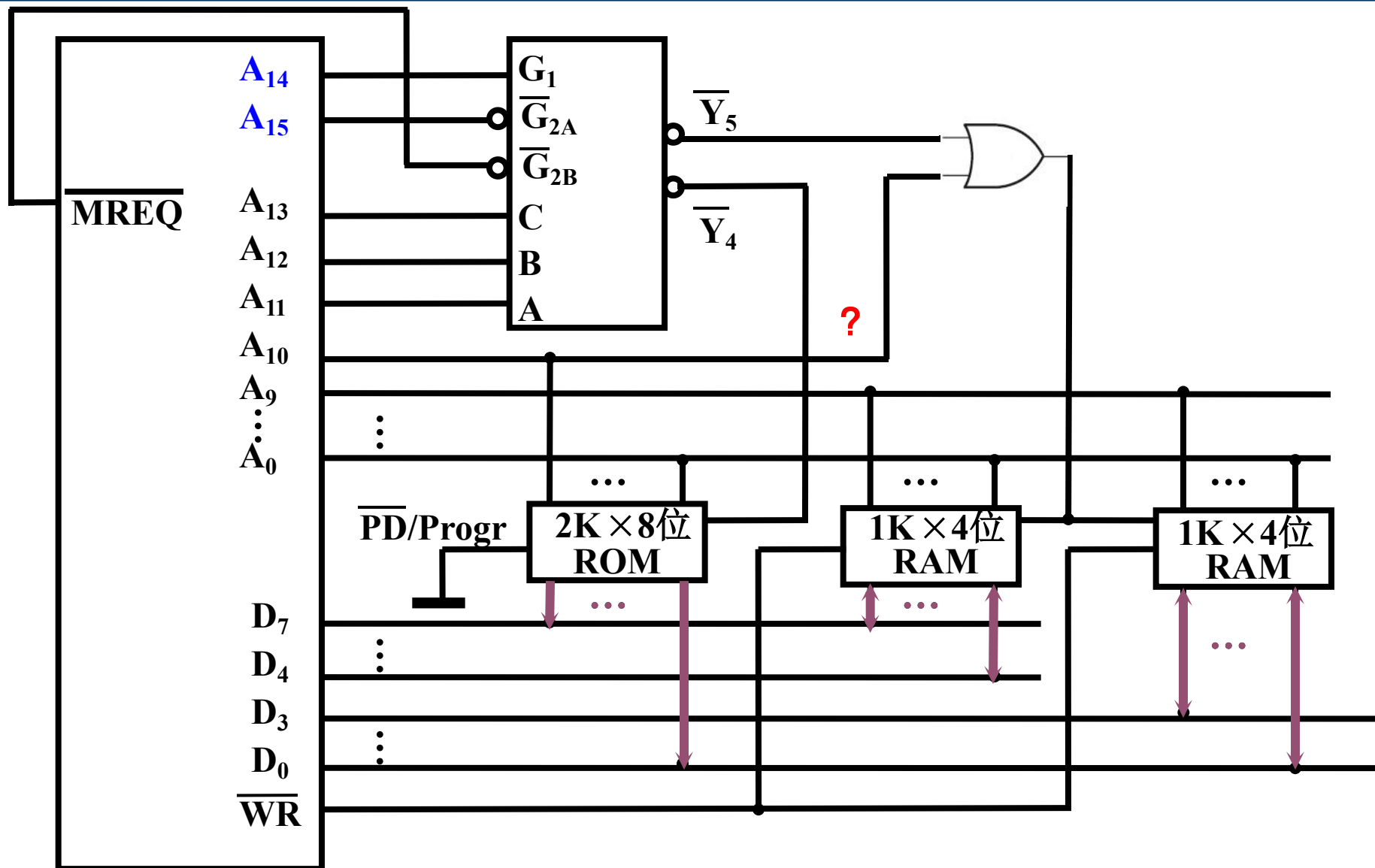
(3) 分配地址线

$A_{10} \sim A_0$ 接 $2K \times 8$ 位 ROM 的地址线

系统程序区	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	...	...	...	A_0	
6000H	0	1	1	0	0	0	0	0	0	...	...	...	0	} $2K \times 8$ 位 ROM 1个
~	⋮													
67FFH	0	1	1	0	0	1	1	1	1	...	...	...	1	
用户程序区														
6800H	0	1	1	0	1	0	0	0	0	...	...	...	0	} $1K \times 4$ 位 RAM 2个
~	⋮													
6BFFH	0	1	1	0	1	0	1	1	1	...	...	...	1	
			C	B	A									$A_9 \sim A_0$ 接 $1K \times 4$ 位 RAM 的地址线

(4) 确定片选信号

例题解析——CPU 与存储器的连接图



例2

假设同前，要求最小 4K 为系统程序区，相邻 8K 为用户程序区。

(1) 写出对应的二进制地址码

(2) 确定芯片的数量及类型

1个 $4K \times 8$ 位 ROM， 2个 $4K \times 8$ 位 RAM (1个 $8K \times 8$ 位)

(3) 分配地址线

$A_{11} \sim A_0$ 接ROM和RAM的地址线（如果用8KRAM，地址线不同）

(4) 确定片选信号

主存储器

1. 概述
2. 半导体存储芯片简介
3. 随机存储器（RAM: Random Access Memory）
4. 只读存储器（ROM: Read Only Memory）
5. 存储器与CPU的连接
- 6. 存储器校验**
7. 存储器的加速

存储器的校验

- 为什么要对存储器的信息进行校验？
- 为了能够校验出信息是否正确，如何进行编码？
- 纠错和检错能力与什么因素有关？
- 校验出信息出错后是如何进行纠错？

存储器校验基本概念

数据传输过程中，发生**1位**出错的约占96%。

合法（二进制）编码集合		检错位数、纠错位数	
{000,001,010,011,100,101,110,111}		0	0
{000,011,101,110}			
100 (000? 101? 110?)		1	0
{000, 111}	编码的检测能力和纠错能力与什么有关?		
100(000) 110(111)		1	1
{0000, 1111}			
1000 1100	任意两组合法编码之间二进制位的最小差异数	2	1
{00000, 11111}			
11000 11100		2	2

编码的最小距离

- 任意两组合法代码之间 二进制位数 的最少差异
- 编码的纠错、检错能力与编码的最小距离有关

$$L - 1 = D + C \quad (D \geq C)$$

L —— 编码的最小距离（汉明距离）

D —— 检测错误的位数

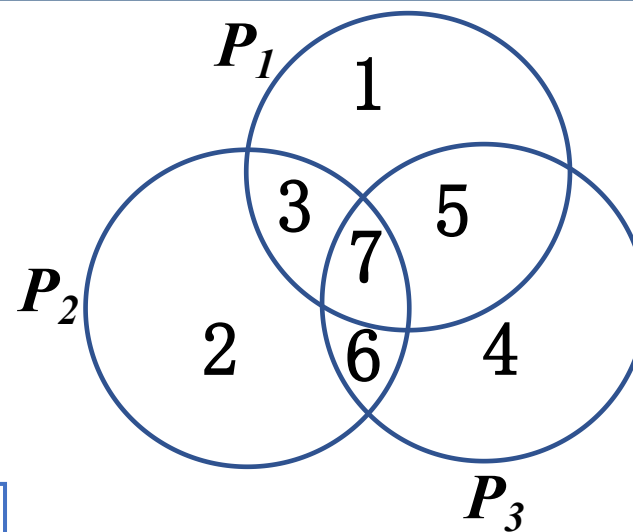
C —— 纠正错误的位数

若某编码的 $L = 3$ ，其最大纠错能力？

具有 1位 纠错能力

存储器校验——汉明码 (Hanming)

- 具有1位纠错能力
- 采用奇偶校验
- 采用分组校验



校验位应放哪里呢?

1, 2, 4, ...
位置放校验位

00100011 100100011 1001000011

校验位 校验位

给出了出
错的位置

$P_3 P_2 P_1$

0 0 0
0 0 1
1 0 1
1 1 0
1 1 1

无差错
1
5
6
7

汉明码的分组是一种非划分方式

1	2	3	4	5	6	7
---	---	---	---	---	---	---

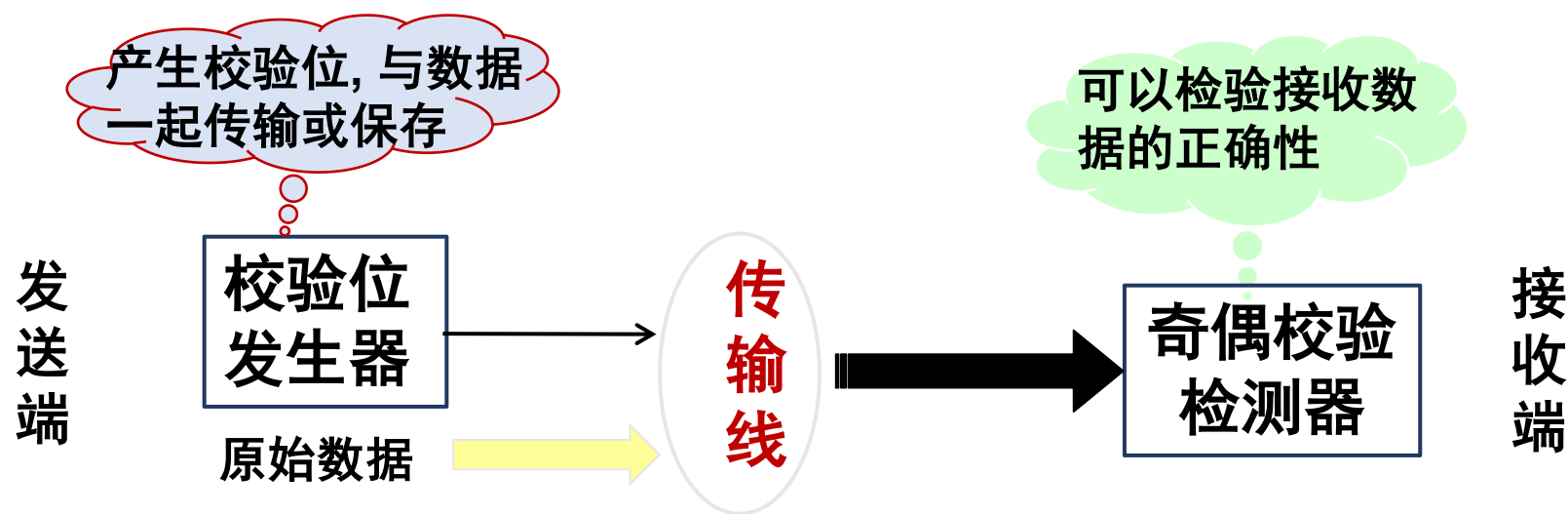
分成3组，每组有1位校验位，共包括4位数据位

如何分组?

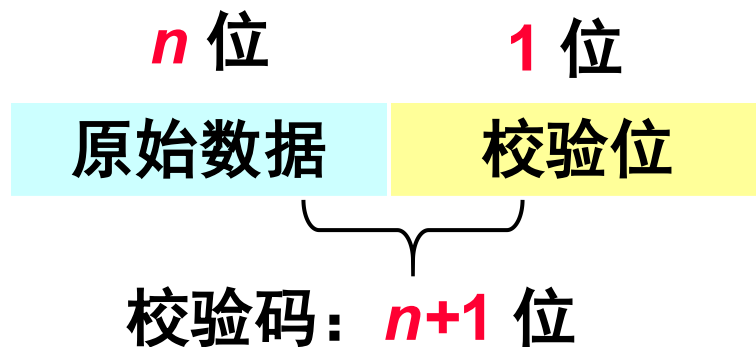
第1组 XXXX1
第2组 XXX1X
第3组 XX1XX
第4组 X1XXX
第5组 1XXXX

奇偶校验器

- 用来检查数据传输和存取过程中是否产生错误的组合逻辑电路。
 - 发生一位错误的可能性一般占96%以上；
- 就是检测数据中包含“1”的个数是奇数还是偶数。
- 能够检测传送出错，但不能确定错误位置，不能纠错。
- 电路简单，容易实现。
- 广泛用于计算机的内存储器以及磁盘等外部设备中。



校验位计算方法



偶校验位逻辑值的表达式:

$$P_E = A_3 \oplus A_2 \oplus A_1 \oplus A_0$$

奇校验位逻辑值的表达式:

$$P_O = A_3 \oplus A_2 \oplus A_1 \oplus A_0$$

异或门真值表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

奇偶校验器一般
由异或门构成

异或门特性

- 两个输入中有奇数个“1”，输出为1；
有偶数个“1”，输出为0。
- 扩展: n 个1位二进制数中有奇数个“1”，输出为1；
有偶数个“1”，输出为0。

4位二进制数校验码真值表

$A_3A_2A_1A_0$	P_E	P_O
0 0 0 0	0	1
0 0 0 1	1	0
0 0 1 0	1	0
0 0 1 1	0	1
0 1 0 0	1	0
0 1 0 1	0	1
0 1 1 0	0	1
0 1 1 1	1	0
1 0 0 0	1	0
1 0 0 1	0	1
1 0 1 0	0	1
1 0 1 1	1	0
1 1 0 0	0	1
1 1 0 1	1	0
1 1 1 0	1	0
1 1 1 1	0	1

组成汉明码的三要素

n 位二进制数的汉明码需增添 多少 位（ k ）检测位？

$$2^k \geq n + k + 1 \quad (\text{汉明不等式})$$

检测位的位置？

位置	1	2	3	4	5	6	7
	C_1	C_2	b_4	C_4	b_3	b_2	b_1

$$2^i \quad (i = 0, 1, 2, 3, \dots)$$

检测位的取值？

与该位所在的检测“小组”中承担的奇偶校验任务有关

各检测位 C_i 所负责的检测小组

- C_1 检测 g_1 小组, 含位 **1**, **3**, 5, 7, 9, **11**, ... (XXXX**1**)
- C_2 检测 g_2 小组, 含位 **2**, **3**, 6, 7, 10, 11, ... (XXX**1**X)
- C_4 检测 g_3 小组, 含位 **4**, 5, 6, 7, 12, 13, ... (XX**1**XX)
- C_8 检测 g_4 小组, 含位 **8**, 9, 10, 11, 12, 13, 14, 15, 24, ... (X**1**XXX)

.....

- g_i 小组独占第 **2^{i-1}** 位
- g_i 和 g_j 小组共同占第 **$2^{i-1} + 2^{j-1}$** 位
- g_i , g_j 和 g_l 小组共同占第 **$2^{i-1} + 2^{j-1} + 2^{l-1}$** 位

例：求 0101 按 “偶校验” 配置的汉明码

解： $\because n = 4$ ，根据 $2^k \geq n + k + 1$
得 $k = 3$

汉明码排序：

位置	C_1 1	C_2 2	3	C_4 4	5	6	7
	0	1	0	0	1	0	1

$$C_1 = \text{位3} \oplus \text{位5} \oplus \text{位7} = 0$$

$$C_2 = \text{位3} \oplus \text{位6} \oplus \text{位7} = 1$$

$$C_4 = \text{位5} \oplus \text{位6} \oplus \text{位7} = 0$$

$\therefore$ 0101 的汉明码为 **0100101**

练习1解析 按偶校验配置 0011 的汉明码

∵ $n = 4$, 根据 $2^k \geq n + k + 1$
得 $k = 3$

汉明码排序:

位置	C_1	C_2		C_4			
	1	2	3	4	5	6	7
	1	0	0	0	0	1	1

$$C_1 = \text{位3} \oplus \text{位5} \oplus \text{位7} = 1$$

$$C_2 = \text{位3} \oplus \text{位6} \oplus \text{位7} = 0$$

$$C_4 = \text{位5} \oplus \text{位6} \oplus \text{位7} = 0$$

∴ 0011 的偶校验汉明码为 1000011

汉明码的校验过程

- 计算新的检测位 P_i ($i=1, 2^1, \dots, 2^{k-1}$, k 为检测位位数)

以 $k=3$ 为例, P_i 的取值为

$$P_1 = \overset{C_1}{\text{位1}} \oplus \text{位3} \oplus \text{位5} \oplus \text{位7}$$

$$P_2 = \overset{C_2}{\text{位2}} \oplus \text{位3} \oplus \text{位6} \oplus \text{位7}$$

$$P_4 = \overset{C_4}{\text{位4}} \oplus \text{位5} \oplus \text{位6} \oplus \text{位7}$$

对于按“偶校验”配置的汉明码

不出错时: P_i 都为 0

已知收到的汉明码为 0100111

（按偶校验配置）试问要求传送的信息是什么？

解：检错过程如下

$$P_1 = \text{位1} \oplus \text{位3} \oplus \text{位5} \oplus \text{位7} = 0$$

$$P_2 = \text{位2} \oplus \text{位3} \oplus \text{位6} \oplus \text{位7} = \mathbf{1}$$

$$P_4 = \text{位4} \oplus \text{位5} \oplus \text{位6} \oplus \text{位7} = \mathbf{1}$$

$$\therefore P_4 P_2 P_1 = 110$$

纠错过程：由 P_i 可知第6位出错，可纠正为 **0100101**

故要求传送的信息为 **0101**。

汉明码练习2

写出按偶校验配置的汉明码0101101 的校验过程。

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7 = 1$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7 = 0$$

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7 = 0$$

$$\therefore P_4 P_2 P_1 = 100$$

第 4 位错，可不纠错

练习3解析 按奇校验配置 0011 的汉明码

∵ $n = 4$, 根据 $2^k \geq n + k + 1$
得 $k = 3$

汉明码排序:

位置	C_1	C_2		C_4			
	1	2	3	4	5	6	7
	0	1	0	1	0	1	1

$$C_1 = \overline{\text{位3} \oplus \text{位5} \oplus \text{位7}} = 0$$

$$C_2 = \overline{\text{位3} \oplus \text{位6} \oplus \text{位7}} = 1$$

$$C_4 = \overline{\text{位5} \oplus \text{位6} \oplus \text{位7}} = 1$$

∴ 0011 的奇校验汉明码为 0101011

主存储器

1. 概述
2. 半导体存储芯片简介
3. 随机存储器（RAM: Random Access Memory）
4. 只读存储器（ROM: Read Only Memory）
5. 存储器与CPU的连接
6. 存储器校验
7. 存储器的加速

提高访问主存储器速度的方法

- 采用高性能存储芯片
 - SDRAM (同步 DRAM): 在系统时钟的控制下进行读出和写入, CPU 无须等待
 - RDRAM (Rambus DRAM) : 主要解决 存储器带宽 问题
 - 带 Cache 的 DRAM: 在 DRAM 的芯片内集成了一个由 SRAM 组成的 Cache, 有利于 猝发式读取
- 采用层次结构 Cache – 主存
- 优化主存结构

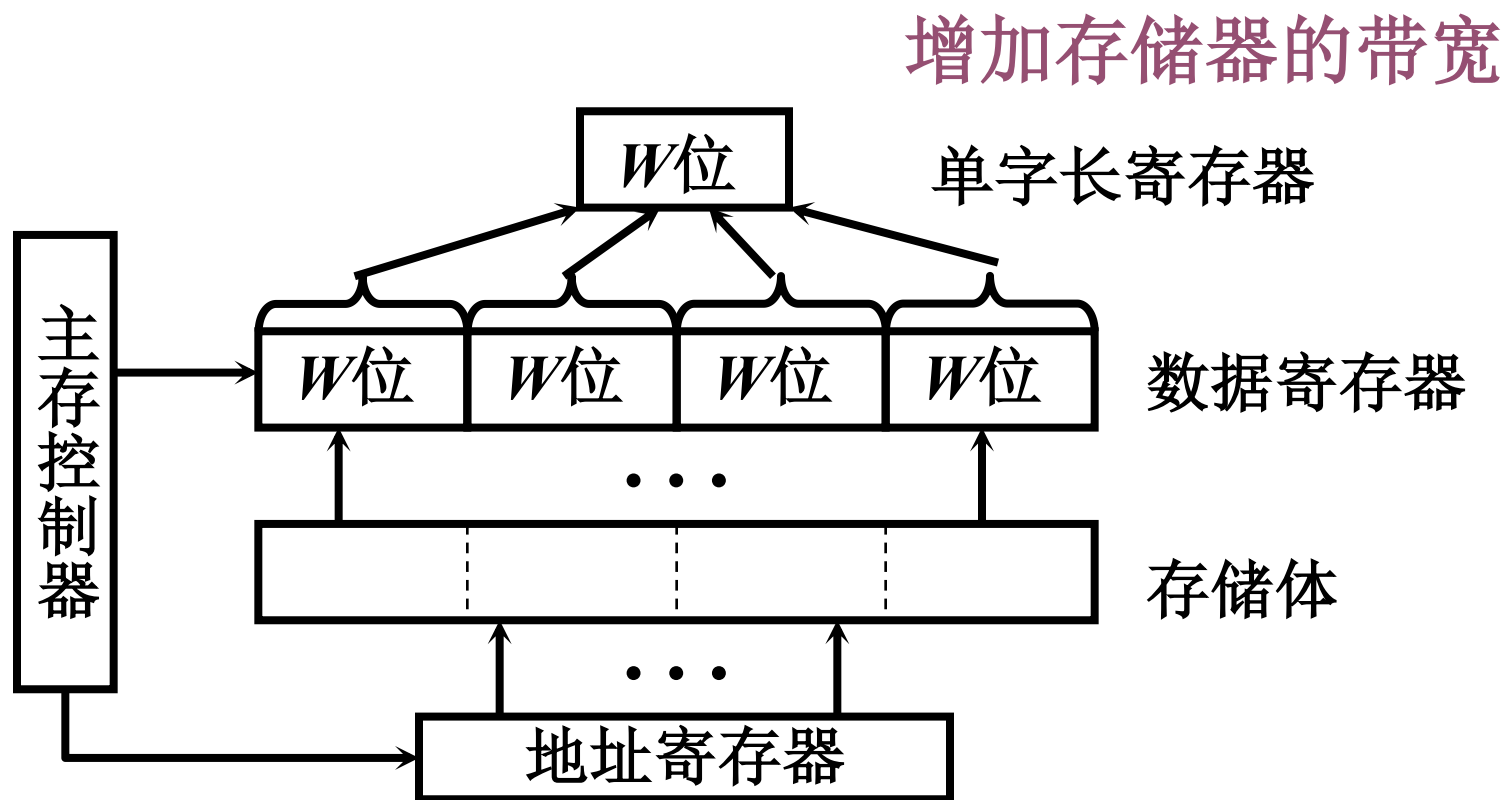
调整主存结构，提高访存速度

1. 单体多字系统

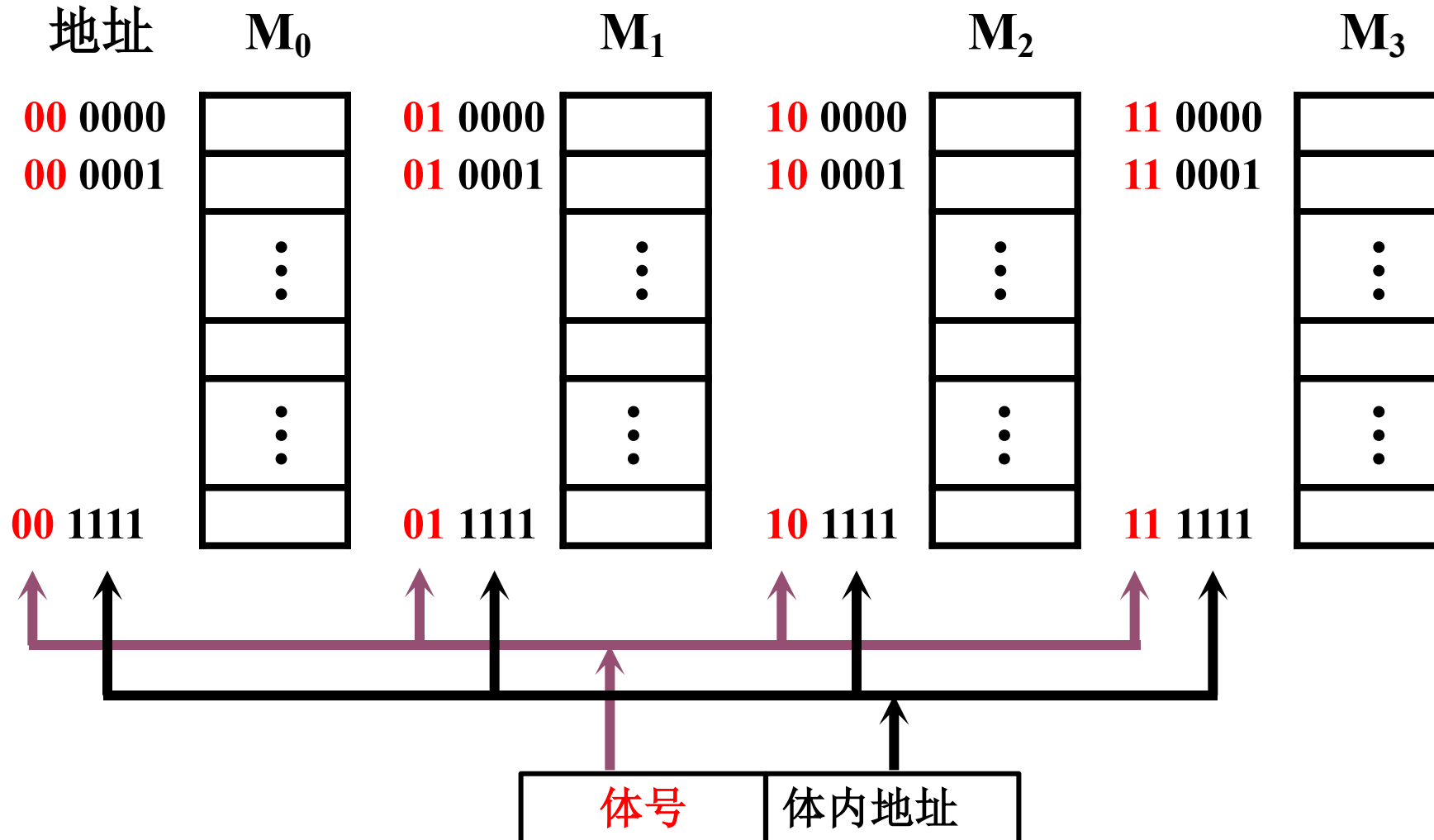
2. 多体并行系统

高位交叉

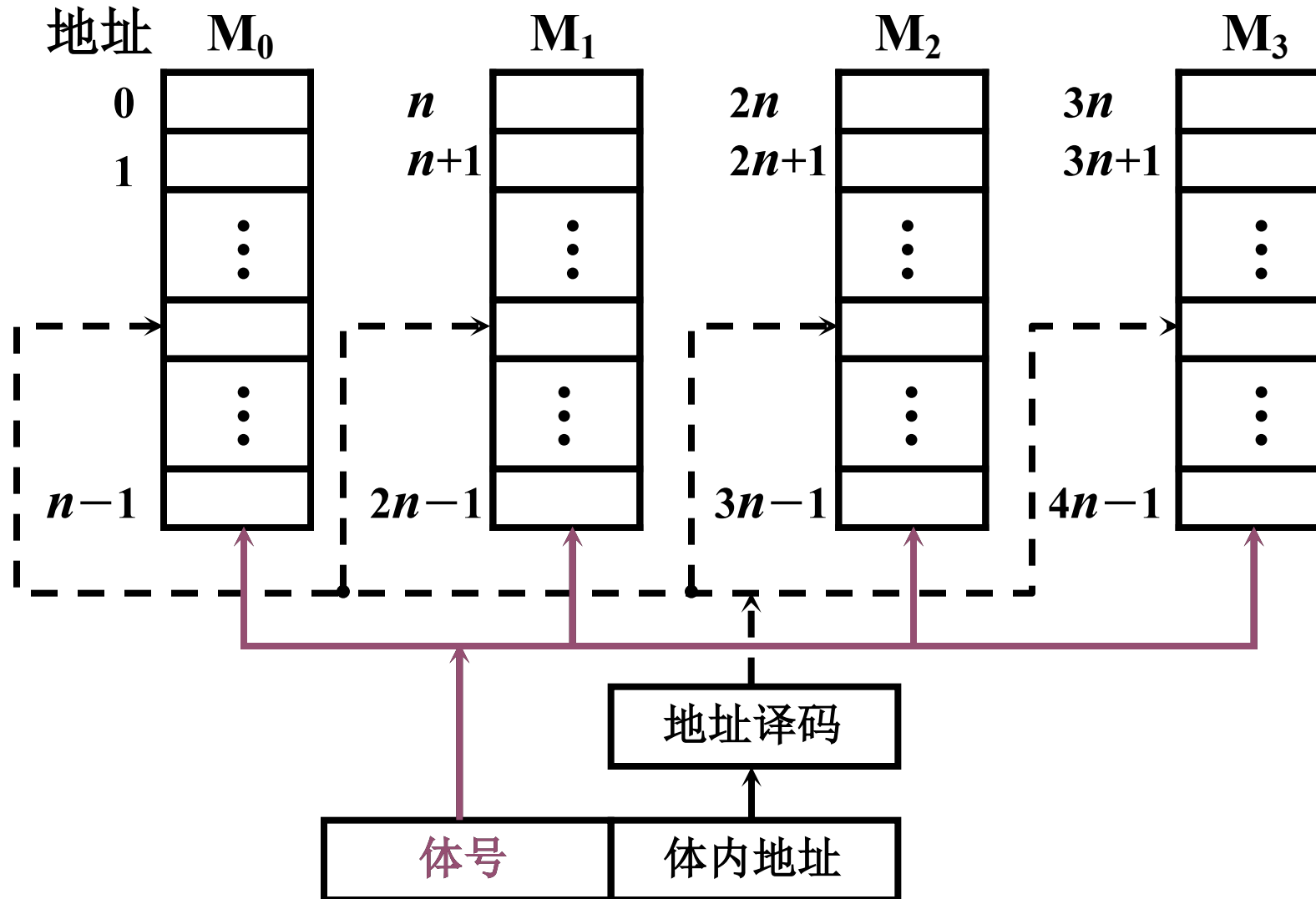
低位交叉



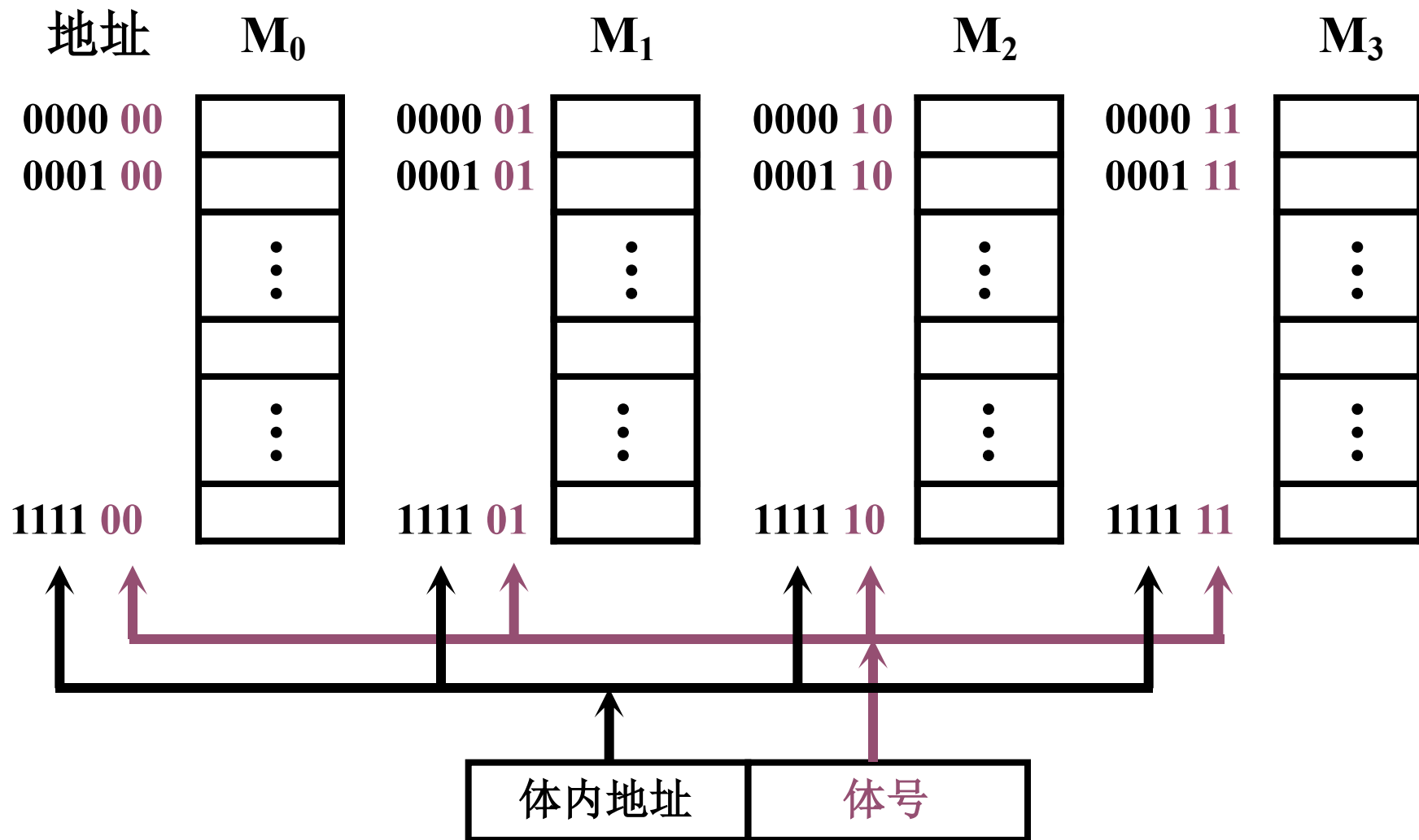
高位交叉编址多体存储器（顺序存储）



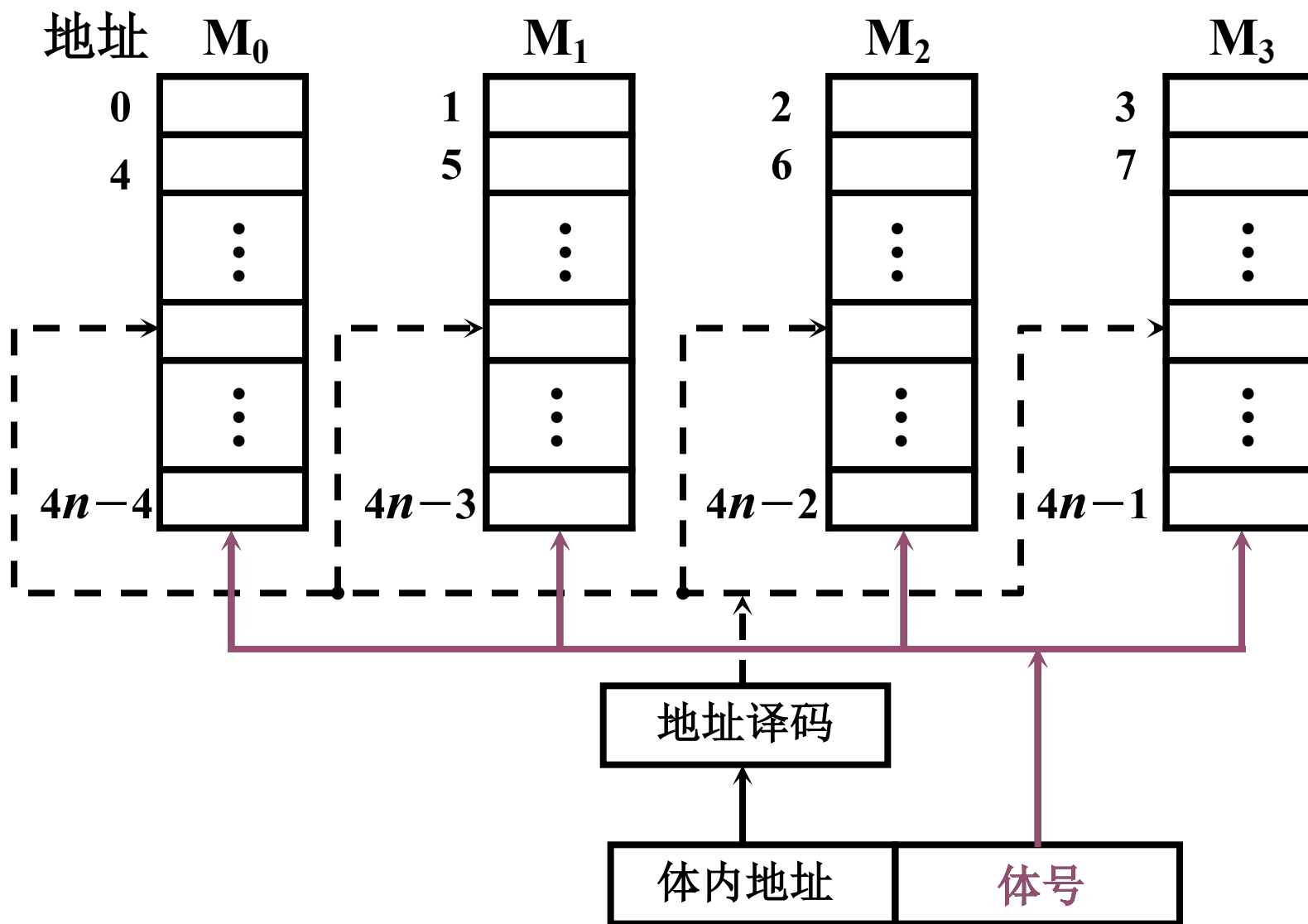
高位交叉编址——各个存储体可并行工作



低位交叉——各个体轮流编址

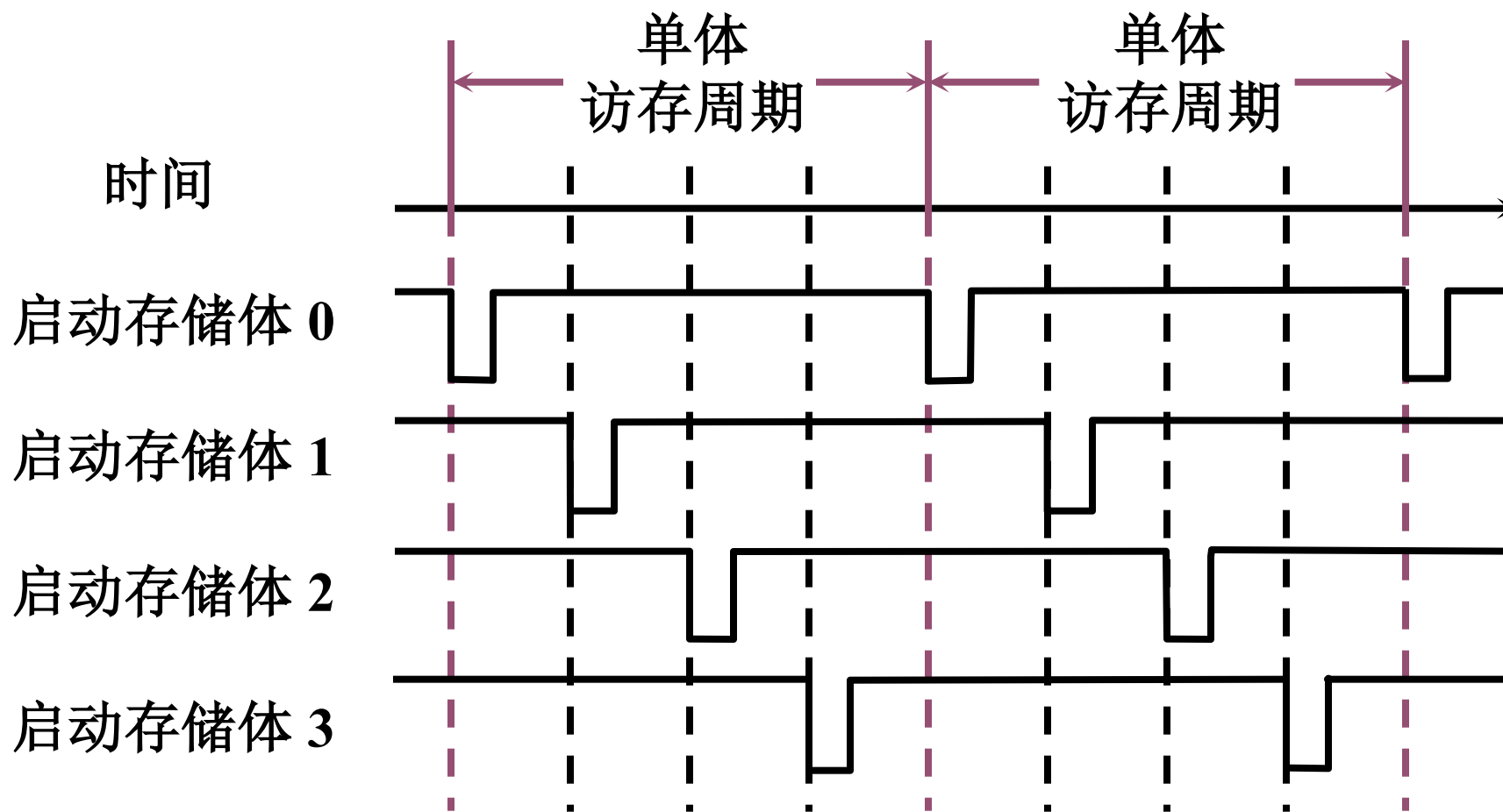


低位交叉——各个存储体轮流编址



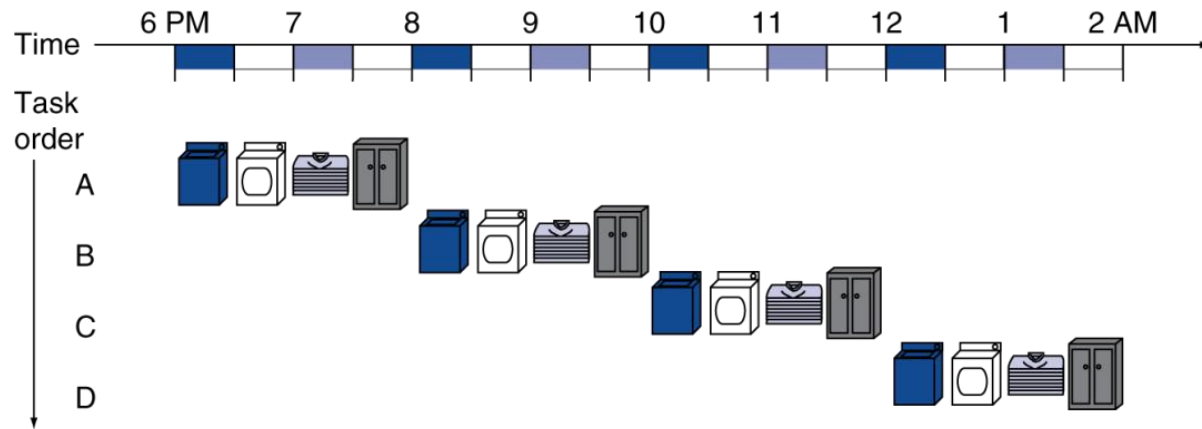
低位交叉编址的特点

在不改变存取周期的前提下，增加存储器的带宽

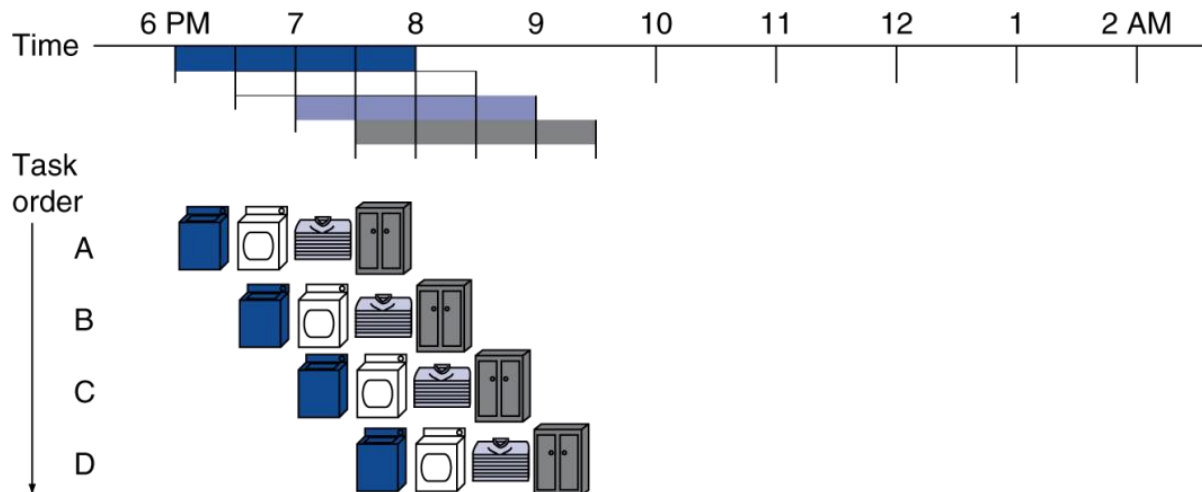


生活中的流水线

- 假设洗衣包括**四个**步骤：洗衣机中洗衣、烘干机中烘干、叠衣服、收纳到柜子中，每个步骤0.5小时。



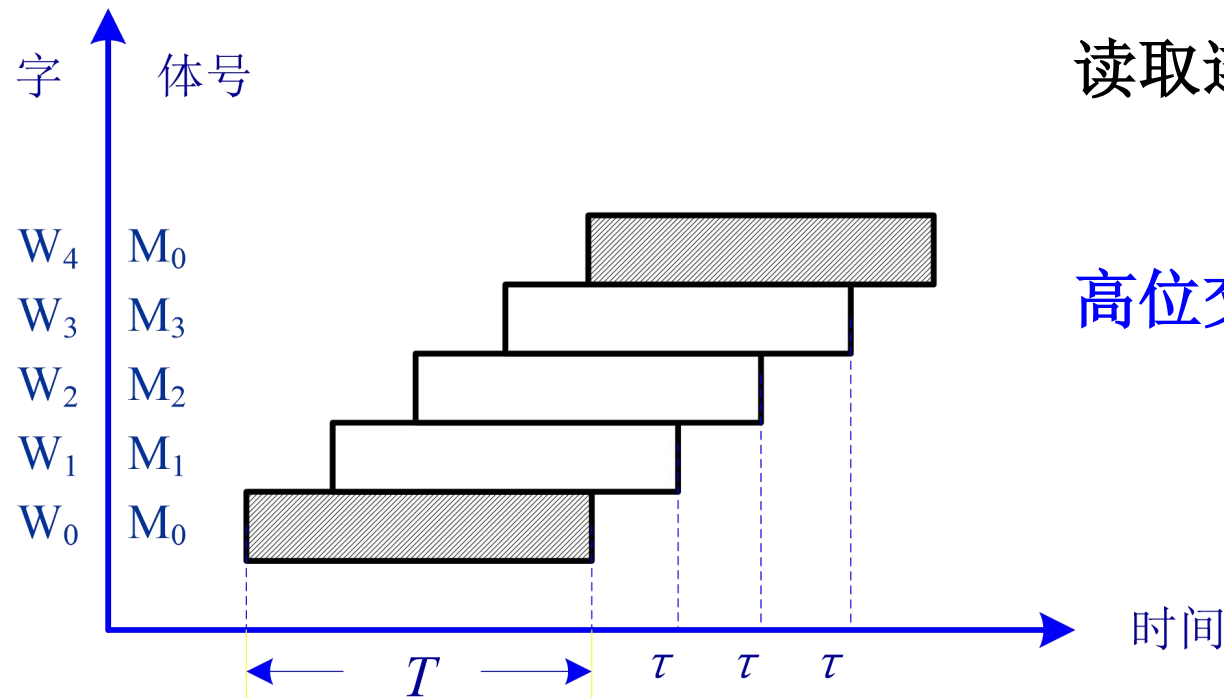
- 洗衣任务为4，**加速比**
 $= 2*4/(2+0.5*3) \approx 2.3$



- 洗衣任务数为n，加速比
 $= 2n/(2+0.5*(n-1))$
 $\approx 4 = \text{流水线中的步骤数}$

流水线方式存取

- 设 n 体低位交叉存储器，存取周期为 T ，总线传输周期为 τ （为实现流水线方式存取，应满足 $T = n\tau$ ）。
- （唐书例4.6中 $T=200\text{ns}$ ， $\tau=50\text{ns}$ ， $n=4$ ）



读取连续的 m 个字所需的时间为
 $T + (m - 1)\tau$

高位交叉存储读取连续 m 个字所需时间为
 mT

第六章 存储器

6.1 存储器概述

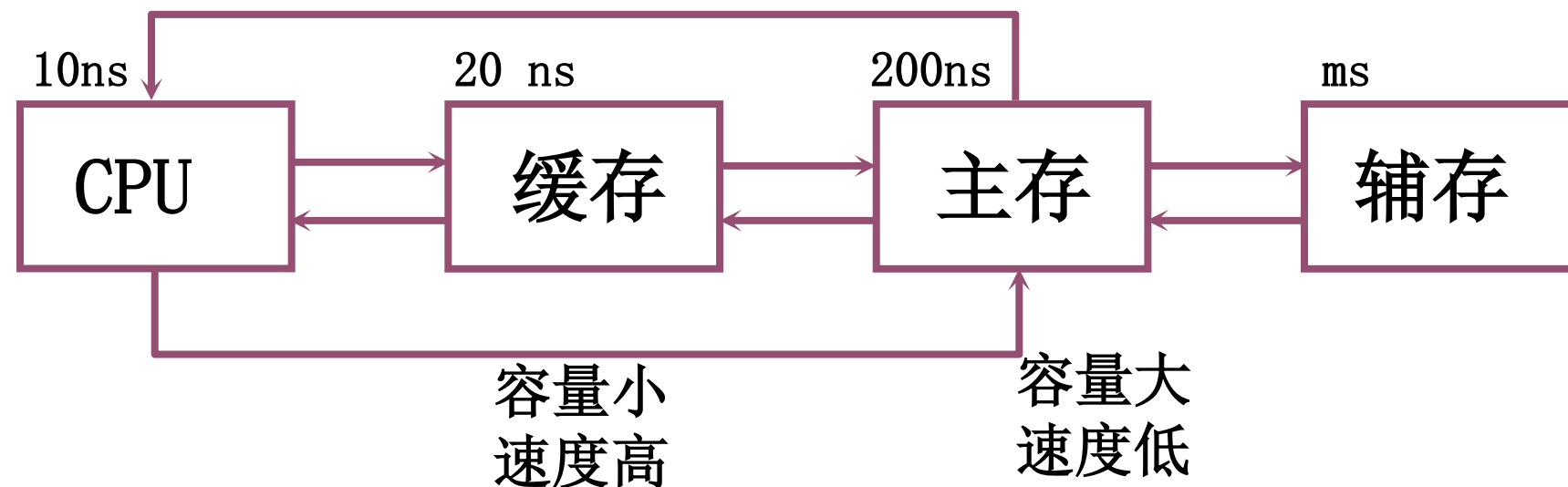
6.2 主存储器

6.3 高速缓冲存储器(Cache Memory)

6.4 虚拟存储

6.5 辅助存储器

高速缓冲存储器(Cache Memory)概述



CPU 和主存（DRAM）的速度差异

避免 CPU “空等” 现象

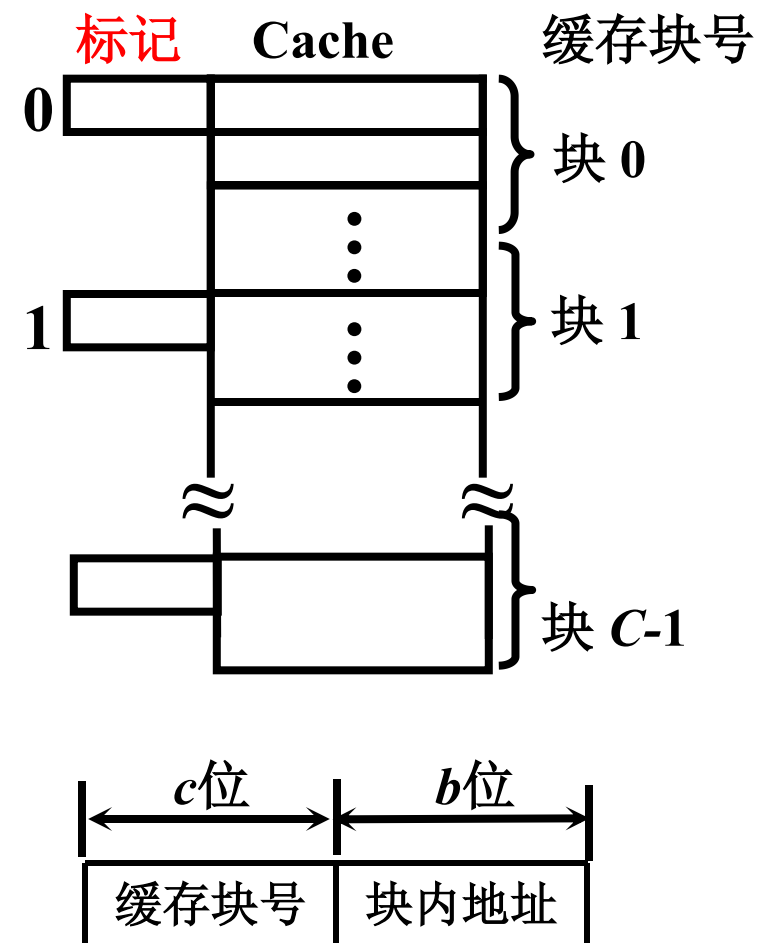
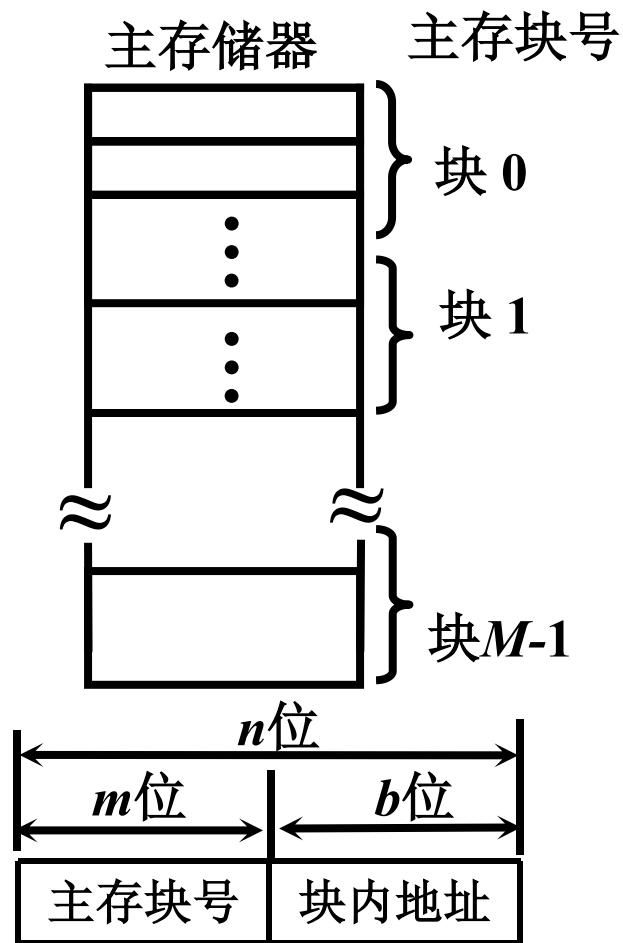
程序访问的局部性原理

Cache n.高速缓冲存储器;隐藏物(如武器);(秘密)贮存物;

vt.高速缓存;匿藏， 隐藏(尤指武器);把(数据)存入高速缓冲存储器

Cache 的工作原理——主存和缓存的编址

- 按块(Cache line)操作
- 块的大小相同(B)
 - $B=2^b$
 - Cache line size
- 主存块数 $M=2^m$
- 缓存块数 $C=2^c$
- $M \gg C$



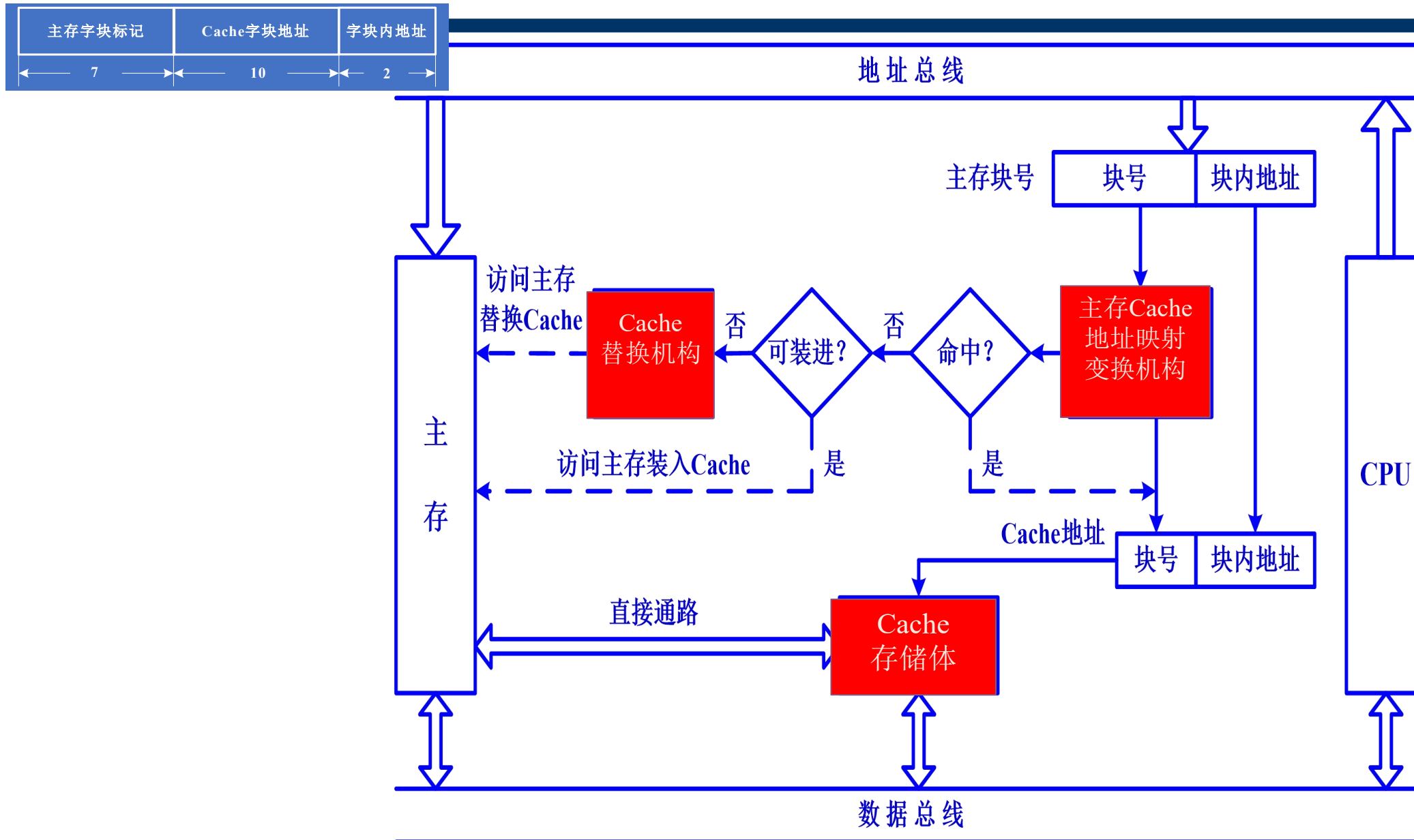
Cache 的工作原理——命中率

- 命中(hit): 主存块调入缓存——建立了对应关系
- 未命中(miss): 主存块未调入缓存——未建立对应关系
- 用 标记 记录与某缓存块建立了对应关系的主存块号
- 命中率(hit rate): 欲访问的信息在 Cache 中的 比率。
 - 与 Cache 容量、块长 等有关
 - 块长 (Cache line size) : 一个存取周期内从主存获取的信息长度(4~8个可编址单位——字/字节)

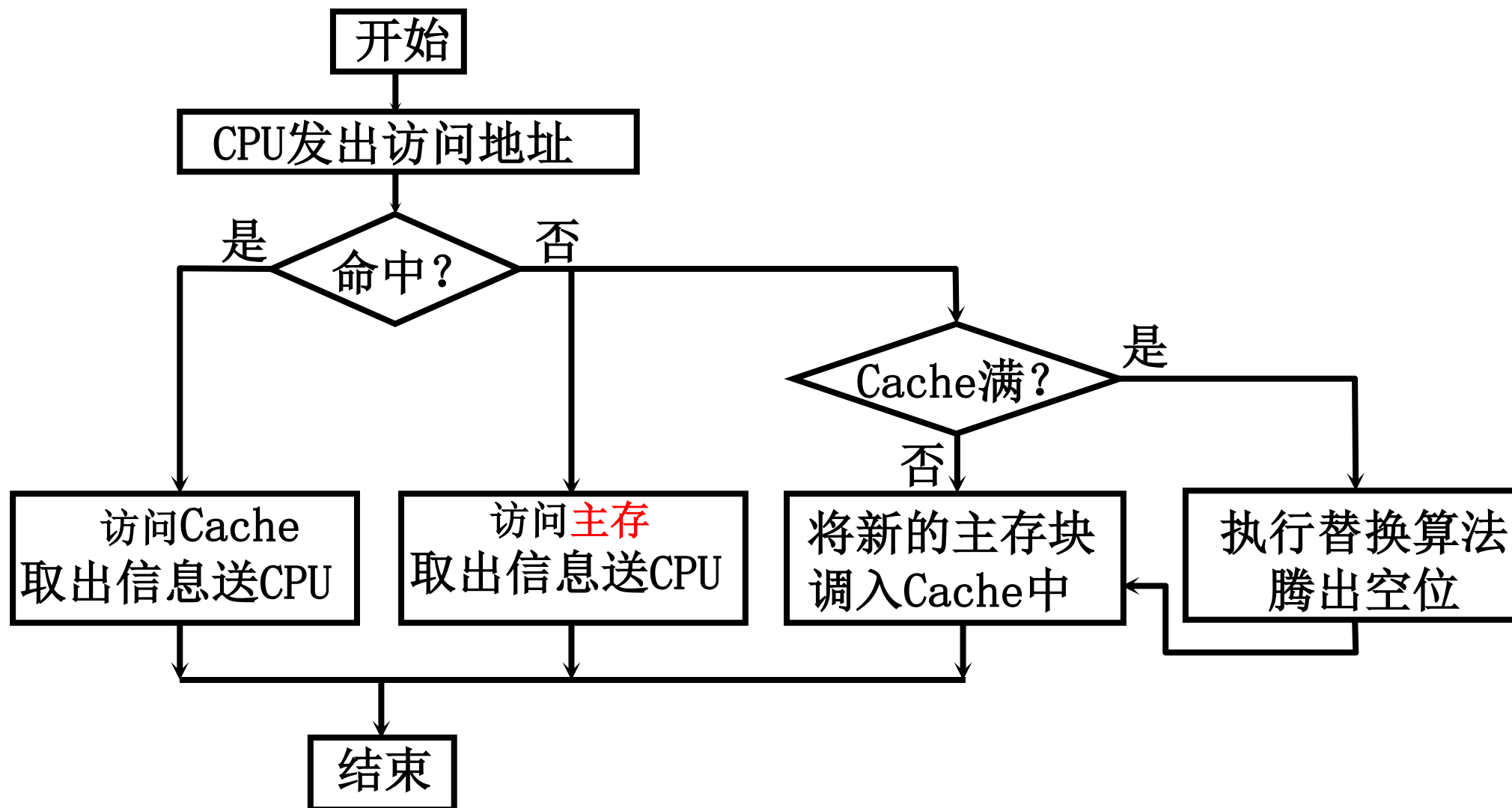
Cache –主存系统的效率

- 效率 e 与 命中率 有关
$$e = \frac{\text{访问 Cache 的时间}}{\text{平均访问时间}} \times 100\%$$
- 设命中率为 h ，访问Cache时间为 t_c ，访问主存时间为 t_m
则
$$e = \frac{t_c}{h \times t_c + (1-h) \times t_m} \times 100\%$$
- 若 $t_m=120ns$, $t_c=12ns$, 当 $h=99\%$ 时 $e=1/1.09$;
当 $h=90\%$ 时 $e=1/1.9 \approx 50\%$
- 自学例4.7

Cache 的工作原理——Cache 的基本结构



Cache Memory的读操作流程图



Cache 的 写 操作

- Cache 和主存的内容一致性
- **写直达法**(Write-through)
 - 同时写数据到Cache和主存
 - 写操作时间就是访问主存的时间
 - 读操作时不涉及对主存的写操作
 - 更新策略比较容易实现
- **写回法**(Write-back):写操作时只写入Cache而不写入主存
 - 写操作时间就是访问 Cache 的时间
 - 当 Cache 数据被替换出去时才写回主存
 - 读操作Cache失效发生数据替换时，被替换的块需写回主存，增加了Cache 的复杂性



Cache的改进

<https://docs.microsoft.com/zh-cn/sysinternals/downloads/coreinfo>

- 增加Cache的级数
 - 片内Cache、片外Cache
- 统一缓存和分立缓存
 - 指令Cache、数据Cache
- 与流水线有关

```
D:\GreenSW>coreinfo64/c
```

```
Coreinfo v3.52 - Dump information on system CPU and mem
Copyright (C) 2008-2021 Mark Russinovich
Sysinternals - www.sysinternals.com
```

```
Logical to Physical Processor Map:
```

```
**----- Physical Processor 0 (Hyperthreaded)
--**----- Physical Processor 1 (Hyperthreaded)
---**----- Physical Processor 2 (Hyperthreaded)
-----**---- Physical Processor 3 (Hyperthreaded)
-----**---- Physical Processor 4 (Hyperthreaded)
-----**---- Physical Processor 5 (Hyperthreaded)
```

```
C:\> 选择 C:\WINDOWS\system32\cmd.exe
```

```
D:\GreenSW>coreinfo64 -l
```

```
Coreinfo v3.52 - Dump information on system CPU and memory topology
Copyright (C) 2008-2021 Mark Russinovich
Sysinternals - www.sysinternals.com
```

```
Logical Processor to Cache Map:
```

**-----	Data Cache	0, Level 1,	32 KB, Assoc	8, LineSize	64
**-----	Instruction Cache	0, Level 1,	32 KB, Assoc	8, LineSize	64
**-----	Unified Cache	0, Level 2,	256 KB, Assoc	4, LineSize	64
*****	Unified Cache	1, Level 3,	12 MB, Assoc	16, LineSize	64
--**-----	Data Cache	1, Level 1,	32 KB, Assoc	8, LineSize	64
--**-----	Instruction Cache	1, Level 1,	32 KB, Assoc	8, LineSize	64
--**-----	Unified Cache	2, Level 2,	256 KB, Assoc	4, LineSize	64
----**-----	Data Cache	2, Level 1,	32 KB, Assoc	8, LineSize	64
----**-----	Instruction Cache	2, Level 1,	32 KB, Assoc	8, LineSize	64
----**-----	Unified Cache	3, Level 2,	256 KB, Assoc	4, LineSize	64
-----**----	Data Cache	3, Level 1,	32 KB, Assoc	8, LineSize	64
-----**----	Instruction Cache	3, Level 1,	32 KB, Assoc	8, LineSize	64
-----**----	Unified Cache	4, Level 2,	256 KB, Assoc	4, LineSize	64
-----**----	Data Cache	4, Level 1,	32 KB, Assoc	8, LineSize	64
-----**----	Instruction Cache	4, Level 1,	32 KB, Assoc	8, LineSize	64
-----**----	Unified Cache	5, Level 2,	256 KB, Assoc	4, LineSize	64
-----**----	Data Cache	5, Level 1,	32 KB, Assoc	8, LineSize	64
-----**----	Instruction Cache	5, Level 1,	32 KB, Assoc	8, LineSize	64
-----**----	Unified Cache	6, Level 2,	256 KB, Assoc	4, LineSize	64

Cache – 主存的地址映射

- 直接映射
- 全相联映射
- 组相联映射

```
C:\WINDOWS\system32\cmd.exe
D:\GreenSW>coreinfo64 -l

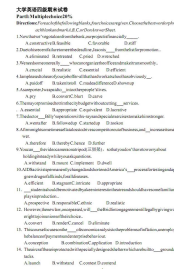
Coreinfo v3.52 - Dump information on system CPU and memory topology
Copyright (C) 2008-2021 Mark Russinovich
Sysinternals - www.sysinternals.com

Logical Processor to Cache Map:
**----- Data Cache 0, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Instruction Cache 0, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Unified Cache 0, Level 2, 256 KB, Assoc 4, LineSize 64
***** Unified Cache 1, Level 3, 12 MB, Assoc 16, LineSize 64
**----- Data Cache 1, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Instruction Cache 1, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Unified Cache 2, Level 2, 256 KB, Assoc 4, LineSize 64
**----- Data Cache 2, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Instruction Cache 2, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Unified Cache 3, Level 2, 256 KB, Assoc 4, LineSize 64
**----- Data Cache 3, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Instruction Cache 3, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Unified Cache 4, Level 2, 256 KB, Assoc 4, LineSize 64
**----- Data Cache 4, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Instruction Cache 4, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Unified Cache 5, Level 2, 256 KB, Assoc 4, LineSize 64
**----- Data Cache 5, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Instruction Cache 5, Level 1, 32 KB, Assoc 8, LineSize 64
**----- Unified Cache 6, Level 2, 256 KB, Assoc 4, LineSize 64
```

例：试卷查阅

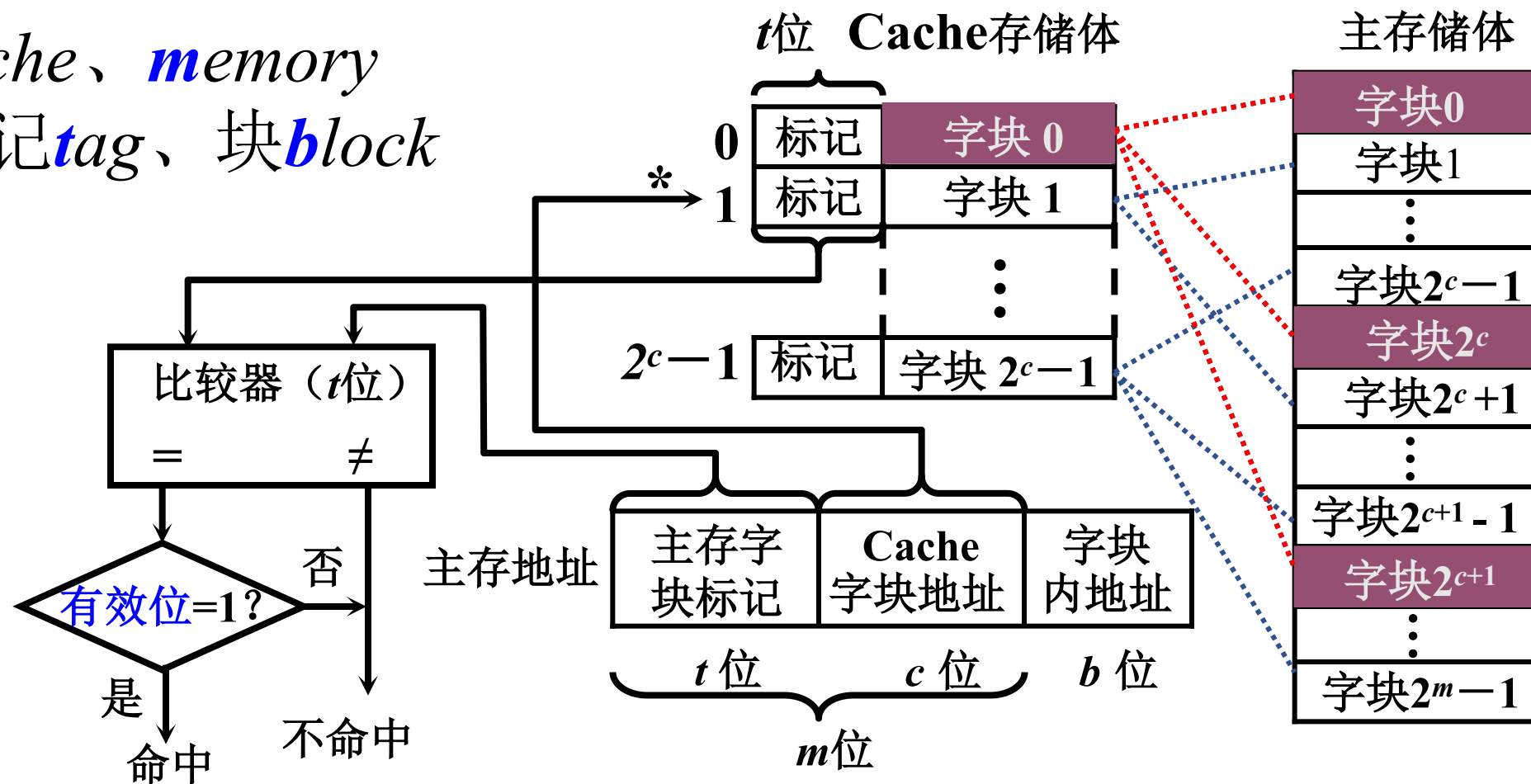
- 档案室（假设有64个小班，已排序）——>主存
- 每个小班一个试卷档案袋——>块
- 桌子（可放8袋试卷）——>缓存

1 1,9,17...	2 2,10,18...	3 3,11,19...	4 4,12,...
5,13.... 5	6,14... 6	7,15... 7	8,16,32... 8



Cache-主存直接映射

cache、*memory*
标记*tag*、块*block*

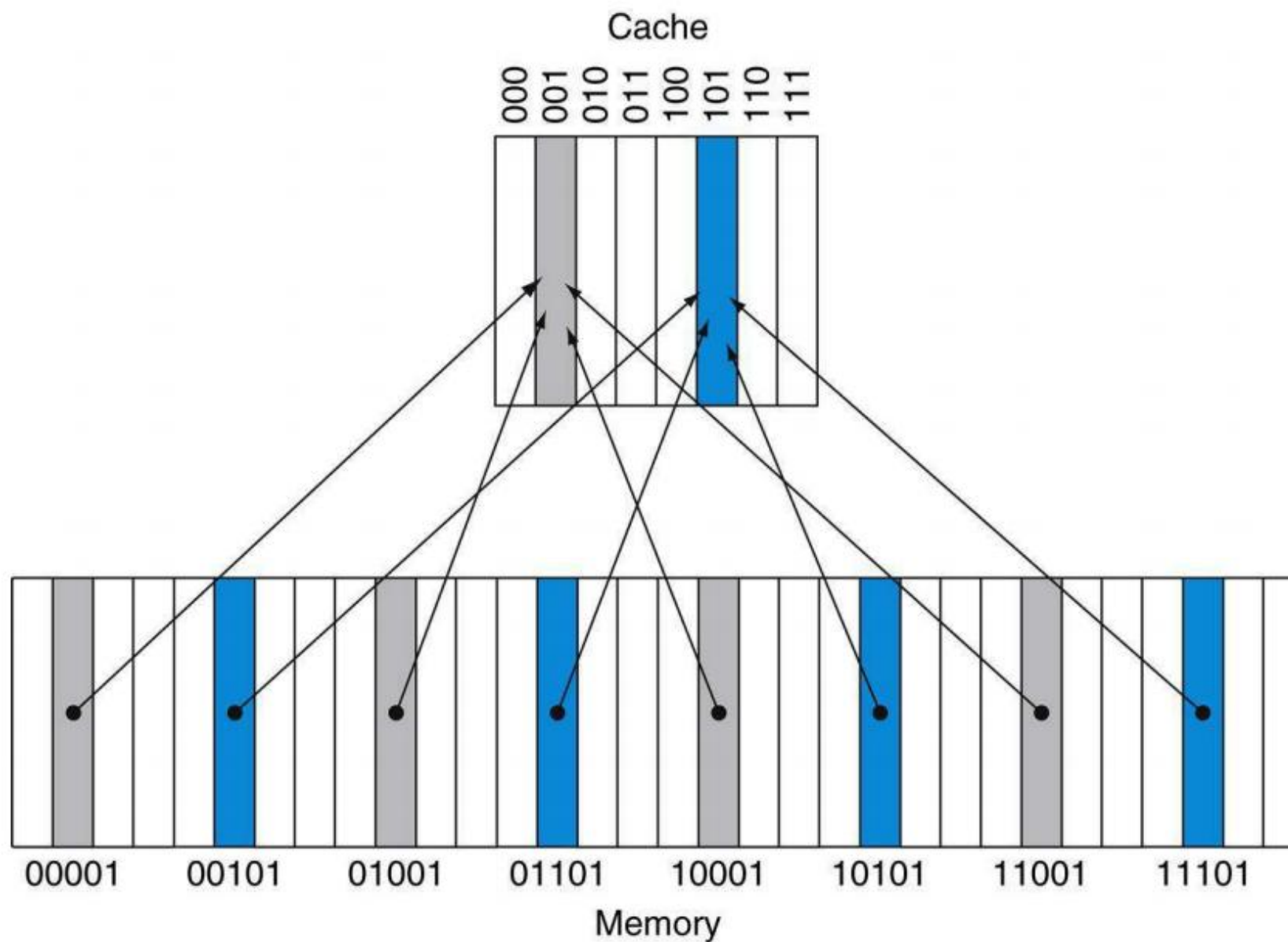


每个缓存块 i 可以和 若干 个 主存块 对应

每个主存块 j 只能和 一个 缓存块 对应

$$i = j \bmod C$$

黑书图5-8 直接映射示例



黑书图5-9

- 下面是对一个大小为8个数据块的空cache进行9次存储访问的操作序列，包括每次存储访问的具体操作。

访存地址 (十进制表示)	访存地址 (二进制表示)	cache命中 或失效	分配的cache数据块 (存放数据的位置)
22	10110_2	失效 (图5-9b)	$(10110_2 \bmod 8) = 110_2$
26	11010_2	失效 (图5-9c)	$(11010_2 \bmod 8) = 010_2$
22	10110_2	命中	$(10110_2 \bmod 8) = 110_2$
26	11010_2	命中	$(11010_2 \bmod 8) = 010_2$
16	10000_2	失效 (图5-9d)	$(10000_2 \bmod 8) = 000_2$
3	00011_2	失效 (图5-9e)	$(00011_2 \bmod 8) = 011_2$
16	10000_2	命中	$(10000_2 \bmod 8) = 000_2$
18	10010_2	失效 (图5-9f)	$(10010_2 \bmod 8) = 010_2$
16	10000_2	命中	$(10000_2 \bmod 8) = 000_2$

每次访问后cache中内容的变化

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

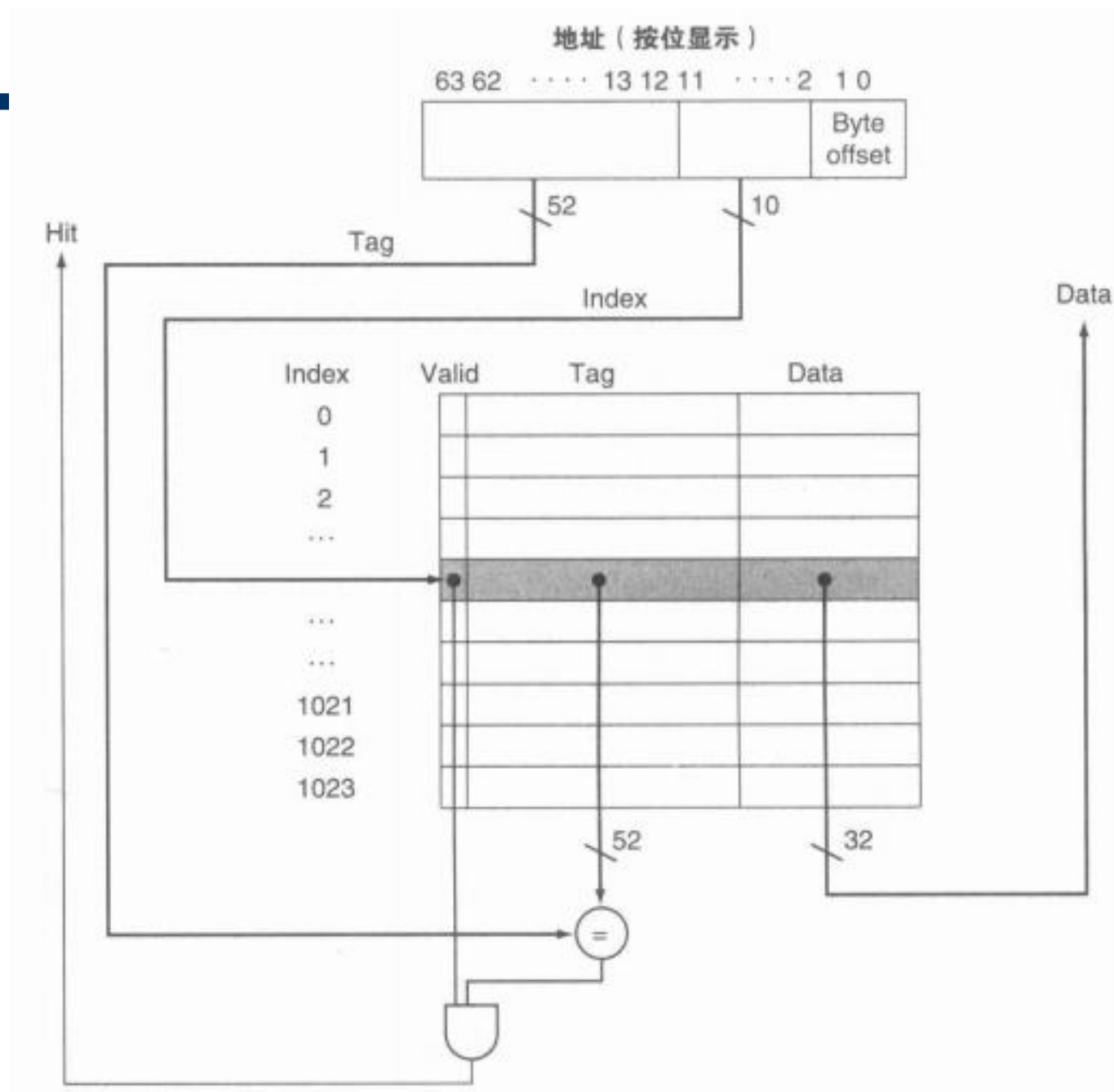
a) 上电后的cache初始状态

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_2	Memory (11010_2)
011	N		
100	N		
101	N		
110	Y	10_2	Memory (10110_2)
111	N		

c) 地址 (11010_2) 访问失效处理后

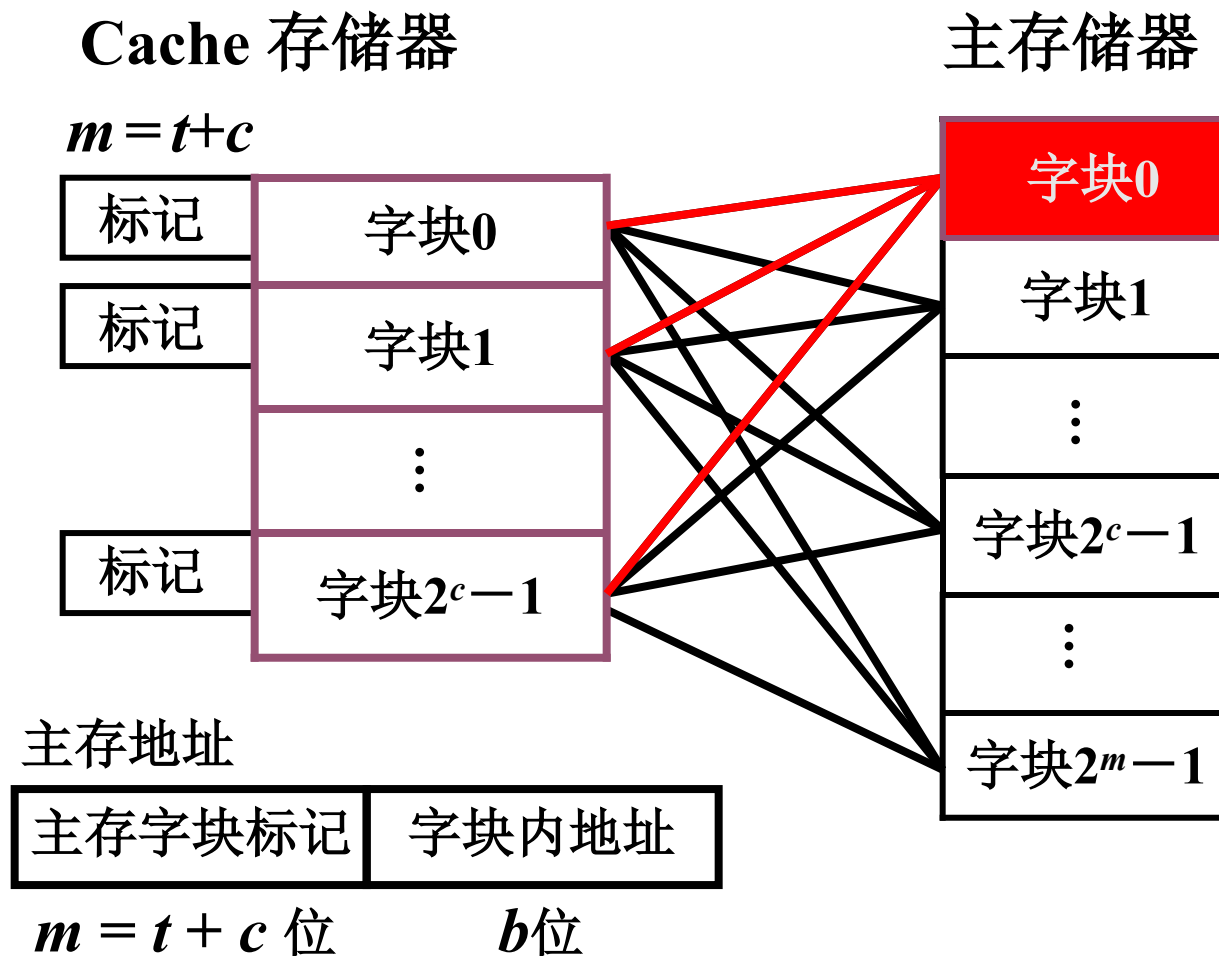
黑书图5-10

- 块大小：4字节
- 64位地址
- 直接映射

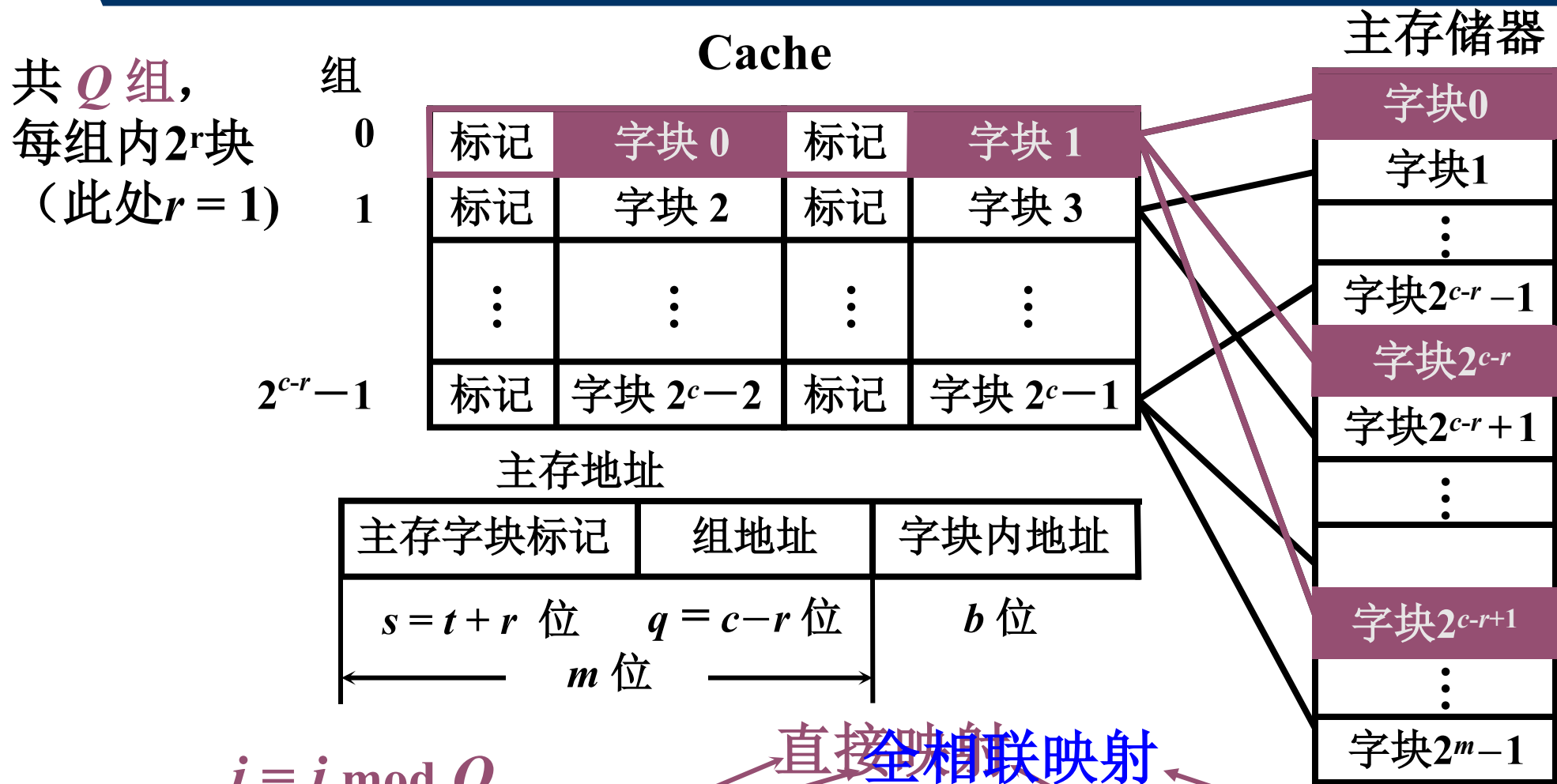


Cache-主存全相联映射

主存 中的 任一块 可以映射到 缓存 中的 任一块



Cache-主存组相联映射



$$i = j \bmod Q$$

某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

Cache-主存映射方式小结

- 主存地址划分如下表，其中：块内地址相同
- Cache块（line）地址 $\approx$ Cache组地址
- 通过主存块标记与Cache块标记的内容比对判断是否命中
- Cache块标记是**Cache内容的一部分**
- 有Cache时，主存储器的结构会变得复杂

映射方式	t位	r位	c-r位	b位	备注
全相联映射	主存块标记			块内地址	主存块—>Cache任一块
直接映射	主存块标记	Cache块地址		块内地址	模 2^c ，标记t位
组相联映射	主存块标记		组地址	块内地址	每组 2^r 路,模 2^{c-r} ,标记t+r位

例（唐书例4.8）

假设主存容量为512KB，Cache容量为4KB，每个块为 16 个字，**每个字32位**。

1)Cache可容纳多少块？

块大小为： $32\text{bit} \times 16 = 64\text{B}$ ； Cache可容纳 $4\text{KB} / (4\text{B} \times 16) = \mathbf{64}$ 块

2)主存地址有多少位？可容纳多少块？

512KB—>主存地址**19**位。 $512\text{KB} / (4\text{B} \times 16) = \mathbf{8192}$ 块。

3) 在直接映射方式下，主存的第几块映射到 Cache 中的第5块（设起始字块为第**1**块）？

主存的第 **5**， $64 + \mathbf{5}$ ， $2 \times 64 + \mathbf{5}$ ， ... ， $8192 - 64 + \mathbf{5}$ 块

例（唐书例4.8）——续

(主存512KB，Cache 4KB，每个字块16个字，每个字32位)

(4) 画出直接映射方式下主存地址字段中各段的位数

(主存字块标记位数 t +缓存字块地址位数 c +字块内地址位数 b)

字块内地址位数 $b = 6$ （字块大小为：4B*16=64B）

缓存共 64 块，故缓存字块地址位数 $c = 6$

主存字块标记为主存地址与Cache地址长度之差

$$t = m - c = 19 - 12 = 7$$



例（唐书例4.9）

- 假设主存容量 $512\text{K} \times 16$ 位，Cache 容量 4096×16 位，每个块为4个**16位的字**，访存地址为**字地址**。

(1) 在直接映射方式下，设计主存的地址格式。

根据 Cache 容量为 4K字，得 **Cache 地址为 12 位**。

根据块长为 4，按字访问，得字块内地址 2 位，

Cache块数 = $4\text{K}/4 = 1024 \rightarrow$ Cache字块地址位数 $c=10$ 。

根据主存容量为 512K字，得主存字地址 **19 位**。



例（唐书例4.9）——续

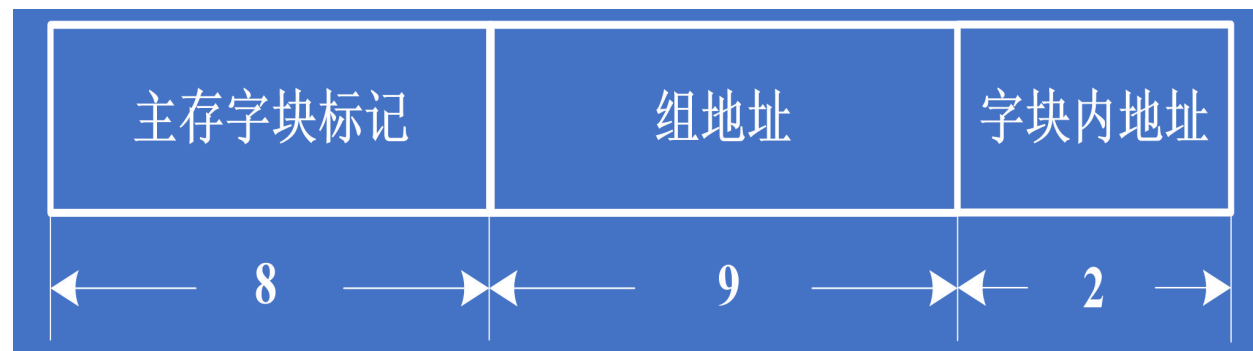
- 假设主存容量 $512\text{K} \times 16$ 位，Cache 容量 4096×16 位，每个块为4个16位的字，访存地址为**字地址**。

2) 在全相联映射方式下，设计主存的地址格式。

字块内地址 **2** 位，其余 **17** 位为主存字块标记

3) 在二路组相联映射方式下，设计主存的地址格式。

一组内有 **2** 块，Cache有**1024**块 ——>组地址 **9** 位



练习解析

- 假设主存容量 $512\text{K} \times 32$ 位，Cache 容量 4096×16 位，每个块为4个16位的字，访存地址为字地址。

在四路组相联映射方式下，设计主存的地址格式。

解：每块4个字—>字块内地址为2位

主存容量 $512\text{K} \times 32$ 位—> $1\text{M} \times 16$ 位 —>主存地址 20 位

四路组相联，一组内4块，Cache共有 $1024/4 = 256$ 组—>8位



6.3 高速缓冲存储器——缓存替换算法

- 先进先出法（FIFO: First In First Out）
- 近期最少使用法（LRU: Least Recently Used）
- 最不经常使用法（LFU: Least Frequently Used）
- 随机替换法

高速缓冲存储器——小结

- **实际应用：** 块设备缓存、硬盘缓存、web cache...

不灵活

直接映射 某一主存块只能固定映射到某一缓存块

全相联映射 某一主存块能映射到任一缓存块

组相联映射 某一主存块只能映射某一缓存组中的任一块

成本高

充分利用局部性原理

时间局部性:

```
for(loop=0; loop<10; loop++){  
    for (i=0; i<N; i++) {  
        ... = ... x[i] ...  
    }  
}
```

空间局部性

```
for i=1..n  
    for j=1..n  
        for k=1..n  
            c[i, j] += a[i, k]*b[k, j]
```

```
for (i=0; i<N; i++) {  
    for (loop=0; loop<10; loop++) {  
        ... = ... x[i] ...  
    }  
}
```

```
for i=1..n  
    for k=1..n  
        for j=1..n  
            c[i,j] += a[i,k]*b[k,j]
```

第六章 存储器

6.1 存储器概述

6.2 主存储器

6.3 高速缓冲存储器

6.4 虚拟存储

6.5 辅助存储器

虚拟存储（VM: Virtual Memory）

- “640K ought to be enough for anybody” 比尔·盖茨？ (1981)
- 主存容量限制带来的诸多不便
 - 用户编写程序必须考虑主存容量的限制
 - 多道程序设计的道数受到限制
- 用户编程行为分析
 - 全面考虑各种情况，执行时有互斥性
 - 顺序性和循环性等空间局部性行为
 - 某一阶段执行的时间局部性行为
- 因此可以考虑部分调入进程内容

虚拟存储的基本思想

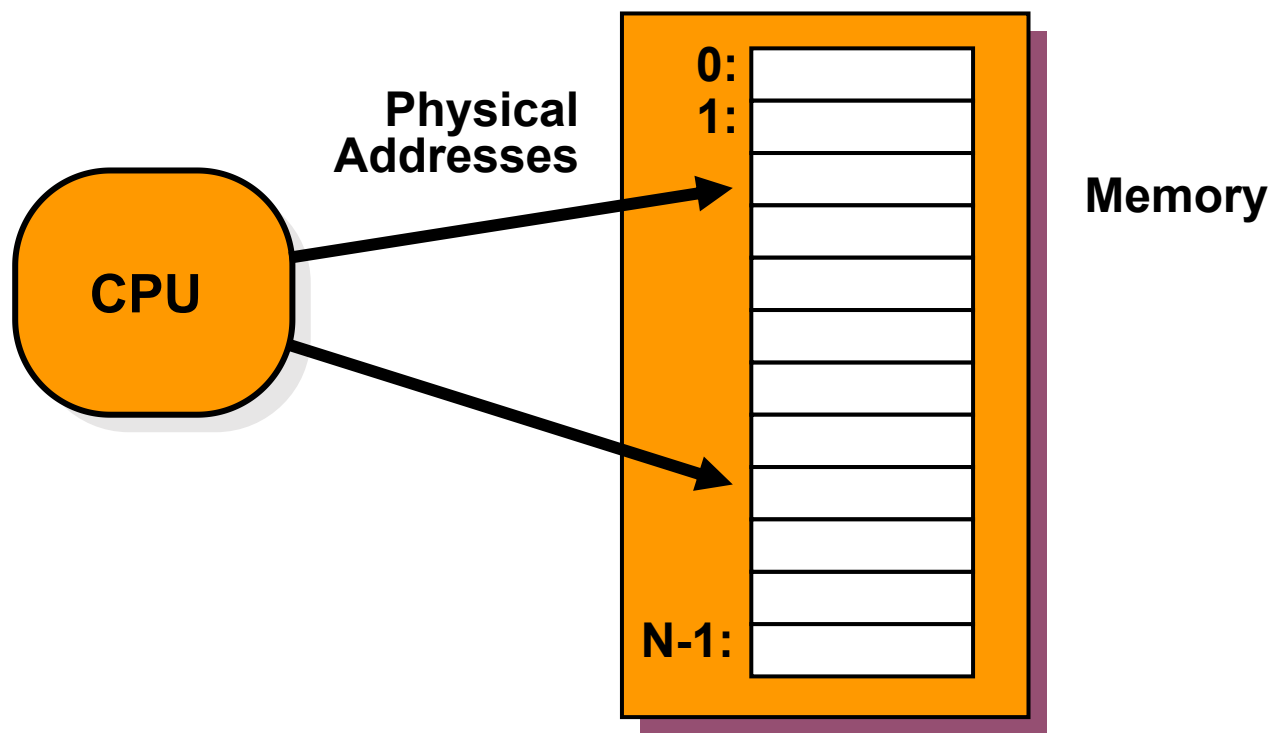
- 存储管理把进程全部信息放在辅存中，执行时先将其中一部分装入主存，以后根据执行行为**随用随调入**
- 如果主存中没有足够的空间，存储管理需要根据执行行为把主存中暂时不用的信息调出到辅存上去
- 虚拟存储采用与Cache类似原理，提高速度，降低成本
- 利用程序局部性，使存储系统的性能接近于高速存储器；价格接近于低速存储器，并扩充主存容量

虚拟存储的实现思路

- 需要建立与自动管理两个地址空间
 - （辅存）虚拟地址空间容纳进程装入
 - （主存）实际地址空间承载进程执行
- 对于用户，计算机系统具有一个容量大得多的主存空间，即虚拟存储，虚存空间是靠辅存（磁盘）来支持的
- 虚拟存储是一种地址空间扩展技术，通常意义上对用户编程是透明的，除非用户需要进行高性能的程序设计。

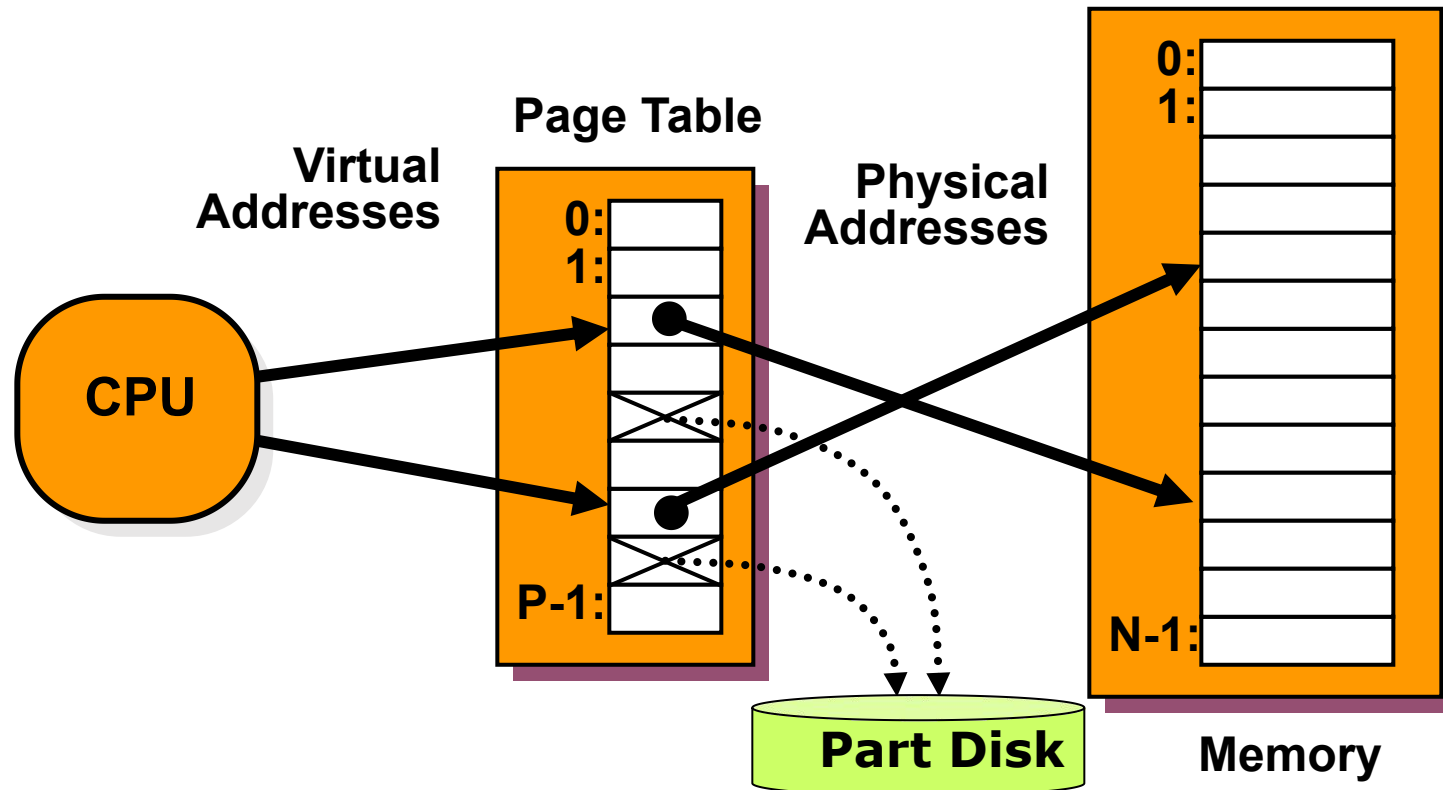
实地址计算机系统

- CPU给出的地址直接访问物理内存
 - 大多数Cray计算机，早期PC，大多数嵌入式系统



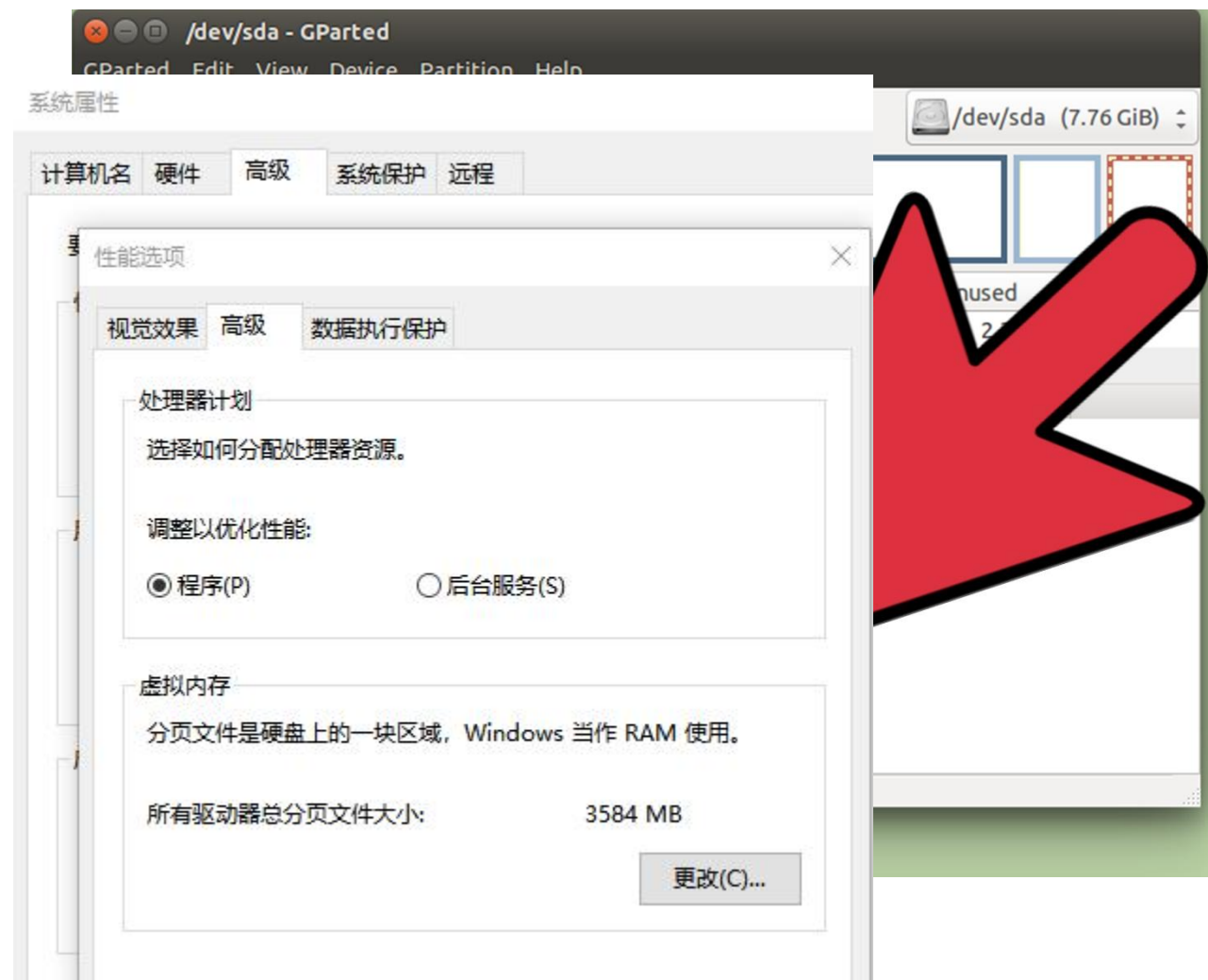
虚地址计算机系统

- CPU地址需要虚实变换
 - 硬件通过OS维护的页表将虚拟地址转换为物理地址
- 工作站，服务器，现代PC



操作系统中的虚拟存储

- Linux
 - 交换分区swap
- Windows
 - 虚拟内存（页面文件）

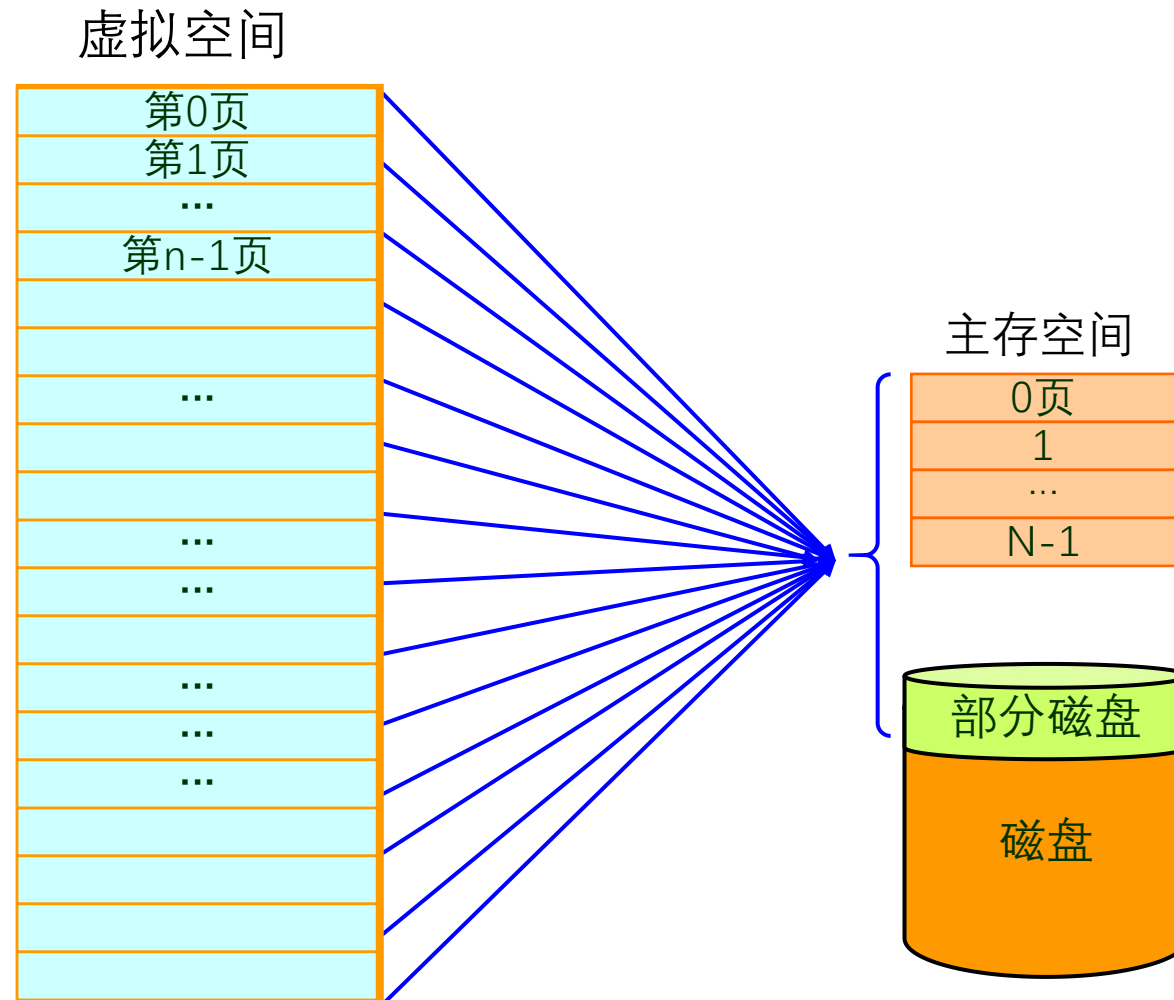


TLB——快表提高页式管理速度

- 页表存放在主存，虚存空间即使命中也须先查页表，再访问主存，两次访存才能取出数据，速度慢
- 为提高页表查找速度，采用cache的方法，引入一个体积小、快表，存放页表中经常被访问的表项
- **TLB(Translation Lookaside Buffer): 快表，页表缓冲**
- CPU寻址时，使用虚拟地址，优先在TLB中寻址，TLB负责将虚拟内存地址翻译成实际的物理内存地址，从而可**加速主存和辅存间访问速度**

虚拟存储

- 页式虚拟地址空间



虚拟存储与Cache的相似之处

- 将程序中常用的部分驻留在高速存储器
 - 程序载入按需载入（不是全部载入）
 - 高速存储器分块或者分页（粒度问题）
 - 主存空间满，将不常用程序或数据淘汰或交换到辅存中
- 数据的换入换出由硬件或操作系统完成，对用户透明
- 利用程序局部性，使存储系统的性能接近于高速存储器；
价格接近于低速存储器，并扩充主存容量

虚拟存储与Cache的差异

- 虚存用于扩大主存容量，Cache用于加速主存性能
- 虚存中未命中性能损失远大于Cache系统
 - 全相联提升命中率
 - 更大的交换单位页
 - CLOCK算法(一种近似LRU算法)
- 虚存由硬件和OS联合管理，Cache由硬件管理

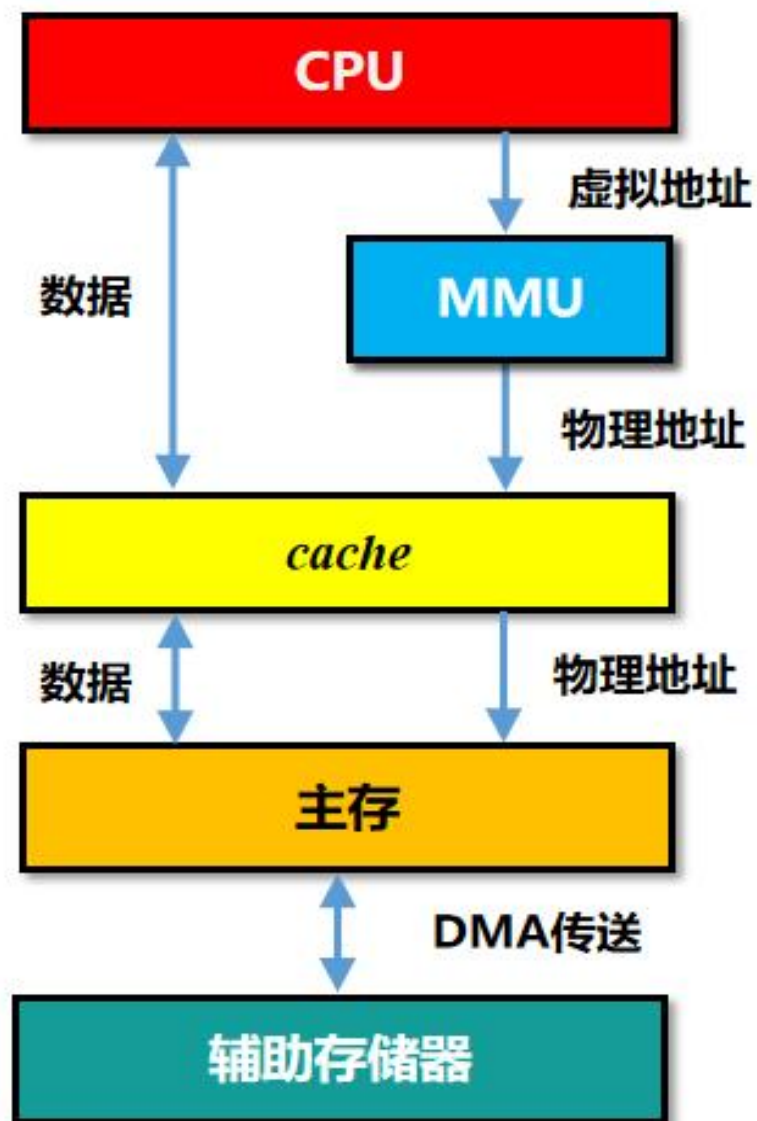
虚拟存储小结

- 程序员在比实际主存大得多的逻辑地址空间中编程
- 程序执行时，按需载入代码和数据，无需全部载入
- 硬件将逻辑地址（虚拟地址）转化为物理地址（实地址）
- 缺页时，由操作系统进行主存和磁盘之间的信息交换
- 虚存机制由硬件与操作系统协作实现
 - 进程及进程上下文切换、存储器分配
 - 虚拟地址空间管理、缺页处理
 - 存储器保护

存储层次结构

MMU: Memory Management Unit

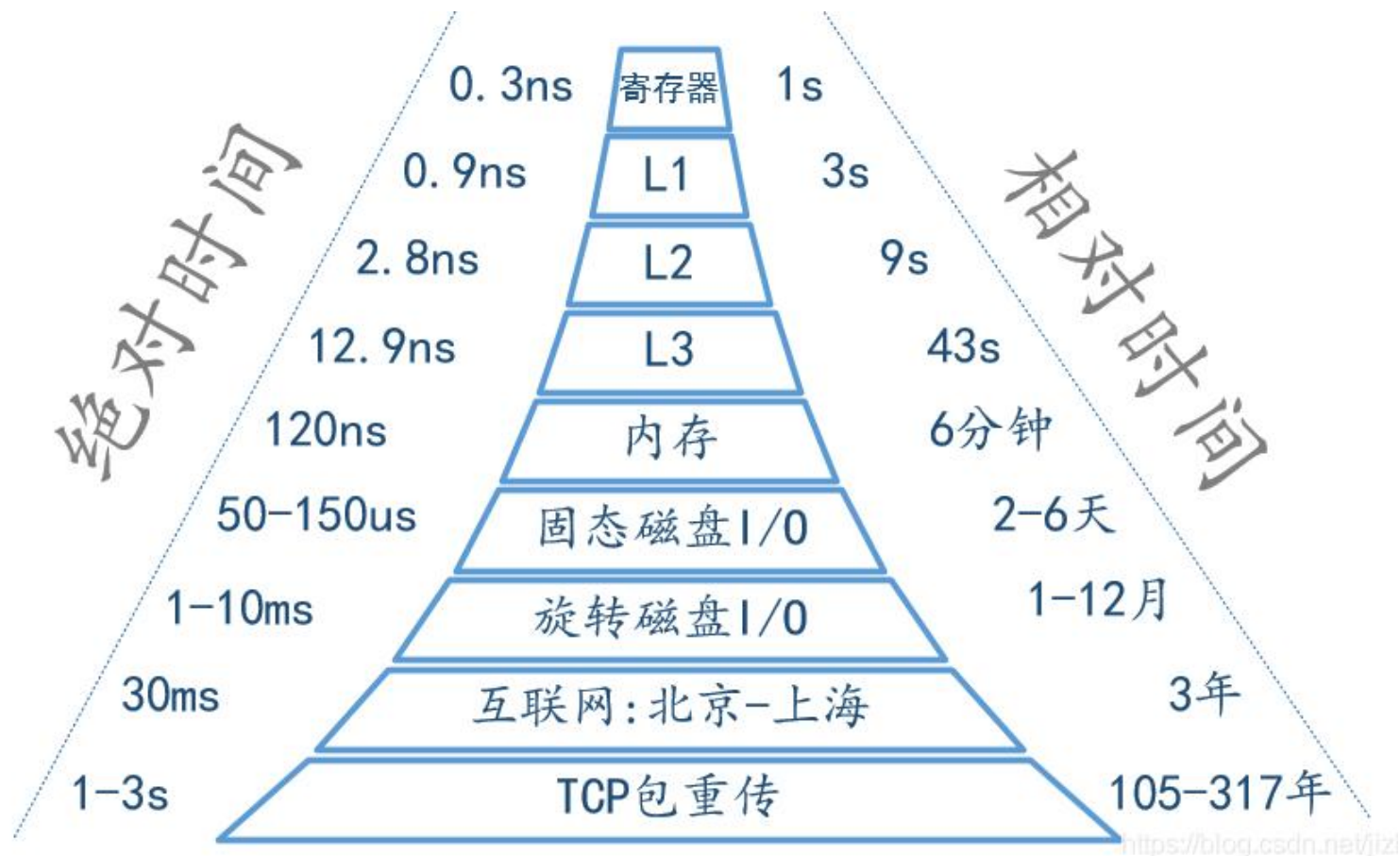
DMA: Direct Memory Access



计算机各种延时示意图

性能之巅:洞悉系统、企业与云计算

<https://blog.csdn.net/jizhu4873/article/details/84341884>



第六章 存储器

6.1 存储器概述

6.2 主存储器

6.3 高速缓冲存储器

6.4 虚拟存储

6.5 辅助存储器

辅助存储器的主要类型

硬盘



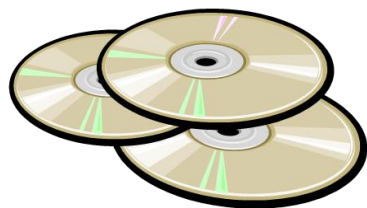
U盘...



软盘



光盘



磁带



辅助存储器——RAID

RAID (Redundant Array of Inexpensive Disks)

- UC Berkeley David A. **Patterson**, 1988
- 替代**SLEDs** (Single Large Expensive Disks)
- 磁盘访问速度过慢
- 可靠性
- 多磁盘管理不方便

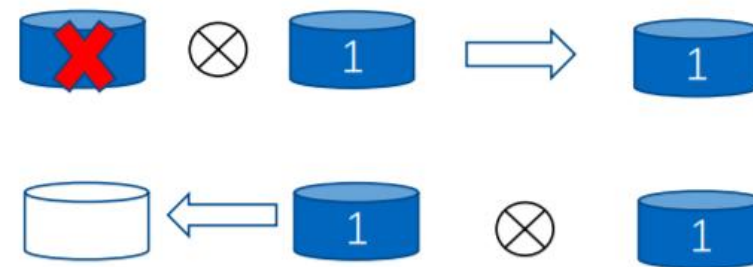
RAID (Redundant Array of Independent Disks)

辅助存储器——RAID的核心技术

- 将数据条带化后存放在不同磁盘上，通过多磁盘的**并行**操作提高磁盘系统的读写速率
- 使用基于**异或**运算为基础的校验恢复损坏的数据

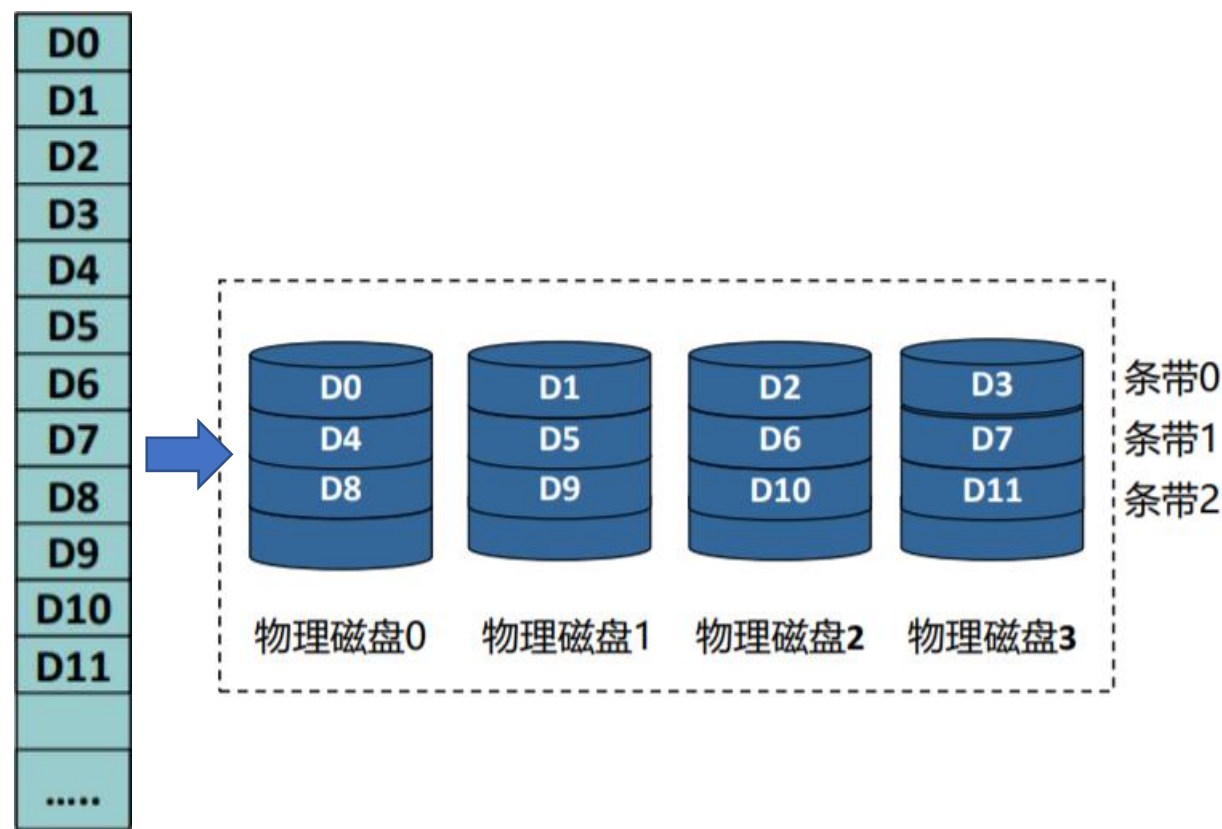
1	$\oplus$	0	=	1
1	$\oplus$	1	=	0
0	$\oplus$	1	=	1
0	$\oplus$	0	=	0

1	=	0	$\oplus$	1
1	=	1	$\oplus$	0
0	=	1	$\oplus$	1
0	=	0	$\oplus$	0



常见的RAID技术——RAID 0

数据以条带方式均匀分散在各磁盘
可用容量是最小容量硬盘的N倍，速度为最小速度的N倍。

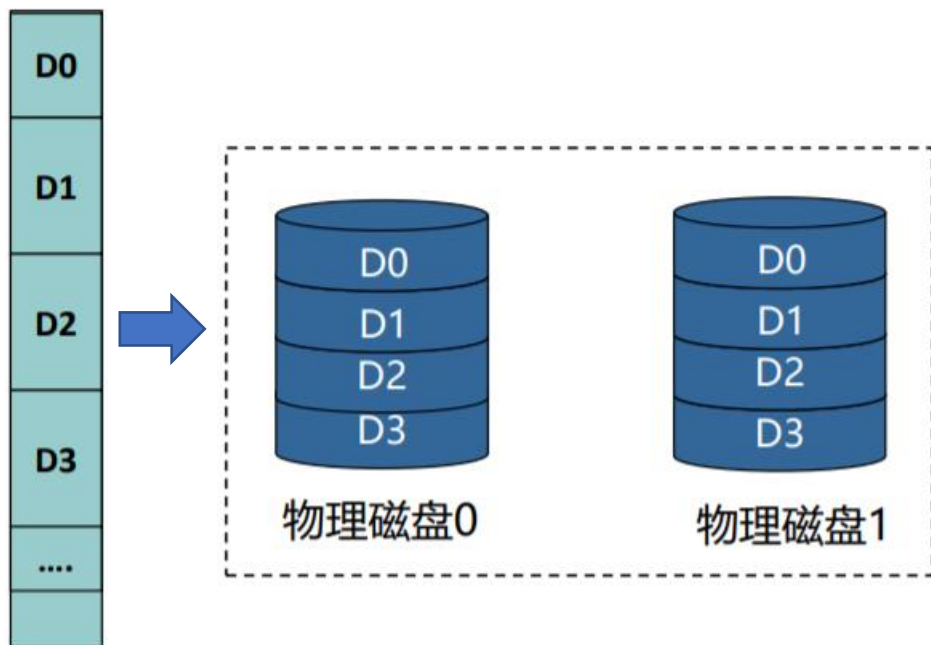


所需磁盘数	2个或更多
优点	磁盘读写效率高 无校验带来使用 and 配置方便
缺点	无冗余，数据安全性低
适用领域	视频、图像等需要高传输带宽的应用

常见的RAID技术——RAID 1

RAID 1 镜像磁盘阵列

数据采用镜像的冗余方式，同一数据有多份拷贝



所需磁盘数	至少2个
优点	100%数据冗余，数据安全性高 理论上可以实现2倍的读取效率
缺点	空间利用率只有50%
适用领域	财务、金融等高可用应用

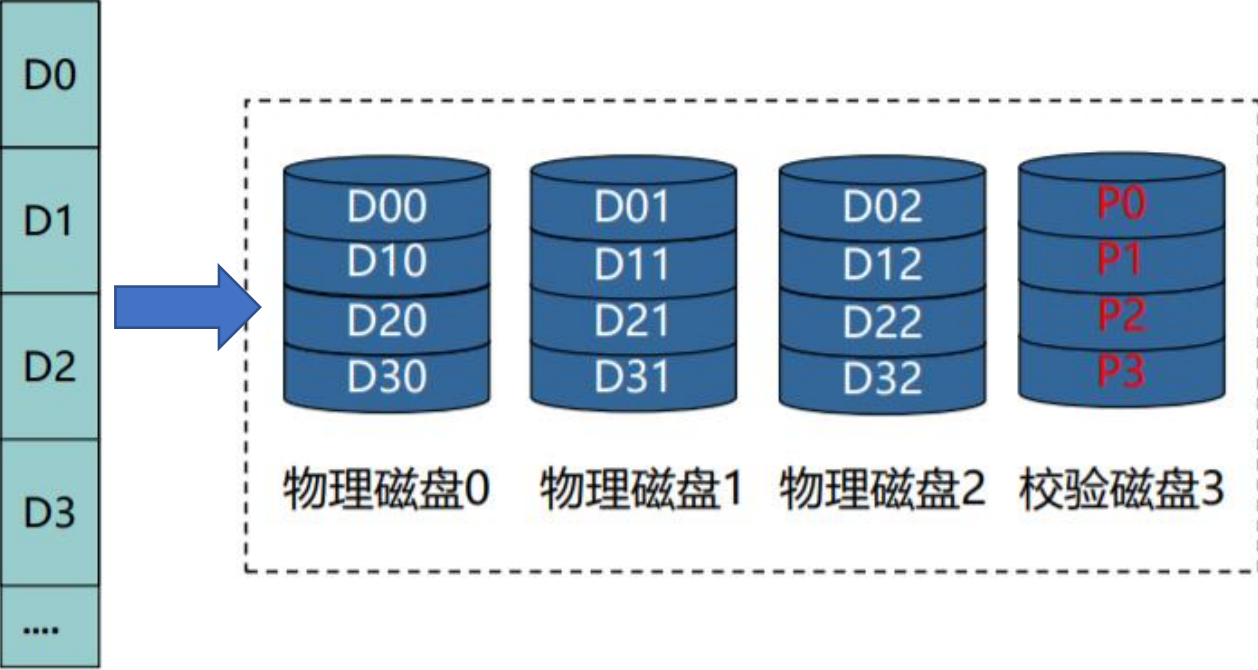
常见的RAID技术——RAID 2

- RAID 2，纠错**汉明码**磁盘阵列。
- 磁盘器组中的第1个、第2个、第4个.....第 2^n 个磁盘是专门的校验盘，用于校验和纠错，其余的用于存放数据。
- 使用的磁盘越多，校验盘的占比越小
- 对大数据量的输入输出有很高的性能，但少量数据的输入输出性能不好。
- RAID 2很少实际使用。

常见的RAID技术——RAID 3/4

奇校验或偶校验的磁盘阵列。

数据按 位/条带 并行传输到多个磁盘上，同时奇偶校验数据存放到**专用校验盘**上。

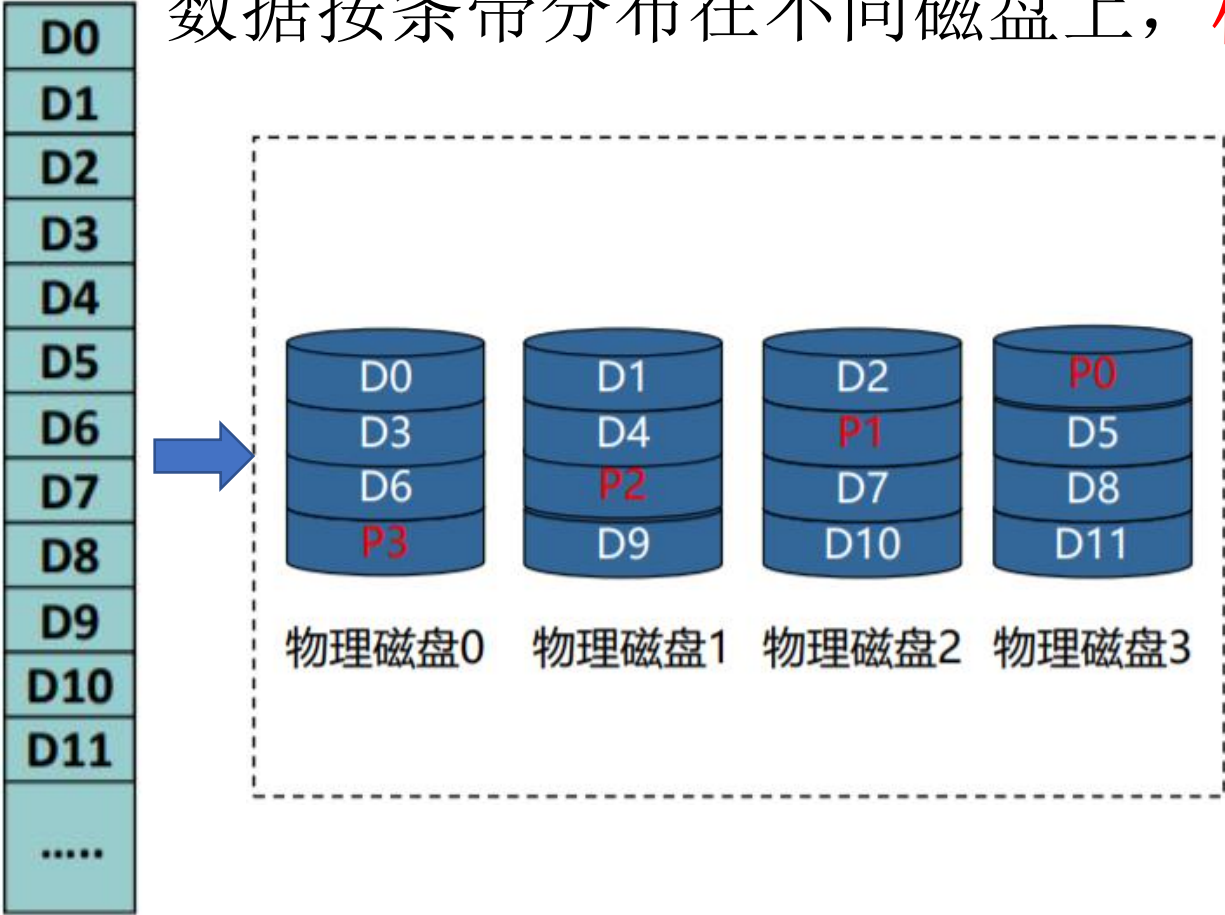


所需磁盘数	至少3个
优点	读写性能比较好 磁盘利用率高， $N-1/N$
缺点	控制器设计复杂 校验磁盘的写性能有瓶颈
适用领域	视频生成和图像、视频编辑 等需要高吞吐量的应用环境

常见的RAID技术——RAID 5

奇校验或偶校验的磁盘阵列

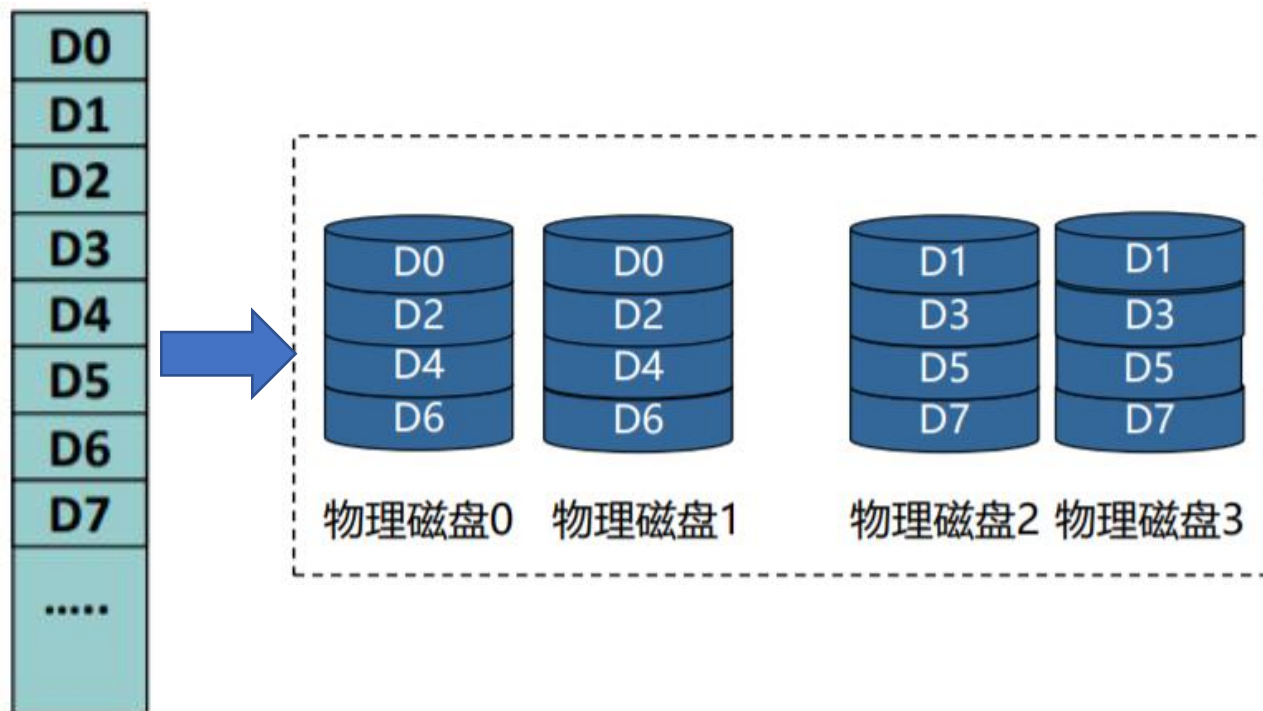
数据按条带分布在不同磁盘上，**校验信息被均匀分散到各磁盘上**



所需磁盘数	至少为3个
优点	读性能比较好 校验信息分布式存取，避免出现写操作的瓶颈
缺点	控制器设计复杂 磁盘重建的过程比较复杂
适用领域	FTP、Email、Web、数据库

常见的RAID技术——RAID 10

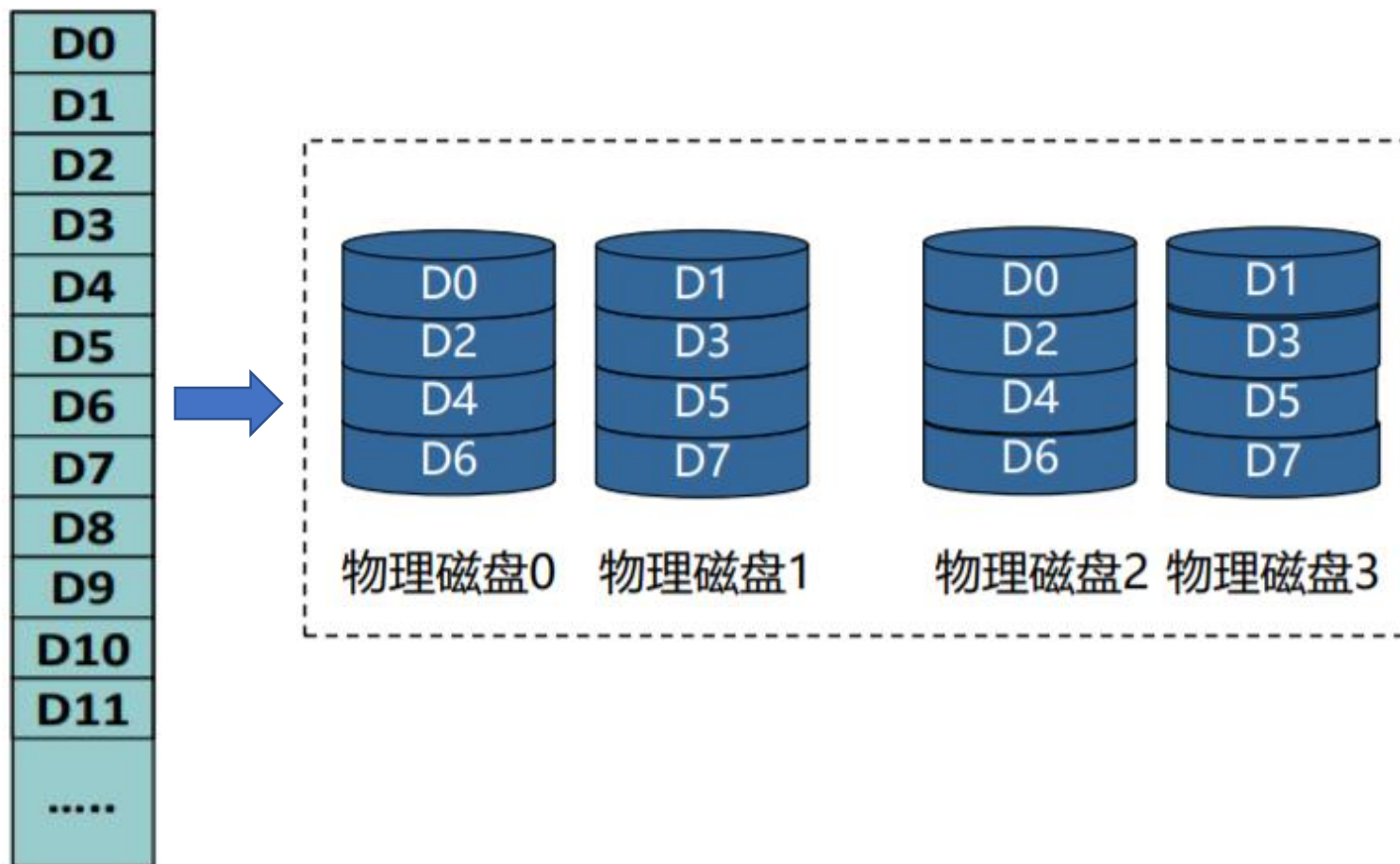
结合RAID 1和 RAID 0，先镜像，再条带化



所需磁盘数	最低为4个
优点	读性能好 数据安全性好，允许 最多 同时有 半数 磁盘失效
缺点	空间利用率也只有50%
适用领域	多用于要求高可用性和高安全性的应用场合

常见的RAID技术——RAID0 1

结合RAID 0和 RAID 1，先条带化，再镜像



辅助存储器——RAID的实现方式

- 软件RAID

依赖于CPU完成，没有第三方的控制器和I/O芯片

- 硬件RAID

专用RAID控制器和I/O芯片处理RAID任务，不占CPU

RAID技术小结

RAID级别	RAID0	RAID1	RAID3	RAID5	RAID1 0
容错性	无	有	有	有	有
冗余类型	无	镜像	奇偶校验	奇偶校验	镜像
备盘	无	有	有	有	有
读性能	高	低	高	高	中间
随机写性能	高	低	最低	低	中间
连续写性能	高	低	低	低	中间
磁盘数	≥ 2	$2N (N \geq 1)$	≥ 3	≥ 3	$2N (N \geq 2)$
可用容量	100%	50%	$(N-1) / N$	$(N-1) / N$	50%

存储器——小结

- 了解各类ROM、SRAM、DRAM的工作原理。
- 理解DRAM和SRAM的异同点
- 熟悉CPU与存储器之间的连接。
- 熟悉原始数据和汉明码之间的相互转换。
- 了解单体多字系统、高位交叉编址多体存储；
- 理解低位交叉编址多体存储的性能提升（例4.6）
- 理解Cache命中率、Cache的读写操作（写直达vs写回）
- 熟悉Cache-主存地址映射。
- 初步了解虚拟存储、RAID。